

Verification Security Application Standard

5.0.0 Version



2025 May

فهرست

7	صفحه اصلی
7	درباره این استاندارد
7	کپی‌رایت و مجوز
7	سرپرستان پروژه
7	گروه کاری
7	سایر هم‌کاران اصلی
8	مشاورکت کنندگان و بازبینی‌های دی‌گر
8	پیش‌گفتار
8	مقدمه
8	اصول کلی دی‌نسخه ۵.۰
9	چشم‌انداز آینده
9	ASVS چیست؟
9	دامنه ASVS
10	Application
10	امنیت
10	تأیید
10	استاندارد
11	الزام (Requirement)
11	تصمیمات امنیتی مستند شده
12	سطوح تأیید امنیتی برنامه‌های کاربردی
12	ارزیابی سطوح
13	سطح ۱
13	سطح ۲
13	سطح ۳
14	کدام سطح را انتخاب کنیم
14	چگونه از ASVS استفاده کنیم
14	ساختار ASVS
14	strategy Release
15	انعطاف‌پذیری در استفاده از ASVS
15	نحوه ارجاع به الزامات ASVS
16	فورک کردن ASVS (Forking)
16	موارد استفاده از ASVS
16	به‌عنوان راهنمای دقیق معماري امنیتی
16	به‌عنوان مرجع تشخیص‌های امنیتی
16	به‌عنوان راهنمای آزمون‌های واحد وی‌کی‌پارچه‌سازی خودکار
17	برای آموزش توسعه امن
17	به‌عنوان چارچوبی برای راهنمای درخرد نرم‌افزار امن
17	کاربرد عملی ASVS

17	ارزیابی و ص دور گ واهی
17	دی دگاه OWASP در مورد گ واهی نام هه و نشانه های اع تم اد ASVS
18	نحوه تأی د انطباق با ASVS
18	گ زارش گیری ارزیابی
18	دامنه ارزیابی
18	روش های ارزیابی
20	تغی رات نس بت به نس خه 4.x
20	م قدمه
20	فلسفه الزامات
20	دامنه و تمرکز
20	تمرکز بر اهداف امنیتی به جای سازوکارها
20	تصمیمات امنیتی مستند
21	تغی رات ساختاری و فصل های جدید
22	حذف نگاشت مستقی م با استانداردهای دی گر
22	کاهش وابستگی به دست ورال عمل های هوی ت دی جی تال NIST
22	فاصله گرفتن از نگاشت CWE
22	بازنگری در تعریف سطوح (Levels)
22	ورود ساده تر به سطح اول
23	افسانه قابلیت آزمون پذیری
23	فقط ریسک مطرح نیست
23	Sanitization and Encoding V1
23	هدف
24	V1.1 ساختار Sanitization and Encoding
24	V1.2 پی ش گیری از تزریق
26	V1.3 پاک سازی (Sanitization)
27	V1.4 ح افظه، رشته و کد بدون مدیری ت (Code) Unmanaged and String, (Memory,
28	V1.5 سریال زدای ای من Safe (Deserialization)
29	References
29	Logic Business and Validation V2
29	هدف
30	V2.1 مستندات اع تبار سنجی و منطق کسب و کار (Documentation) Logic Business and Validation
30	V2.2 اع تبار سنجی ورودی (Input Validation)
31	V2.3 امنیتی منطق کسب و کار (Business Logic Security)
32	V2.4 مقابله با خود کار سازی (Anti-automation)
33	References
33	V3 امنیتی رابط کاربری وب (Web Security) Frontend
33	هدف کنترلی
33	V3.1 مستند سازی امنیتی رابط کاربری وب (Documentation) Security Frontend Web

34	 Interpretation) Content (Unintended محتواه	V3.2
34	 Setup) (Cookie کی کوکی	V3.3
35	 Headers) Mechanism Security (Browser مرورگر	V3.4
37	 Separation) Origin (Browser مرورگر	V3.5
38	 Integrity) Resource (External خارج	V3.6
39	 Considerations) Security Browser (Other مرورگر	V3.7
39	 References	
40	Service Web and API V4	
40	 هدف	
40	 Security Service Web Generic	V4.1
41	 Validation Structure Message HTTP	V4.2
42	 GraphQL	V4.3
42	 WebSocket	V4.4
43	 References	
43	Handling File V5	
43	 هدف	
44	 Documentation Handling File	V5.1
44	 Content and Upload File	V5.2
45	 Storage File	V5.3
45	 Download File	V5.4
46	 References	
46	Authentication V6	
46	 Objective Control	
47	 Documentation Authentication	V6.1
47	 Security Password	V6.2
48	 Security Authentication General	V6.3
50	 Recovery and Lifecycle Factor Authentication	V6.4
51	 requirements authentication Multi-factor General	V6.5
52	 mechanisms authentication Out-of-Band	V6.6
53	 mechanism authentication Cryptographic	V6.7
54	 Provider Identity an with Authentication	V6.8
55	 References	
55	Management Session V7	
55	 هدف	
56	 Documentation Management Session	V7.1
57	 Security Management Session Fundamental	V7.2
57	 Timeout Session	V7.3
58	 Termination Session	V7.4

59	Abuse Session Against Defenses V7.5
59	Re-authentication Federated V7.6
60	References
60	Authorization V8
60	هدف
60	Documentation Authorization V8.1
61	Design Authorization General V8.2
62	Authorization Level Operation V8.3
62	Considerations Authorization Other V8.4
63	References
63	Tokens Self-contained V9
63	هدف
63	integrity and source Token V9.1
64	content Token V9.2
65	References
65	OIDC and OAuth V10
65	هدف
67	Security OIDC and OAuth Generic V10.1
68	Client OAuth V10.2
69	Server Resource OAuth V10.3
70	Server Authorization OAuth V10.4
72	Client OIDC V10.5
73	Provider OpenID V10.6
73	Management Consent V10.7
74	References
75	Cryptography V11
75	هدف
75	Documentation and Inventory Cryptographic V11.1
76	Implementation Cryptography Secure V11.2
77	Algorithms Encryption V11.3
78	Functions Hash-based and Hashing V11.4
79	Values Random V11.5
79	Cryptography Key Public V11.6
80	Cryptography Data In-Use V11.7
80	References
81	Communication Secure V12
81	هدف
81	Guidance Security TLS General V12.1

82	Services Facing External with Communication HTTPS V12.2
82	Security Communication Service to Service General V12.3
83	References
83	Configuration V13
83	هدف
83	Documentation Configuration V13.1
84	Configuration Communication Backend V13.2
85	Management Secret V13.3
86	Leakage Information Unintended V13.4
87	References
87	Protection Data V14
87	هدف
87	Documentation Protection Data V14.1
88	Protection Data General V14.2
89	Protection Data Client-side V14.3
90	References
90	Architecture and Coding Secure V15
90	Objective Control
90	Documentation Architecture and Coding Secure V15.1
91	Dependencies and Architecture Security V15.2
92	Coding Defensive V15.3
93	Concurrency Safe V15.4
94	References
94	Handling Error and Logging Security V16
94	هدف
95	Documentation Logging Security V16.1
95	Logging General V16.2
96	Events Security V16.3
97	Protection Log V16.4
97	Handling Error V16.5
98	References
98	WebRTC V17
98	هدف
99	Server TURN V17.1
100	Media V17.2
101	Signaling V17.3
102	References

102	Glossary A: Appendix
108	References B: Appendix
109	Projects Core OWASP
109	project Series Sheet Cheat OWASP
109	Projects Related Security Mobile
109	projects related Things of Internet OWASP
109	projects Serverless OWASP
109	Others
110	Standards Cryptography C: Appendix
110	Documentation and Inventory Cryptographic
110	Parameters Cryptographic of Strengths Equivalent
111	Values Random
112	Algorithms Cipher
113	Modes Cipher AES
114	Wrapping Key
114	Encryption Authenticated
115	Functions Hash
115	Cases Use General for Functions Hash
116	Storage Password for Functions Hash
117	(KDFs) Functions Derivation Key
117	Functions Derivation Key General
117	Functions Derivation Key Password-based
118	Mechanisms Exchange Key
118	Schemes KEX
118	groups Diffie-Hellman
119	(MAC) Codes Authentication Message
120	Signatures Digital
120	Standards Encryption Post-Quantum
121	Recommendations D: Appendix
121	Introduction
121	mechanisms in-scope Recommended,
121	principles Security Software
122	processes Security Software
123	Contributors - E Appendix

صفحه اصلی

درباره این استاندارد

استاندارد تأیید امنیتی نرم افزار (ASVS) فهرستی از الزامات امنیتی برای نرم افزار است که معماران، توسعه دهندگان، آزمونگران، متخصصان امنیت، فروشندگان ابزار و مصرف کنندگان میتوانند از آن برای تعریف، ساخت، آزمون و تأیید امنیتی برنامه های نرم افزاری استفاده کنند.

کپی رایت و مجوز

2025 May 5.0.0, Version



شکل 1: license

Foundation. OWASP The 2008-2025 © Copyright

این سند تحت مجوز Creative Commons Attribution-ShareAlike 4.0 International License منتشر شده است.

در هرگونه بازنشر یا توزیع، باید شرایط این مجوز را به روشنی به دیگران اعلام نماید.

سرپرستان پروژه

Elar Lang Josh C Grossman
Jim Manico Daniel Cuthbert

گروه کاری

Ahnoff Tobias	Andalis Ralph	Armstrong Ryan	Corona Gabriel
Jacquot Meghan	Prutchi Shanni	Sharafaldin Iman	Yardeni Eden

سایر همکاران اصلی

Sjoerd Langkemper Isaac Lewis

Mark Carney Sandro Gauci

مشارکت کنندگان و بازبینی‌های دی‌گر

فهرست ساینده‌های مشارکت کنندگان در ضمیمه E درج شده است.

در صورتی که نام فردی در فهرست اعتبارهای نسخه x.5 ذکر نشده باشد، لطفاً با ثبت‌ت‌یکت در گیتهاب، نسبت به شناساندن ایشان در به‌روزرسانی‌های آتی نسخه x.5 اقدام فرمایید.

استاندارد ASVS برپایه‌ی تلاش‌های افراد درگیر در نسخه‌های ۱.۰ (۲۰۰۸) تا ۴.۰ (۲۰۱۹) بنا شده است. بسیاری از ساختارها و اقلام تأییدی که امروزه در ASVS باقی مانده‌اند، در ابتدا توسط Andrew van der Williams، Jeff Boberski، Mike Stock، و Dave Wichers نوشته شده‌اند، در کنار بسیاری دی‌گرازمشارکت‌کنندگان. از همه کسانی که در گذشته مشارکت داشته‌اند صمیمانه سپاس‌گزاری می‌م. برای مشاهده فهرست کامل مشارکت کنندگان پیشین، لطفاً به هر نسخه‌ی قبلی مراجعه نمائید.

پیش‌گفتار

به نسخه ۵.۰ استاندارد تأییدی امنیتی نرم‌افزار (ASVS) خوش آمدید.

مقدمه

استاندارد ASVS که در سال ۲۰۰۸ و از طریق همکاری یک جامعه جهانی راه‌اندازی شد، ASVS مجموعه‌ای جامع از الزامات امنیتی برای طراحی، توسعه و آزمون برنامه‌ها و خدمات وب مدرن تعریف می‌کند.

پس از انتشار نسخه ۴.۰ در سال ۲۰۱۹ و به‌روزرسانی جزئی آن (نسخه ۴.۰.۳) در سال ۲۰۲۱، نسخه ۵.۰ یک نقطه‌ی عطف مهم به شمار می‌آید—این نسخه برای هم‌راستایی با آخرین پیشرفت‌ها در حوزه‌ی امنیتی نرم‌افزار، به‌روزرسانی و مدرن‌سازی شده است.

نسخه ۵.۰ نتایج‌ی مشارکت گسترده سرپرستان پروژه، اعضای گروه کاری و جامعه بزرگ OWASP در راستای به‌روزرسانی و ارتقای این استاندارد مهم است.

اصول کلی دی‌نسخه ۵.۰

این بازنگری عمده با چند اصل کلی طراحی و توسعه یافته است:

- حوزه و تمرکززدی‌وقت: این نسخه از استاندارد به‌گونه‌ای طراحی شده تا هم‌راستا با ارکان اصلی موجود در نام آن باشد: برنامه کاربردی، امنیت، تأیید و استانداردها. الزامات به‌گونه‌ای بازنویسی شده‌اند که پی‌گیری از نقص‌های امنیتی را در اولویت قرار دهند، نه الزام به پیاده‌سازی‌های فنی خاص. متن الزامات به‌صورت خودتوضیحی نگاشته شده‌اند تا چرایی وجود هر مورد را روشن کنند.

- پشتیبانی از تصمیمات مستند امنیتی: نسخه ۵.۰ الزامات جدیدی برای مستندسازی تصمیمات کلیدی امنیتی معرفی کرده است. این اقدام موجب افزایش قابلیت ردیابی شده و پیاده‌سازی‌های متناسب با زمینه (context) را پشتیبانی می‌کند، به سازمان‌ها اجازه می‌دهد تا موضوع امنیتی خود را با نیازها و ریسک‌های خاص خود تطبیق دهند.
- سطوح به‌روزشده: درحالی‌که مدل سه سطحی ASVS حفظ شده است، تعاریف سطوح بازنطراحی شده‌اند تا پذیری آن ساده‌تر گردد. سطح ۱ به عنوان گام آغازین برای پذیری ASVS طراحی شده و لایه‌ی نخست دفاعی را فراهم می‌آورد. سطح ۲ نمایانگر ردیابی جامع از رویه‌های استاندارد امنیتی است. سطح ۳ به نیازه‌ای پیشرفته و با تضمین بالا می‌پردازد.
- محتوای بازآرایی‌شده و گسترش یافته: نسخه ۵.۰ شامل حدود ۳۵۰ الزام در قالب ۱۷ فصل است. فصل‌ها به‌منظور وضوح بیشتر و سهولت استفاده، بازسازمانده‌ی شده‌اند. نگاشته‌ی دوسویه بین نسخه‌های ۴.۰ و ۵.۰ نیز ارائه شده تا فرآیند مهاجرت را تسهیل نماید.

چشم‌انداز آینده

همان‌طور که تأمین امنیت یک نرم‌افزار هرگز نقطه‌ی پایان ندارد، ASVS نیز به‌عنوان یکی که استاندارد همواره در حال توسعه است. گنجینه نسخه ۵.۰ یکی که انتشار عمده به‌شمار می‌آید، اما روند توسعه ادامه خواهد داشت. این نسخه، هم‌زمان با فراهم‌سازی دسترسی جامع‌تر به به‌بوده‌ها و افزودن‌های صورت‌گرفته، بستری برای ارتقاءهای آینده نیز فراهم می‌کند. از جمله این موارد، می‌توان به تلاش‌های جامع‌محور برای ایجاد راه‌نمای پیاده‌سازی و تأیید برپایه‌ی مجموعه الزامات اصلی اشاره کرد.

ASVS 5.0 به‌منظور فراهم‌سازی یک پایه‌ی قابلیت‌ماد برای توسعه‌ی نرم‌افزار امن طراحی شده است. ما از جامعه دعوت می‌کنیم تا این استاندارد را بپذیرند، در آن مشارکت کنند و برپایه‌ی آن، اقدامات پیش‌تری انجام دهند؛ تا به‌صورت جمعی، وضعیت امنیتی برنامه‌های نرم‌افزاری را ارتقاء دهیم.

ASVS چیست؟

استاندارد تأیید امنیتی نرم‌افزار (ASVS) مجموعه‌ای از الزامات امنیتی برای برنامه‌ها و خدمات وب تعریف می‌کند و منبعی ارزشمند برای تمامی افرادی است که قصد دارند برنامه‌های امن طراحی، توسعه یا نگهداری کنند یا امنیت آن‌ها را ارزیابی نمایند.

این فصل به جنبه‌های اساسی استفاده از ASVS، از جمله دامنه کاربرد، ساختار سطوح اولویت‌بندی‌شده، و موارد اصلی استفاده از این استاندارد می‌پردازد.

دامنه ASVS

دامنه استاندارد ASVS از نام آن الهام گرفته شده است: برنامه کاربردی (Application)، امنیت (Security)، تأیید (Verification) و استاندارد (Standard). این دامنه مشخص می‌کند که چه الزامی گنجانده می‌شود. مستثنی شده است و هدف اصلی آن، شناسایی اصول امنیتی‌ای است که باید محقق شوند. دامنه همچنین الزامات مستندسازی را نیز در بر می‌گیرد که پایه‌ای برای الزامات پیاده‌سازی محسوب می‌شود.

برای مهاجمان، چیزی به نام «دامنه» معنا ندارد؛ بنابراین الزامات ASVS باید در کنار راهنامه‌ای برای ساری جنبه‌های چرخه عمر نرم‌افزار، مانند فرآیندهای CI/CD، میزبانی و عملیات اجرایی مورد ارزیابی قرار گیرد.

Application

ASVS، «برنامه» را به عنوان محصول نرم‌افزاری در حال توسعه تعریف می‌کند که کنترل‌های امنیتی باید درون آن گنجانده شود. ASVS فعالیتهای چرخه عمر توسعه یا نحوه ساخت برنامه از طریق فرآیند CI/CD را تجویز نمی‌کند، بلکه بر نتایج امنیتی مورد انتظار درون خود محصول تمرکز دارد.

اجزای مانده balancer WAF، load یا پراکسی‌ها که ترافیکی که HTTP را پردازش، اصلاح یا اعتراف‌ساز می‌کنند، ممکن است به منظور خاصی بخشی از برنامه تلقی شوند، زیرا برخی کنترل‌های امنیتی مستقیماً به آنها وابسته هستند یا از طریق آنها پیاده‌سازی می‌شوند. این اجزا باید در الزاماتی که مرتبط با Response، Rate Cache، Limite یا کنترل ارتباطات ورودی و خروجی بر اساس مبدا و مقصد در نظر گرفته شوند.

در مقابل، ASVS به طور کلی الزاماتی را که مستقیماً مرتبط با برنامه نیستند یا پی‌کربندی آنها خارج از حوزه مسئولیت برنامه است، مستثنی می‌کند. به عنوان مثال، مسائلی مرتبط با DNS معمولاً توسط تیم یا واحدی مجزا مدیریت می‌شود.

به طور مشابه، اگرچه برنامه مسئول نحوه دریافت ورودی و تولید خروجی است، ولی اگر یک فرآیند خارجی با برنامه یا داده‌های آن تعامل داشته باشد، خارج از دامنه ASVS محسوب می‌شود. به عنوان نمونه، پشتیبان‌گیری از برنامه یا داده‌های آن، عموماً برعهده‌ی یک فرآیند خارجی است و در کنترل برنامه یا توسعه‌دهندگان آن قرار ندارد.

امنیت

هر الزام باید تأثیری قابل اندازه‌گیری بر امنیت داشته باشد. عدم وجودی که الزام باید منجر به ناامن‌تر شدن برنامه شود و پیاده‌سازی آن الزام باید احتمال وقوع یا تأثیری که ریسک امنیتی را کاهش دهد.

سایر ملایمات مانند جنبه‌های عمل‌کردی، سبک کدنویسی یا الزامات سیستمی خارج از دامنه ASVS هستند.

تأیید

هر الزام باید قابل تأیید باشد و نتیجه تأیید باید منجر به تصمیم «موفق» یا «ناموفق» گردد.

استاندارد

ASVS مجموعه‌ای از الزامات امنیتی است که برای انطباق با استاندارد باید پیاده‌سازی شوند. به همین دلیل، الزامات به تعریف هدف امنیتی‌ای محدود می‌شوند که باید محقق گردد. اطلاعات جانبی

می‌تواند بر پایه ASVS ساخته شود یا از طریق نگاشت به آن متصل گردد.

به طور خاص، OWASP دارای پروژه‌های متعددی است و ASVS به صورت آگاهانه از تکرار محتوای سایر پروژه‌ها اجتناب کرده است. به عنوان مثال، اگر توسعه دهنده‌ای بپرسد “چگونه باید فلان الزام را در فناوری خاص خود پیاده‌سازی کنم”، این سؤال در حوزه پروژه Series Sheet Cheat قرار می‌گیرد. اگر ارزیاب بپرسد “چگونه باید این الزام را در این محیط آزمون کنم”، پاسخ آن در پروژه Security Web Guide Testing یافت می‌شود.

در حالی که ASVS فقط برای استفاده متخصصان امنیتی طراحی نشده، انتظار می‌رود که خواننده دانش فنی کافی برای درک محتوای توانایی جستجو و درباره مفاهیم خاص را داشته باشد.

الزام (Requirement)

واژه “الزام” در ASVS دقیقاً برای توصیف آنچه باید برای تأمین امنیتی انجام شود به کار می‌رود. ASVS فقط شامل الزامات (must) است و توصیه‌ها (should) را به عنوان شرط اصلی شامل نمی‌شود.

به عبارت دیگر، توصیه‌هایی که ممکن است فقط یکی از راه‌های ممکن یا ملحظات سبک کنونی باشند، در تعریف “الزام” نمی‌گنجانند.

الزامات ASVS برای پاسخ‌گویی به اصول امنیتی خاص طراحی شده‌اند، بدون آنکه به روش پیاده‌سازی یا فناوری خاصی وابسته باشند، و در عین حال به گونه‌ای نوشته شده‌اند که خود به خود دلیلی وجودشان را توضیح دهند. این بدین معناست که الزامات مبتنی بر روش تأیید یا پیاده‌سازی خاص نیستند.

تصمیمات امنیتی مستند شده

در امنیت نرم افزار، طراحی و مشخص کردن سازوکارهای امنیتی به موقع منجر به پیاده‌سازی امن‌تر و قابلیت اطمینان در محصولات نه‌ای می‌شود.

علاوه بر این، برخی الزامات به گونه‌ای هستند که پیاده‌سازی آنها بسویار خاص و وابسته به نیازهای هر برنامه است. برای مثال، کنترل مجوزها، اعتبارسنجی ورودی‌ها و کنترل‌های محافظتی بر روی سطوح مختلف داده‌های حساس.

برای حل این موضوع، به جای بی‌انتهایی کلی مثل “تمام داده‌ها باید رمزگذاری شوند” یا تلاش برای پوشش همه موارد ممکن در قالب یک الزام، الزامات مستندسازی در نظر گرفته شده‌اند که توسعه‌دهنده را ملزم می‌سازند نحوه پیاده‌سازی و پی‌کربندی این نوع کنترل‌ها را به صورت مستند ارائه دهد. این مستندات سپس می‌توانند از نظر تناسب مورد بازبینی قرار گرفته و پیاده‌سازی واقعی با مستندات مقایسه شود تا میزان انطباق بررسی گردد.

این الزامات برای ثبت تصمیمات طراحی شده‌اند که سازمان توسعه‌دهنده درباره نحوه پیاده‌سازی الزامات امنیتی اتخاذ کرده است.

الزامات مستندسازی همواره در بخش اول هر فصل قرار دارند (هرچند همه فصل‌ها چنین الزامی ندارند) و همواره با یک الزام پیاده‌سازی مرتبط هستند، جایی که تصمیمات مستندسازی شده باید عملاً پیاده‌سازی شوند. نکته اینجاست که بررسی مستندات و بررسی پیاده‌سازی واقعی، دو فعالیت مستقل هستند.

دو عامل اصلی برای گنجاندن این الزامات وجود دارد. اولی الزام این است که بسبب الزامات امنیتی شامل تعریف قوانین خاصی هستند. مانند انواع فایلهای که مجاز به آپلود هستند، کنترلرهای کسب و کاری مورد نیاز، یا کاراکترهای مجاز برای یک فایل خاص. این قوانین در هر برنامه متفاوت است، بنابراین ASVS نمیتواند به صورت امری آنها را تعریف کند، وحتی Cheat sheet یا پاسخ مفصل هم در این موارد کم کم نخواهد کرد. بدون مستند بودن این تصمیمات، تأیید پیاده سازی آنها هم ممکن نخواهد بود.

دومین عامل این است که توسعه دهندگان نرم افزار در انتخاب برخی الزامات آزادی عمل داشته باشند. برای مثال، در نسخه های پیشین، ASVS قواعد مربوط به پایان جلسه (session timeout) بسبب امری بودند. ولی بسبب الزامات از برنامه ها، به ویژه برنامه های مشتری محور، قواعدی آسانگری را ترجیح میدهند و از کنترلرهای جایگزین استفاده می کنند. به همین دلیل، الزامات مربوط به مستندسازی به صورت صریح اجزای چنن انعطافی را فراهم می کنند.

بدیهی است که انتظار نمی رود توسعه دهندگان به صورت فردی این تصمیمات را بگیرند و مستند کنند، بلکه این وظیفه ای سازمان است که تصمیم گیری کرده، آنها را مستند کرده و به توسعه دهندگان منتقل کند تا در کد نویسی لحاظ شوند.

در توسعه نرم افزار، همانطور که برای طراحی ویژگی ها و ظاهر برنامه چارچوب ها، راهنما و مؤلفه های مشخصی وجود دارد و انتظار می رود که توسعه دهندگان از آنها پیروی کنند، در مورد امنیت هم باید همین رویکرد وجود داشته باشد. یعنی امنیت نباید چیزی باشد که هر توسعه دهنده بنا به سلیقه خود تصمیم بگیرد، بلکه باید از اسناد داردها، ابزارها و طراحی های مشخصی تبعیت نماید.

همچنین، نحوه مستندسازی نیز انعطاف پذیر است. تصمیمات امنیتی میتواند در قالب یک سند رسمی نوشته شده باشند که توسعه دهندگان به آن مراجعه می کنند، یا این تصمیمات می توانند به صورت کد در یک کتابخانه مشتری پیاده سازی شده باشند که استفاده از آن برای همه توسعه دهندگان الزامی است. در هر دو حالت، هدف نهایی محقق می شود.

سطوح تأیید امنیتی برنامه های کاربردی

ASVS سه سطح تأیید امنیتی تعریف می کند که با افزایش سطح، عمق و پیچیدگی الزامات امنیتی نیز افزایش می یابد. هدف کلی این است که سازمانها از سطح اول شروع کرده و مهمترین دغدغه های امنیتی را برطرف کنند، و سپس متناسب با نیازهای سازمان و برنامه، به سطوح بالاتر حرکت کنند. در اسناد و متن الزامات، این سطوح با نمادهای L1، L2 و L3 نمایش داده می شوند.

هر سطح از ASVS بیانگر الزامات امنیتی ای است که باید در آن سطح محقق شود؛ الزامات سطوح بالاتر به عنوان توصیه در نظر گرفته می شوند.

به منظور جلوگیری از تکرار الزامات یا گنجاندن الزامات نامربوط در سطوح بالاتر، برخی الزامات فقط در یک سطح مشخص اعمال می شوند اما در سطوح بالاتر شرایط سخت گیری را ندارند.

ارزیابی سطوح

سطوح ASVS بر پایه ی اولویت بندی الزامات و براساس تجربه ی پیاده سازی و آزمون آنها تعریف شده اند. تمرکز اصلی بر مقایسه ی کاهش ریسک در برابر هزینه ی پیاده سازی الزام است. عامل کلیدی دیگر، سهولت در ورود و پذیرش اسناد است.

کاهش ریسک به میزبان کاهش ریسک امنیتی در برنامه توجه دارد و عوامل سنتی مانند محرمانگی (Confidentiality)، یکپارچگی (Integrity)، دسترس پذیری و (Availability) همچنین لایه دفاع اولیه یا دفاع در عمق بودن الزام را در نظر می‌گیرد.

بحث‌های دقیقی درباره معیارها و تصمیم‌گیری‌های سطح‌بندی منجر به توزیع شده که در بیش‌تر موارد قابل اعتماد است، هرچند ممکن است برای همه موارد ۱۰۰٪ مناسب نباشد. در نتیجه، ممکن است سازمان‌ها تصمیم‌گیری‌های برخی الزامات سطوح بالاتر را زودتر و بر اساس تحلیل ریسک خود در اولویت قرار دهند.

انواع الزامات در هر سطح را می‌توان به صورت زیر توصیف کرد.

سطح ۱

این سطح حداقل الزامات برای ایمن‌سازی برنامه را شامل می‌شود و نقطه شروعی حیاتی محسوب می‌شود. حدود ۲۰٪ از کل الزامات ASVS در این سطح قرار دارد. هدف این سطح، داشتن کمترین تعداد الزامات برای سهولت در ورود و پذیرش استاندارد است.

این الزامات معمولاً حیاتی یا پایه‌های هستند و اولین خط دفاعی در برابر حملات رایج محسوب می‌شوند، حملاتی که برای بهره‌برداری از آنها به آسیب پذیری یا پیش‌شرطی گریزی نیست.

همچنین برخی الزامات مثل موارد مربوط به گرواژه‌ها در سطح ۱ اهمیت بیش‌تری دارند، زیرا در سطوح بالاتر استفاده از احراز هویت چند مرحله‌ای مطرح می‌شود.

سطح ۱ الزاماً از طریق آزمون نفوذ خرابی و بدون دسترسی به کدی استندارت (مانند آزمون چعبه-سیاه) قابل ارزیابی نیست، ولی تعداد کم‌تر الزامات آن فرآیند تأیید را ساده‌تر می‌کند.

سطح ۲

بیش‌تر برنامه‌ها باید به دنبال دستیابی به این سطح از امنیت باشند. حدود ۵۰٪ از الزامات ASVS در سطح ۲ هستند، و بنابراین یک برنامه باید حدود ۷۰٪ از الزامات ASVS (یعنی تمام الزامات L1 و L2) را برای انطباق با این سطح پیاده‌سازی کند.

این الزامات معمولاً به حملات کم‌تر رایج یا دفاع‌های پیچیده‌تر در برابر حملات رایج مرتبط هستند. ممکن است این الزامات همچنان دفاع اولیه باشند، یا برای بهره‌برداری نیازی به پیش-شرط‌های داشته باشند.

سطح ۳

این سطح باید هدف برنامه‌های باشد که می‌خواهند بالاترین سطح امنیت را اثبات کنند و شامل حدود ۳۰٪ نهایی الزامات است.

الزامات این سطح عمدتاً شامل مکانیسم‌های دفاع در عمق یا کنترل‌های پیچیده و دشوار برای پیاده‌سازی هستند.

کدام سطح را انتخاب کنیم

سطوح اولویت‌بندی شده ASVS برای نشان دادن بلوغ امنیتی برنامه و سازمان طراحی شده‌اند. به جای آنکه ASVS الزام کند که یک برنامه باید در چه سطحی باشد، این مسئولیت برعهده‌ی سازمان است که با تحلیل ریسک، و با توجه به حساسیت برنامه و انتظارات کاربران، سطح مناسب را انتخاب کند.

برای نمونه، یک استارت‌آپ نوپا که فقط مقدار محدودی از داده‌های حساس را جمع‌آوری می‌کند، ممکن است تصمیم بگیرد سطح ۱ را برای اهداف اولیه امنیتی انتخاب کند؛ اما یک بانک برای برنامه بانکی آنلاین خود، به سختی می‌تواند چیزی کمتر از سطح ۳ را به مشتریانش توجیه کند.

چگونه از ASVS استفاده کنیم

ساختار ASVS

ASVS شامل حدود ۳۵۰ الزام امنیتی است که در قالب ۱۷ فصل دسته‌بندی شده‌اند. هر فصل نیز به بخش‌های کوچک‌تر تقسیم شده است.

هدف از این تقسیم‌بندی فصل‌ها و بخش‌ها، ساده‌سازی انتخاب یا فیلتر کردن موارد مرتبط با برنامه‌ی خاص است. برای مثال، در یک API ماشین‌بنده‌ماشین، الزامات فصل ۳ که مربوط به Frontend است، کاربرد نخواهد داشت. اگر از OAuth یا WebRTC استفاده نشده، آن فصل‌ها نیز می‌توانند نادیده گرفته شوند.

strategy Release

انتشارهای ASVS از الگوی "Major.Minor.Patch" پیروی می‌کنند که این سه عدد، نوع تغییرات را نشان می‌دهند، در یک Major عدد اول تغییری می‌کند، در Minor عدد دوم تغییری می‌کند، و در Patch عدد سوم تغییری می‌کند.

- (Major release): بازسازمانده‌ی کامل؛ ممکن است تقریباً همه چیز از جمله شماره‌های الزامات تغییری کند. ارزیابی مجدد برای انطباق لازم است. مثال: 4.0.3 → 5.0.0
- (Minor release): ممکن است الزامات جدیدی اضافه یا حذف شوند، اما شماره گذاری کلی حفظ می‌شود. ارزیابی مجدد لازم است، ولی ساده‌تر خواهد بود. مثال: 5.0.0 → 5.1.0
- (Patch release): ممکن است الزامات تک‌راری یا منسوخ حذف شوند یا سخت‌گیری‌ها آنها کاهش یابد، ولی برنامه‌ای که با نسخه قبلی انطباق داشته، با نسخه وصله‌شده نیز انطباق خواهد داشت. مثال: 5.0.0 → 5.0.1

نکته: آنچه در بالا آمد، صرفاً مربوط به الزامات ASVS است. تغییرات متنی در بخش‌های جانبی یا ضمائم، به‌عنوان تغییری بنیادی در نظر گرفته نمی‌شوند.

انعطاف پذیری در استفاده از ASVS

برخی از موارد پیشگفته، مانند الزامات مستندسازی و سازوکار سطوح، این امکان را فراهم می‌کنند که از ASVS به صورت انعطاف پذیری و متناسب با نیازه‌ای هر سازمان استفاده شود.

علاوه بر این، قویاً توصیه می‌شود که سازمان‌های یک نسخه مشخصی سازی شده از ASVS ایجاد کنند که الزامات را با توجه به ویژگی‌ها و سطح ریسک برنامه‌های خود تنظیم نمایند. با این حال، حفظ قابلیت رهگیری (traceability) الزامی است؛ به گونه‌ای که مثلاً گفراندن الزام 4.1.1 در تمام نسخه‌ها معنای یکسانی داشته باشد.

در حالت ایده‌آل، هر سازمان باید نسخه اختصاصی ASVS خود را تهیه کند و بخش‌های نامربوط مانند GraphQL، WebSockets یا SOAP (در صورت عدم استفاده) را حذف نماید. چنین نسخه‌ای همچون محل مناسبی برای افزودن راه‌نمای‌های خاص سازمان، مانند کتابخانه‌ها یا منابع توصیف شده برای پیاده‌سازی الزامات، خواهد بود.

نحوه ارجاع به الزامات ASVS

هر الزام دارای شناسه‌ای با قالب `<chapter>.<section>.<requirement>` است که هر بخش‌ی عدد می‌باشد. برای مثال: 1.11.3.

- مقدار `<chapter>` نشان‌دهنده فصل الزام است؛ برای نمونه، تمام الزامات 1.#.1 متعلق به فصل "رمزگذاری و پاک‌سازی" هستند.
- مقدار `<section>` نشان‌دهنده بخش درون آن فصل است؛ مثلاً تمام الزامات 1.2.# در بخش "پیش‌گیری از تزریق" فصل رمزگذاری و پاک‌سازی قرار دارند.
- مقدار `<requirement>` بی‌انگیزه شماره الزام خاص درون فصل و بخش است؛ مثلاً 1.2.5 که در نسخه 5.0.0 چنین تعریف شده است:

تأیید شود که برنامه از OS Command Injection محافظت کند و فراخوانی‌های سیستم‌عامل از پرس‌وجوی پارامتری با Encoding مناسب استفاده نماید.

از آنجا که شناسه‌ها مهم‌کن است بین نسخه‌های مختلف تغیری کنند، توصیه می‌شود که در اسناد، گزارش‌ها یا ابزارها از قالب زیر استفاده شود: `v<version>-<chapter>.<section>.<requirement>` که در آن version نشان‌دهنده برچسب نسخه ASVS می‌باشد. برای مثال: 5.0.0-1.2.5 به صورت دقیقی به الزام پنجم از بخش «Prevention Injection» در فصل «Sanitization» and Encoding در نسخه 5.0.0 اشاره دارد. (این قالب را می‌توان به صورت خلاصه شده به شکل `v<version>-<requirement_idenfier>` نامایش داد).

نکته: حرف v پیش از شماره نسخه باید همواره با حروف کوچک نوشته شود.

اگر شناسه‌ها بدون عنصر `v<version>` استفاده شوند، باید فرض شود که به آخرین محتوای ASVS اشاره دارند. با گسترش و تغیری استناد دارد، این کار مشکلساز می‌شود؛ به همین دلیل توصیه می‌شود نویسنندگان و توسعه‌دهندگان همواره نسخه را نیز ذکر کنند.

فهرست الزامات ASVS در قالب‌های CSV،JSON و سایر فرمت‌ها نیز برای ارجاع یا استفاده برنامه‌نویسی در دسترس است.

فورک کردن (Forking) ASVS

سازمان‌ها می‌توانند با انتخاب یکی از سه سطح یا ایجاد نسخه فورک شده‌ی دامنه‌محور، از مزایای ASVS بهره‌مند شوند. این نسخه فورک شده می‌تواند الزامات را بر اساس سطح ریسک برنامه تنظیم کند. این اقدام تشویق می‌شود، مشروط بر آنکه قابلیت رهگیری حفظ شود (یعنی مثلاً الزام 4.1.1 در همه نسخه‌ها معنای یکسانی داشته باشد).

در حالت ایده‌آل، هر سازمان باید نسخه‌ای متناسب با نیازه‌ی خود از ASVS تهیه کند و بخش‌های غیرمرتبط (مانند GraphQL، SOAP Websockets در صورتی که مورد استفاده قرار نمی‌گیرد) را حذف نماید. ایجاد نسخه‌ی اختصاصی (forking) باید با سطح ۱ ASVS به عنوان پایه آغاز شود و سپس بر اساس سطح ریسک برنامه، به سطوح ۲ یا ۳ ارتقاء یابد.

موارد استفاده از ASVS

ASVS می‌تواند برای ارزیابی امنیتی یک برنامه کاربردی مورد استفاده قرار گیرد. که این موضوع در فصل بعد با جزئیات بیشتری بررسی خواهد شد. با این حال، موارد استفاده‌ی دیگری نیز برای ASVS (یا نسخه‌های فورک شده آن) شناسایی شده‌اند.

به عنوان راهنمای دقیقی معماری امنیتی

یکی از رایج‌ترین کاربردهای ASVS، استفاده از آن به عنوان مرجعی برای معماران امنیتی است. منابع محدودی برای نحوه طراحی معماری امن برای برنامه‌های مدرن وجود دارد ASVS می‌تواند خلأ موجود را با ارائه‌ی کنترل‌های مناسب برای چالش‌های رایج، مانند الگوهای حفاظت از داده‌ها یا استراتژی‌های اعتبارسنجی ورودی، پر کند. الزامات مربوط به معماری و مستندسازی در این زمینه به‌ویژه مفید خواهند بود.

به عنوان مرجع تخصصی برای برنامه‌نویسی امن

ASVS می‌تواند به عنوان مبنای برای تهیه مرجع برنامه‌نویسی امن در طول توسعه نرم‌افزار مورد استفاده قرار گیرد و به توسعه‌دهندگان کمک کند تا در حین ساخت نرم‌افزار، امنیت را به‌طور مستمر مدنظر داشته باشند. اگرچه ASVS می‌تواند پایه‌ی این مرجع باشد، اما توصیه می‌شود سازمان‌ها راهنمای اختصاصی خود را تهیه کنند—راهنمای که شفاف، یکپارچه، و ترجیحاً با مشارکت مهندسان امنیتی یا معماران امنیتی تدوین شده باشد. به عنوان مکمل این راهنما، به سازمان‌ها تاکید می‌شود تا در صورت امکان، سازوکارها و کتابخانه‌های امنیتی تأیید شده‌ی تهیه کنند که در راهنما معرفی شده و توسعه‌دهندگان در پروژه‌های خود از آنها استفاده نمایند.

به عنوان راهنمای آزمون‌های واحد و یکپارچه‌سازی خودکار

ASVS به گونه‌ای طراحی شده که بسیاری از قابل آزمون باشد. برخی از الزامات ماهیتی فنی دارند، در حالی که برخی دیگر (مانند الزامات معماري یا مستندسازی) نیاز به بررسی مستندات یا بازبینی معماري دارند. با طراحی آزمون‌های واحد و یکپارچه‌سازی که آسب‌پذیری و سوءاستفاده‌های خاص

مرتبط با الزامات قابلیت‌آیی را بررسی و fuzz می‌کنند، می‌توان اطمینان یافت که این کنترل‌ها در هر نسخه از برنامه (build) به‌درستی عمل می‌کنند. برای نمونه، می‌توان تست‌های اضافی به مجموعه تست‌ها برای یک کنترل ورود (login) (controller) افزود که پارامتر نام کاربری (username) را از نظر مواردی مثل نام‌های کاربری پیش‌فرض رایج، تشخیص وجود حساب (account enumeration)، حملات brute، force، تزریق LDAP و SQL، XSS بررسی کنند. به‌صورت مشابه، تست پارامتر رمز عبور (password) نیز باید شامل بررسی رمزه‌ای رایج، طول رمز عبور، تزریق null، byte حذف پارامتر، XSS، و موارد دیگر باشد.

برای آموزش توسعه امن

ASVS می‌تواند برای تعریف ویژگی‌های نرم‌افزار امن نیز مورد استفاده قرار گیرد. بسیاری از دوره‌های “برنامه‌نویسی امن” در واقع صرفاً دوره‌ای که اخلاقی هستند که نکاتی جزئی در مورد کنونی‌سی امن به آن‌ها اضافه شده است. و این لزوماً به‌به‌بود مهارت توسعه‌دهندگان در نوشتن کد امن کم‌کم نمی‌کند. در عوض، دوره‌های آموزش توسعه امن می‌توانند با محوری ASVS و تمرکز قوی بر سازوکارهای مثبت و کاربردی آن طراحی شوند، نه صرفاً تمرکز بر فهرست 10 Top آسیب‌پذیری‌ها یا کاره‌ای که نباید انجام داد. ساختار ASVS همچنین یکی که چارچوب منطقی برای مرور گام‌به‌گام موضوعات مختلف مرتبط با ایمن‌سازی برنامه فراهم می‌کند.

به‌عنوان چارچوبی برای راهنمای خرید نرم‌افزار امن

ASVS یکی که چارچوب عالی برای کم‌کم به خرید نرم‌افزار امن یا خدمات توسعه اختصاصی امن است. خریدار می‌تواند به‌سادگی مشخص کند که نرم‌افزاری که قصد تهیه آن را دارد، باید مطابق با سطح X از ASVS توسعه یافته باشد و از فروشنده بخواهد اثبات کند که نرم‌افزار با آن سطح از استاندارد ASVS منطبق است.

کاربرد عملی ASVS

ته‌دی‌ده‌ای مختلف، انگیزه‌های متفاوتی دارند. برخی صنایع دارای دارای‌های اطلاعاتی و فناوری منحصر به فرد، و همچنین الزامات انطباق قانونی خاص حوزه‌ی خود هستند.

به‌سازمان‌ها تکیه می‌شود که با توجه به ماهیت کسب‌وکار خود، ویژگی‌های خاص ریسک‌های خود را عمیقاً بررسی کنند و بر اساس این تحلیل، سطح مناسب ASVS را که با نیازهای کسب‌وکارشان مطابق دارد، تعیین نمایند.

ارزیابی و صدور گواهی

دیدگاه OWASP در مورد گواهی‌نامه‌ها و نشان‌های اعتماد ASVS

OWASP به‌عنوان یکی که سازمان غیرانتفاعی و بی‌طرف از نظر تجاری، هیچ فروشنده، ارزیاب یا نرم‌افزاری را گواهی نمی‌کند. هرگونه ادعا مبنی بر گواهی‌بودن ASVS، ارائه نشان اعتماد یا تضمین

انطباق با ASVS توسط اشخاص ثالث، به‌هی‌چ‌وجه تأیید رسمی OWASP نیست. از این‌رو، سازمان‌ها باید نسبت به چنین ادعاهایی با احتیاط عمل کنند.

البته سازمان‌ها می‌توانند خدمات تضمین امنیتی ارائه دهند، مشروط بر آن‌که ادعای دریافت گواهی رسمی از سوی OWASP نداشته باشند.

نحوه تأیید انطباق با ASVS

ASVS به‌طور عمده چارچوبی از ارزیابی انطباق در سطحی مانندی که راهنمای تست را تجویز نمی‌کند، اما تأکید بر برخی نکات کلیدی اهمیت دارد.

گزارش‌گیری ارزیابی

گزارش‌های سنتی تست نفوذ تنها به ارائه "موارد استثناء" می‌پردازند و صرفاً شکست‌ها را فهرست می‌کنند. اما گزارش گواهی‌نامه ASVS باید شامل موارد دامنه ارزیابی (Scope)، خلصه‌ای از تمام الزامات بررسی شده، الزامات دارای استثناء، اعلام انطباق و راهنمایی برای رفع مشکلات شناسایی شده. برخی الزامات ممکن است به دلایل فنی یا عمل‌کردی، قابل اعمال نباشند (مثلاً مدیری نشست در های API بدون وضعیت) که در این صورت، باید در گزارش ذکر شود.

دامنه ارزیابی

سازمانی که یکی از نرم‌افزار را توسعه می‌دهد، معمولاً همه الزامات ASVS را پیاده‌سازی نمی‌کند، زیرا برخی الزامات ممکن است برای آن نرم‌افزار خاص نامربوط یا کم‌اهمیت باشند. بنابراین، ارزیاب باید دامنه بررسی را به‌روشنی مشخص کند؛ از جمله اینکه سازمان در پی دستیابی به کدام سطح (Level) است و کدام الزامات را شامل کرده است. این شفاف‌سازی باید بر اساس آنچه در گزارش گنجانده شده انجام‌گیری، نه آنچه حذف شده است. همچنین ارزیاب باید نظر خود را در مورد دلایل مستند حذف یا پیاده‌سازی نشدن الزامات خاص ارائه دهد.

این موضوع به مصرف‌کننده گزارش کمک می‌کند تا زمینه ارزیابی را بهتر درک کرده و در مورد میزان اعتماد به نرم‌افزار، تصمیم‌گیری آگاهانه‌تری بگیرد.

سازمان‌هایی که گواهی صادر می‌کنند می‌توانند روش‌های مختللفی برای آزمون انتخاب کنند، اما باید این روش‌ها را در گزارش خود به‌طور شفاف ذکر کنند و بهتر است این روش‌ها قابل تکرار باشند. بسته به نوع نرم‌افزار و نیازهای آن، ممکن است برای بررسی مواردی مانند اعتبارسنجی ورودی، از روش‌های مثل تست نفوذ دستی یا تحلیلی کدام‌نوع استفاده شود.

روش‌های ارزیابی

تکنیکی که‌های مختللفی برای تأیید الزامات خاص ASVS ممکن است مورد نیازی باشند. علاوه بر تست نفوذ (با استفاده از اعتبارنامه‌های معتبر برای پوشش کامل برنامه)، تأیید الزامات ASVS ممکن است به دست‌ترسی به مستندات، کدام‌نوع، پی‌کربندی، و افراد درگیر در فرآیند توسعه نیاز داشته باشد؛ به‌ویژه برای تأیید الزامات سطح ۲ (L2) و سطح ۳ (L3) ارائه شواهد مستند و قوی از

یافته‌ها با مستندسازی دقیقی، از جمله اوراق کاری، تصاویر صفحه (اسکرین‌شات)، اسکرین‌پت‌ها و لاگ‌های تست، یک رویه استاندارد محسوب می‌شود. اجرای صرف یک ابزار خودکار بدون انجام تست‌های دقیقی، برای دریافت گواهی‌نامه کافیه نیست؛ چراکه هر الزام باید به‌طور قابل تأیید بررسی و آزموده شود.

استفاده از ابزارهای خودکار برای تأیید الزامات ASVS همواره موضوعی مورد توجه بوده است. بنابراین، مهم است که نکاتی در مورد تست خودکار و تست جعبه سیاه (black box testing) روشن و شفاف بیان شوند.

نقش ابزارهای خودکار امنیتی زمانی که ابزارهای خودکار تست امنیتی مانند DAST (تست امنیتی پویا) و SAST (تست امنیتی ایستا) به‌درستی در خط ساخت (build pipeline) پیاده‌سازی شوند، ممکن است بتوانند برخی از مشکلات امنیتی را شناسایی کنند که اصولاً نباید وجود داشته باشند. با این حال، اگر این ابزارها به‌دقت پی‌کربندی و تنظیم نشوند، پوشش لازم را فراهم نخواهند کرد و حجم زیاد هشدارهای نادرست (noise) باعث می‌شود مشکلات امنیتی واقعی قابل شناسایی و رفع نباشند.

در حالی که این ابزارها ممکن است برخی از نیازمندی‌های فنی ساده‌تر مانند کدگذاری خروجی یا پاک‌سازی (sanitization) را پوشش دهند، نکته بسیار مهم این است که نمی‌توانند بسیاری از الزامات پیچیده‌تر ASVS (استاندارد تأیید امنیتی نرم‌افزار) را، به‌ویژه آن‌هایی که به منطق تجاری یا کنترل دسترسی مربوط می‌شوند، به‌طور کامل بررسی کنند.

برای نیازمندی‌هایی که ساده نیستند، همچنان می‌توان از خودکارسازی استفاده کرد، اما باید بررسی‌های خاص برای هر نرم‌افزار نوشته شود تا این الزامات پوشش داده شوند. این بررسی‌ها می‌توانند شبیه به تست‌های واحد (unit test) یا تست‌های یکپارچه‌سازی (integration test) باشند که ممکن است قبلاً در سازمان وجود داشته باشند. بنابراین ممکن است بتوان از زیرساخت فعلی تست خودکار برای نوشتن تست‌های مخصوص ASVS استفاده کرد. اگرچه انجام این کار در کوتاه‌مدت نیازی به سرمایه‌گذاری بل‌ندمدت آن درام‌کان بررسی مستمر الزامات امنیتی ASVS بسیاری از قابلیت‌ها خواهد بود.

به‌طور خلاصه قابلیت تست بودن از طریق خودکارسازی به این معنا نیست که می‌توان صرفاً یک ابزار آماده را اجرا کرد.

نقش تست نفوذ در نسخه 4.0 سطح L1 برای انجام تست نفوذ «جعبه سیاه» (بدون دسترسی به مستندات یا کد) به‌یمنه‌سازی شده بود، اما حتی در آن نسخه نیز تأکید شده بود که این روش نباید به‌عنوان فعالیت اصلی تضمین امنیتی تلقی شود و باید از آن پرهیز کرد.

تست بدون دسترسی به اطلاعات کم‌لی، شیوه‌ای ناکارآمد و ضعیف برای ارزیابی امنیتی است، چراکه فرصت بررسی کد منبع، شناسایی تهدیدها، کشف کنترل‌های ناقص و انجام تست جامع‌تر در زمان کمتر را از بین می‌برد.

به‌طور جدی توصیه می‌شود که از روش‌های ترکیبی (Hybrid) برای تست نفوذ استفاده شود، یعنی تست‌هایی که با دسترسی کامل به مستندات، کد و توسعه‌دهندگان انجام می‌شوند؛ زیرا چنین سطحی از دسترسی برای ارزیابی دقیقی بسیاری از الزامات ASVS ضروری خواهد بود.

تغییرات نسبت به نسخه 4.x

مقدمه

کاربرانی که با نسخه 4.x این استاندارد آشنا می‌دارند، ممکن است مرور تغییرات کلی‌دی معرفی شده در نسخه 5.0 را مفید بیابند؛ تغییراتی که شامل به‌روزرسانی‌های در محتوا، دامنه و فلسفه زیربنایی استاندارد هستند.

از میان 286 الزام موجود در نسخه 4.0.3، تنها 11 مورد بدون تغییری باقی مانده‌اند و 15 مورد صرفاً اصلاحات جزئی گرامری داشته‌اند که معنای آن‌ها را تغییری نداده است. در مجموع، 109 الزام (معادل 38%) در نسخه 5.0 دی‌گربه‌صورت الزامات جداگانه وجود ندارند؛ که از این میان، 50 مورد حذف شده‌اند، 28 مورد به‌عنوان موارد تک‌راری حذف شده‌اند و 31 مورد در الزامات دی‌گراذغام شده‌اند. مابقی الزامات نیز به‌نوعی مورد بازنگری قرار گرفته‌اند. حتی الزامات تغییریافته‌ای که از لحاظ محتوایی تغییری زیادی نداشته‌اند نیز به‌دلیل بازآرایی یا بازساختاردهی، شناسه‌های متفاوتی دارند.

برای تسهیل پذیرش نسخه 5.0، اسنادی برای نگاشت الزامات ارائه شده‌اند تا به کاربران کمک کنند نحوه تطابق الزامات نسخه 4.x با نسخه 5.0 را دنبال کنند. این نگاشت‌ها به نسخه انتشار خاصی وابسته نیستند و در صورت لزوم به‌روزرسانی یا شفاف‌سازی خواهند شد.

فلسفه الزامات

دامنه و تمرکز

در نسخه 4.x الزامات متعددی گنجانده شده بودند که با دامنه مورد نظر این استاندارد هم‌راستا نبودند؛ این موارد حذف شده‌اند. همچنین، الزاماتی که با معیارهای دامنه نسخه 5.0 سازگار نبودند یا قابلیت ارزیابی نداشتند نیز کنار گذاشته شده‌اند.

تمرکز بر اهداف امنیتی به‌جای سازوکارها

در نسخه 4.x بسیاری از الزامات بر سازوکارهای خاصی متمرکز بودند و نه بر اهداف زیربنایی امنیتی. در نسخه 5.0، تمرکز الزامات بر اهداف امنیتی است و تنها در مواردی که یک سازوکار خاص تنها راه حل عملی است، به آن اشاره شده یا به‌عنوان نمونه یا راه‌نمایی مکمل آورده شده است.

این رویکرد به رسمیت می‌شناسد که روش‌های متعددی ممکن است برای رسیدن به یک هدف امنیتی وجود داشته باشند و از تجویزهای غیرضروری که می‌تواند انعطاف‌پذیری سازمان‌ها را محدود کند، اجتناب می‌کند.

همچنین، الزامات مرتبط با یک دغدغه امنیتی واحد، در صورت لزوم با هم ادغام شده‌اند.

تصمیمات امنیتی مستند

هرچند مفهومی تصمیمات امنیتی مستند ممکن است در نسخه 5.0 جدید به‌نظر برسد، در واقع ادامه و تکامل الزاماتی در نسخه 4.0 است که به تحلیلی سیستمی است و مدل‌سازی تهدید اشاره داشتند.

در گذشته، برخی الزامات به‌طور ضمنی نیازیمند تحلیلی برای پیاده‌سازی کنترل‌های امنیتی بودند، مانند تعین اتصالات شبکه مجاز.

به‌منظور اطمینان از در دسترس بودن اطلاعات لازم برای اجرا و ارزیابی، این انتظارات اکنون به‌طور صریح به‌عنوان الزامات مستندسازی تعریف شده‌اند تا شفاف، قابل اجرا و قابل ارزیابی باشند.

تغییرات ساختاری و فصل‌های جدید

چند فصل در نسخه 5.0 محتوای کاملاً جدیدی را معرفی کرده‌اند:

- OAuth و OIDC: با توجه به رواج گسترده این پروتکل‌ها برای واگذاری دسترسی و ورود یک‌پارچه (SSO)، الزامات اختصاصی برای این حوزه تعریف شده‌اند تا سناریوهای متنوعی که توسعه‌دهندگان ممکن است با آن‌ها مواجه شوند، پوشش داده شود. این حوزه ممکن است در آینده به استانداردی مستقل تبدیل شود (مشابه آنچه برای حوزه موبایل و IoT در نسخه‌های قبلی رخ داد).

- WebRTC: با توجه به رشد سریعی این فناوری، ملایمات و چالش‌های امنیتی خاص آن اکنون در بخشی اختصاصی پوشش داده شده‌اند.

همچنین، تلاش شده ساختار فصل‌ها به‌گونه‌ای تنظیم شود که مجموعه‌های منسجم و مرتبطی از الزامات را در کنار هم قرار دهد.

این بازسازماندهی منجر به ایجاد فصل‌های جدیدی نیز شده است:

- توکن‌های مستقل (Self-contained Tokens): که پیشتر زیرمجموعه مدیریتی نشست بودند، اکنون به‌عنوان یکی از سازوکار مستقل شناخته می‌شوند و یکی از عناصر بنیادی در ارتباطات بدون وضعیت (stateless) مانند OAuth و OIDC به‌شمار می‌روند. به‌دلیل پیاده‌های امنیتی خاص این نوع توکن‌ها، در نسخه x.5 یکی فصل اختصاصی به آن‌ها اختصاص داده شده و برخی الزامات جدیدی نیز در این زمینه معرفی شده‌اند.
- امنیت فرانت‌اند وب: با توجه به پیچیدگی فزاینده برنامه‌های تحت مرورگر و معماری‌های مبتنی بر API، الزامات امنیتی فرانت‌اند به فصل مستقلی منتقل شده‌اند.
- کد نویسی و معماری ایمن: الزاماتی که به شیوه‌های کلی امنیتی مربوط می‌شدند ولی در فصل‌های قبلی جای نمی‌گرفتند، در این بخش جمع‌آوری شده‌اند.

سایر تغییرات ساختاری در نسخه 5.0 به‌منظور شفاف‌سازی مقصود اعمال شده‌اند. برای مثال، الزامات مربوط به اعتبارسنجی ورودی به‌جای قرارگیری در کنار پاک‌سازی و کدگذاری، در کنار منطق تجاری قرار داده شده‌اند تا نقش آن‌ها در اجرای قوانین تجاری را بهتر نشان دهند.

فصل قبلی با عنوان معماری V1 به‌طور کامل حذف شده است. بخش ابتدایی آن شامل الزاماتی بود که خارج از محدوده موضوعی قرار داشتند، در حالی که بخش‌های بعدی به فصل‌های مرتبط منتقل شده‌اند و الزامات نیز در صورت لزوم بازنویسی، شفاف‌سازی و حذف موارد تکراری شده‌اند.

حذف نگاشت مستقیمی با استانداردهای دی گر

نگاشت مستقیمی با استانداردهای دی گراز متن اصلی استاندارد حذف شده است. هدف آن است که نگاشتی با پروژه (Common CRE OWASP Requirement Enumeration) ارائه شود که به نوبه خود ASVS را به طیفی از پروژه های OWASP و استانداردهای خارجی متصل کند. نگاشت مستقیمی با CWE و NIST دی گر حفظ نمی شود، که در ادامه توضیح داده شده است.

کاهش وابستگی به دست ورال عمل های هویت دیجیتالی NIST

دست ورال عمل های هویت دیجیتالی (SP Guidelines Identity Digital) (800-63) مدت ها به عنوان مرجع برای کنترل های احراز هویت و مجوزدهی استفاده می شدند. در نسخه x.4 برخی فصل ها با ساختار و واژگان NIST هم راستا بودند.

با آنکه این دست ورال عمل ها همچنان مرجع مهمی محسوب می شوند، این هم راستایی سخت گیری رانه چالش های ایجاد کرده بود؛ از جمله واژگان کمتر شناخته شده، تکرار الزامات مشابه، و نگاشت های ناقص. نسخه 5.0 از این رویکرد فاصله گرفت تا وضوح و ارتباط مؤثرتر را ارتقا دهد.

فاصله گرفتن از نگاشت CWE

Enumeration Weakness Common (CWE) طبقه بندی مفیدی از ضعف های امنیتی نرم افزار ارائه می دهد. با این حال، چالش هایی مانند های CWE که فقط شامل دسته بندی هستند، دشواری در نگاشت دقیقی الزامات به ی ک CWE خاص، و نگاشت های نامشخص در نسخه x.4 باعث شد تصمیم گرفت شود که در نسخه 5.0 نگاشت مستقیمی با CWE حذف شود.

بازنگری در تعریف سطوح (Levels)

نسخه x.4 سطوح امنیتی را به صورت L1 ("حداقلی")، L2 ("استاندارد") و L3 ("پیشرفته") تعریف می کرد و تلویحاً بیان می کرد که تمام برنامه های که با داده های حساس سروکار دارند باید حداقل به سطح L2 برسند.

نسخه 5.0 به چندین ایراد در این رویکرد پرداخته است که در پاراگراف های ذیل آمده است:

از لحاظ عملی برخلاف نسخه x.4 که الزامات تی ک (II) برای نمایش سطح استفاده می کرد، نسخه x.5 از عدد ساده در تمام قالب های سند (شامل، CSV، JSON، DOCX، PDF، markdown، XML) استفاده می کند. برای حفظ سازگاری با نسخه های قبلی، خروجی های قدیمی CSV، JSON و XML با الزامات تی ک نیز تولید می شوند.

ورود ساده تر به سطح اول

بازخوردها نشان دادند که تعداد بالای الزامات سطح اول (120 مورد) و اینکه این سطح به عنوان حداقل اما ناکافی معرفی شده بود، موجب کاهش تمایل به استفاده می شد. نسخه 5.0 تلاش کرده این مانع را با تعریف سطح L1 حول الزامات لایه اول دفاعی کاهش دهد؛ نتیجه این کار الزامات کمتر و روشن تر

در این سطح است. برای مقایسه عددی: در نسخه 4.0.3 تعداد 128 الزام L1 از مجموع 278 (معادل 46%) وجود داشت، اما در نسخه 5.0.0 این تعداد به 70 الزام از مجموع 345 (معادل 20%) کاهش یافته است.

افسانه قابلیت آزمون پذیری

در نسخه x.4 ی کی از معیارهای اصلی انتخاب الزامات سطح 1، قابلیت ارزیابی آنها از طریق تست نفوذ خراجی به صورت “جمع به سیاه” بود. اما این رویکرد با هدف واقعی سطح 1 (حداقل کنترل‌های امنیتی) همخوانی کامل نداشت. برخی کاربران معتقد بودند سطح 1 برای تأمین امنیتی کافی نیست، و برخی دیگر آن را بیش از حد سخت برای آزمون تلقی می‌کردند.

ات‌کا به قابلیت آزمون‌پذیری به عنوان معیاری، رویکردی نسبی و گاه گمراه‌کننده است؛ چراکه آزمون‌پذیری بودن الزام لزوماً به معنای سهولت یا امکان خودکارسازی آزمون آن نیست. همان‌طور که الزامات آسان‌تر برای آزمون لزوماً پراثرترین یا آسان‌ترین گزینیه‌ها برای اجرا نیستند.

بنابراین در نسخه 5.0، تصمیم‌گیری در مورد سطوح عمدتاً بر اساس کاهش ریسک انجام شده و می‌زان تلاش موردنیاز برای پیاده‌سازی نیز در نظر گرفته شده است.

فقط ریسک مطرح نیست

استفاده از سطوح نسخه‌ای و مبتنی بر ریسک که الزام سطح خاصی را برای برنامه‌ها تجویز می‌کنند، در عمل بیش از حد خشک و انعطاف‌ناپذیر بوده است. در واقع، اولویت‌بندی و پیاده‌سازی کنترل‌های امنیتی به عوامل متعددی بستگی دارد، از جمله کاهش ریسک و می‌زان تلاش اجرایی.

بنابراین توصیف می‌شود سازمان‌ها سطحی از ASVS را انتخاب کنند که متناسب با بلوغ امنیتی آنها و پیامی که می‌خواهند به کاربران خود منتقل کنند، باشد.

Sanitization and Encoding V1

هدف

این فصل به ضعف‌های رایج امنیتی در برنامه‌های وب می‌پردازد که ناشی از پردازش ناامن داده‌های غیرقابل‌اعتماد است. چنین ضعف‌هایی ممکن است منجر به آسیب‌پذیری‌های فنی مختلفی شوند، در شرایطی که داده‌های غیرقابل‌اعتماد، مطابق با دستور زبان مفسر مربوطه تفسیر می‌شوند.

در برنامه‌های وب مدرن، همواره به تری‌ن کار استفاده از API امن‌تر مانند auto-queries، parameterized escaping یا frameworks templating است. درگیری این صورت، اجرای دقیق و محتاطانه‌ی escaping encoding، sanitization برای امن‌ت برنامه حیاتی خواهد بود.

اعتبارسنجی ورودی به عنوان مکانیسم «دفاع در عمق» عمل می‌کند و از ورود محتوای غیرمنتظره یا خطرناک جلوگیری می‌کند. با این حال، از آنجا که هدف اصلی آن تطابق محتوای الزامات عمل‌کردی و تجاری است، الزامات مرتبط با آن در فصل «اعتبارسنجی و منطق کسب‌وکار» آمده است.

V1.1 ساختار Sanitization and Encoding

در بخش‌های زیر، الزامات مختص سورس کدی مختص مفسر برای پردازش امن داده‌های ناامن به منظور جلوگیری از آسیب‌پذیری‌های امنیتی ارائه شده‌اند. این الزامات، ترتیب انجام این پردازش و محل مناسب اجرای آن را پوشش می‌دهند. همچنین، هدف آن‌ها این است که هنگام ذخیره‌سازی داده، محتوای در حالت اولیه خود باقی بماند و به صورت encoded یا escaped (مثلاً HTML encoding) ذخیره نشود تا از بروز مشکلات ناشی از رمزگذاری دوگانه (double encoding) جلوگیری شود.

#	Description	Level
1.1.1	ورودی فقط یک بار به فرم است‌اندارد decoded یا unescaped شود، و این تنها زمانی انجام شود که انتظار می‌رود داده واقعاً در آن قالب رمزگذاری شده دریافت شده باشد. این فرآیند باید قبل از هرگونه پردازش بعدی روی ورودی انجام شود، و به‌هیچ‌وجه نباید پس از اعابت‌بارسنجی یا پاک‌سازی انجام گیرد.	2
1.1.2	برنامه فرآیند encode و escape خروجی را به‌عنوان آخرین مرحله انجام می‌دهد؛ به‌گونه‌ای که این عملیات یا دقیقاً پیش از استفاده داده در مفسر مربوطه انجام شود، یا اینکه خود مفسر به‌طور داخلی این کار را انجام دهد.	2

V1.2 پیش‌گیری از تزریق

encode و escape خروجی‌ها زمان دریافت ورودی‌های حساس که ممکن است منجر به تزریق شوند، برای امنیت برنامه حیاتی است. معمولاً این encode و escape خروجی ذخیره نمی‌شوند، بلکه صرفاً برای ایمن‌سازی نمایش در همان لحظه در مفسر استفاده می‌شوند. انجام زودهنگام این عملیات ممکن است باعث خرابی محتوای بی‌اثر شدن فرآیند encoding و escaping شود.

در بسیاری از موارد، کتابخانه‌های نرم‌افزاری توابع امن یا امن‌تری را برای انجام خودکار این کار ارائه می‌دهند، با این حال باید اطمینان حاصل شود که برای زمینه (context) موردنظر صحیح هستند.

#	Description	Level
1.2.1	encoding خروجی برای پاسخ HTTP، سند HTML یا سند XML با زمینه‌ای که داده در آن استفاده می‌شود باید متناسب باشد؛ برای مثال، در رمزگذاری کراک‌ترهای مربوط به عناصر HTML، صفات HTML، توضیحات HTML، CSS یا فیلدهای در HTTP به‌گونه‌ای عمل شود که ساختار پی‌ام‌ایا سند تغیری نکنند.	1
1.2.2	هنگام ساخت پویا آدرس‌های URL، داده‌های غیرقابل‌اعتتامد باید متناسب با زمینه استفاده‌شان رمزگذاری شوند (مثلاً Encoding URL یا base64url برای پارامترهای کوئری یا مسیری) همچنین اطمینان حاصل شود که فقط پروتکل‌های امن در URL مجاز باشند (برای نمونه، استفاده از JavaScript یا data: ممنوع شود).	1

#	Description	Level
1.2.3	هن گام تولید پویا محتوای JavaScript (از جمله JSON)، از encoding یا escaping خروجی استفاده شود تا از تغیری ساختار پیام یا سند جلوگیری گردد (و از حملات تزریق JavaScript یا JSON پیشگیری شود).	1
1.2.4	در انتخاب داده یا اجرای کوئری‌های پایگاه داده (مانند SQL، NoSQL HQL، ...)، از ORMها، queries، parameterized Framework Entity استفاده شود یا به روش‌های دیگری، برنامه در برابر تزریق SQL و سایر حملات تزرقی پایگاه داده ایمن شده باشد. این موضوع هنگام نوشتن procedure stored نیز صدق می‌کند.	1
1.2.5	برنامه در برابر تزریق دستورات سیستم عامل (OS command injection) محافظت شده است و در فراخوانی‌های سیستم عامل، از کوئری‌های پارامتری شده سیستم عامل یا Encoding خروجی دستورات سیستم عاملی استفاده می‌شود.	1
1.2.6	برنامه در برابر آسب پذیری‌های تزریق LDAP محافظت شده است، یا اینکه کنترلرهای امنیتی مشخصی برای جلوگیری از تزریق LDAP پیاده‌سازی شده‌اند.	2
1.2.7	برنامه در برابر حملات تزریق XPath، به هوسیلهی استفاده از کوئری‌های پارامتری شده یا کوئری‌های از پیش کامپایل شده محافظت شده باشد.	2
1.2.8	پردازشگرهای LaTeX به صورت امن پی‌کر بندی شده‌اند (برای نمونه با عدم استفاده از گزینه‌ی shell-escape -) و از فهرست مجاز دستورات (allowlist) برای جلوگیری از حملات تزریق LaTeX استفاده شده است.	2
1.2.9	برنامه کاراکترهای ویژه در عبارات Regex را escape می‌کند (معمولاً با استفاده از،) تا این کاراکترها به اشتباه به عنوان متاکاراکتر تفسیر نشوند.	2
1.2.10	برنامه در برابر حملات تزریق CSV و فرمول (Formula Injection) محافظت شده است. برنامه باید هنگام خروجی گرفتن فایلهای CSV، از قوانین escape تعیی نشده در بخش‌های ۲.۶ و ۲.۷ استاندارد RFC 4180 پیروی کند. همچنین، هنگام خروجی گرفتن به فرمت CSV یا سایر فرمت‌های صفحه گسترده (مانند XLS، XLSX یا ODF)، اگر کاراکترهای ویژه‌ای مانند «'»، «+»، «-»، «@»، «\t» (tab) و Null 0 (character) در ابتدای مقدار ی ک فیلد قرار داشته باشند، باید با یک تکت کوتیشن (') escape شوند.	3

نکته: استفاده از کوئری‌های پارامتری‌شده (Parameterized Queries) یا داده‌های Escaping ها در SQL هم‌شبه به تون‌های کافیه نیست. بخش‌های از کوئری مثل نام جدول‌ها و ستون‌ها (از جمله نام ستون‌های که در دست‌ور BY ORDER استفاده شده‌اند) را نمی‌توان با escape کردن امن کرد. اگر داده‌ای که کاربر وارد کرده (حتی بعد از escape کردن) در این بخش‌ها استفاده شوند، ممکن است: کوئری به درستی اجرا نشود (خطای SQL برده)، یا باعث حمله‌ی SQL Injection شود.

V1.3 پاک سازی (Sanitization)

ایده آلترین روش محافظت در برابر استفاده از محتوای غیرقابل اعتماد در یک بسترنام، استفاده از encoding یا escaping متناسب با همان بستراست؛ روشی که معنای اصلی محتوای نام را حفظ می کند اما آن را برای استفاده در همان زمینه ایمن می سازد - همانطور که در بخش قبلی به تفصیل توضیح داده شد.

اما اگر این امکان وجود نداشته باشد، باید از sanitization (پاک سازی) استفاده شود؛ یعنی حذف کاراکترهای یا بخش‌هایی از محتوا که ممکن است خطرناک باشند. در برخی موارد، این کار ممکن است باعث تغیری معنای ورودی شود، اما از منظر امنیتی، چاره‌ای جز این وجود ندارد.

#	Description	Level
1.3.1	تمام ورودی‌های HTML غیرقابل اعتماد (مثلاً از طریق وی‌رایش‌گرهای WYSIWYG یا ابزارهای مشابه)، با استفاده از یک کتابخانه یا قابلیت امن و معتبر sanitization (پاک سازی) شوند.	1
1.3.2	برنامه استفاده از تابع eval یا سایر قابلیت‌های اجرای پویا (dynamic) که مانند Language Expression Spring (SpEL)، اجتناب کنند. در مواردی که استفاده از این قابلیت‌ها اجتناب‌ناپذیر است، ورودی‌های کاربر که در اجرای کد گنجانده می‌شوند باید پیش از اجرا، به‌طور کامل پاک‌سازی (sanitized) شوند.	1
1.3.3	بخش‌های حائز اهمیت که احتمال وقوع حملات ناشی از ورودی‌های کاربر در آنها بالاست؛ باید پیش‌اپیش عملیات Sanitization جهت اقدامات ایمنی روی داده‌ها انجام شود؛ برای مثال، تنها کاراکترهایی که برای آن محتوا ایمن محسوب می‌شوند مجاز باشند، و ورودی‌های بی‌شمار حذف‌نشدنی کوتاه (trim) شوند.	2
1.3.4	محتوای اسکریپت‌پذیر (scriptable) فایل‌های SVG که توسط کاربر ارائه می‌شود، مورد اعتبارسنجی یا پاک‌سازی (sanitization) قرار گرفته باشد تا فقط شامل تگ‌ها و ویژگی‌هایی (attributes) باشد که برای برنامه ایمن هستند؛ برای مثال، از درج اسکریپت‌ها و عناصر خطرناک مانند foreignObject جلوگیری شده باشد.	2
1.3.5	برنامه محتوای زبان‌های قالب‌بنویسی یا اسکریپت‌پذیر ارائه‌شده توسط کاربر، مانند CSS Markdown، استایل‌شی‌های BBCode XSL یا موارد مشابه را پاک‌سازی (sanitize) کرده یا غیرفعال کرده باشد.	2
1.3.6	برنامه در برابر حملات جعل درخواسست سمت سرور (SSRF) محافظت شده باشد؛ به این صورت که داده‌ای غیرقابل اعتماد را در برابر یک لیست مجاز از پروتکل‌ها، دامنه‌ها، مسیری‌ها و پورته‌ها اعتبارسنجی کرده و کاراکترهای بالقوه خطرناک را پیش از استفاده از داده برای فراخوانی سرویس دی‌گر پاک‌سازی (sanitize) می‌کند.	2

#	Description	Level
1.3.7	برنامه در برابر حملات تزریق قالب (Template Injection) محافظت شده باشد؛ به این صورت که ساخت قالبها بر پایه داده‌های غیرقابل اعتماد مچ‌از نباشد. در صورتی که هیچ راه حل جایگزینی وجود نداشته باشد، هرگونه ورودی غیرقابل اعتماد که به صورت پویا در فرآیند ایجاد قالب وارد می‌شود، باید پیش از استفاده یا پاک‌سازی (sanitize) شود یا به صورت سخت‌گیرانه اعتبارسنجی (validate) گردد.	2
1.3.8	برنامه، ورودی غیرقابل اعتماد را پیش از استفاده در کوئری‌های Naming Java Naming Interface Directory and (JNDI) به درستی پاک‌سازی (sanitize) می‌کند و همچنین اطمینان حاصل شود که پی‌کرنددی JNDI به گونه‌ای امن انجام شده تا از بروز حملات تزریق JNDI جلوگیری شود.	2
1.3.9	برنامه، محتوا را پیش از ارسال به memcache پاک‌سازی (sanitize) می‌کند تا از حملات تزریق (injection attacks) جلوگیری شود.	2
1.3.10	رشته‌های قالب‌بندی (format strings) که ممکن است هنگام استفاده، به شکل‌های ناخواسته یا مخرب تفسیر شوند، پیش از پردازش پاک‌سازی (sanitize) شوند.	2
1.3.11	برنامه ورودی‌های کاربر را پیش از ارسال به سامانه‌های ایمیل (مانند SMTP یا IMAP) پاک‌سازی (sanitize) کند تا از حملات تزریق در SMTP یا IMAP جلوگیری شود.	2
1.3.12	عبارات منظم (Regular Expressions) عاری از اجزای حساس هستند که باعث بازگشت پذیری نمایی (exponential backtracking) می‌شوند، و همچنین ورودی‌های غیرقابل اعتماد به درستی پاک‌سازی (sanitize) شوند تا از حملات ReDoS (انکار سرویس با استفاده از عبارات منظم) یا Regex Runaway جلوگیری گردد.	3

V1.4 حافظه، رشته و کد بدون مدیریت (Memory, String, and Code) Unmanaged

الزامات زیر به ریسک‌های ناشی از استفاده نایم از حافظه می‌پردازند، که معمولاً در شرایطی مطرح هستند که برنامه از زبان‌های سیستمی یا کدهای مدیریتی نشده (unmanaged code) استفاده می‌کند.

در برخی موارد، می‌توان با تنظیم پرچم‌های کامپایلر (compiler flags) به این هدف دست یافت؛ مانند فعال‌سازی محافظت‌ها و هشدارهای مربوط به سرریز بافر (buffer overflow)، (stack randomization)، جلوگیری از اجرای داده‌ها (DEP)، و همچنین توقف فرایند ساخت (build) در صورت شناسایی عملیات نایم روی اشاره‌گرها، حافظه، رشته‌های قالب‌بندی، اعداد صحیح یا رشته‌های متنی.

#	Description	Level
1.4.1	برنامه از رشته‌های ایمن از نظر مدیریت حافظه، کپی حافظه‌های ایمن تر و حساب‌گری ایمن تر روی اشاره‌گرها استفاده می‌کند تا از سرریز پشته (stack overflow)، (buffer overflow) (heap overflow) سرریز حافظه پویای (heap overflow) جلوگیری کرده یا آنها را شناسایی کند.	2
1.4.2	از تکنیک‌های اعتباربارسنجی علامت (sign validation)، (range validation) و ورودی (input validation) استفاده شده است تا از بروز سرریز عدد صحیح (integer overflow) جلوگیری شود.	2
1.4.3	حافظه و منابعی که به صورت پویا تخصیص داده شده‌اند، به درستی آزاد (release) می‌شوند و همچنین ارجاع‌ها یا اشاره‌گرها (pointers) به حافظه آزاد شده حذف یا به null تنظیم می‌شوند تا از اشاره‌گرهای معلق (dangling pointers) و آسب‌پذیری‌های استفاده پس از آزادسازی (use-after-free) جلوگیری شود.	2

V1.5 سریال‌زدایی ایمن (Safe Deserialization)

تبدیل داده‌ها از یک قالب ذخیره‌شده یا انتقال‌یافته به یک object واقعی (برنامه‌فرایند - deserialization) در گذشته منبع بسیاری از آسیب‌پذیری‌های تزریق کد بوده است. بنابراین، انجام این فرایند باید با دقت و به صورت ایمن صورت گیرد تا از بروز چنین مشکلاتی جلوگیری شود.

به‌ویژه، برخی از روش‌های deserialization طبق مستندات زبان‌های برنامه‌نویسی یا چارچوب‌ها به‌عنوان ناایمن شناخته شده‌اند و نمی‌توانند هنگام کار با داده‌های غیرقابل اعتماد به صورت ایمن استفاده شوند. برای هر مکانیسم مورد استفاده، باید بررسی دقیقی و مسئولانه (due diligence) انجام شود.

#	Description	Level
1.5.1	برنامه، XML Parser را به گونه‌ای پی‌کربندی کرده باشد که از تنظیمات محدودکننده استفاده کنند، و ویژگی‌های ناایمنی مانند resolve کردن موجودیتهای خارجی (external entities) غیرفعال شده باشند تا از حملات XML Entity External (XXE) جلوگیری شود.	1
1.5.2	فرآیند deserialization داده‌های غیرقابل اعتماد، با اجرای مدیریت ایمن ورودی همراه باشد؛ برای مثال، استفاده از لیست مجاز (allowlist) انواع object یا محدود کردن انواع تعریف شده توسط کالاینت برای جلوگیری از حملات مرتبط با deserialization. همچنین، مکانیسم‌های deserialization که به‌طور صریح به‌عنوان ناایمن شناخته شده‌اند، نباید با داده‌های غیرقابل اعتماد مورد استفاده قرار گیرند.	2

#	Description	Level
1.5.3	Parser های مختللی که در برنامه برای یک نوع داده مشخص استفاده می‌شوند (برای مثال: Parser های XML، JSON، یا URL، رفتار تحلیلی یکسانی دارند و از مکانیسم یکسانی برای encoding کاراکترها استفاده می‌کنند، تا از بروز مشکلاتی مانند آسیب پذیری در سازگاری JSON (Interoperability) یا تفاوت در نحوه تحلیلی URI یا مسیریابی که ممکن است در حملات RFI یا SSRF سوءاستفاده شود، جلوگیری شود.	3

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- Sheet Cheat Prevention Injection LDAP OWASP
- Sheet Cheat Prevention Scripting Site Cross OWASP
- Sheet Cheat Prevention Scripting Site Cross Based DOM OWASP
- Sheet Cheat Prevention Entity External XML OWASP
- Testing Client-Side Guide: Testing Security Web OWASP
- Project Encoding Java OWASP
- Library Sanitization HTML Client-side - DOMPurify
- Files (CSV) Values Comma-Separated for Type MIME and Format Common - RFC4180

برای اطلاعات بیشتر، به ویژه در مورد مشکلات deserialization یا parsing، لطفاً مراجعه کنید به:

- Sheet Cheat Deserialization OWASP
- Vulnerabilities Interoperability JSON of Exploration An
- Languages Programming Trending In Parser URL Exploiting SSRF of Era New A - Tsai Orange

Logic Business and Validation V2

هدف

یکی که برنامه مورد تایید باید الزامات سطح بالا را به شرح زیر برآورده سازد:

- داده‌های دریافتی برنامه با الزامات تجاری یا عمل‌کردی مطابقت داشته باشد.
- منطق تجاری (Business Logic) دارای جریان ترتیبی بوده، به صورت گام به گام پردازش شود و قابل دور زدن نباشد.
- منطق تجاری شامل محدودیت‌ها و کنترل‌های برای شناسایی و جلوگیری از حملات خودکار است؛ مانند انتقال‌های مکرر با مبالغ کم یا اضافه کردن میلی‌ونها دوست به صورت اتوماتیک.

• در طراحی و پیاده‌سازی چریانه‌های منطق تجاری با ارزش بالا، سناریوهای سوءاستفاده و کاربران مخرب در نظر گرفته شود و تدابیری نظیر محافظت در برابر جعل هویت (Spoofing)، دستکاری (Tampering)، افشای اطلاعات و ارتقای سطح دسترسی (Privilege Escalation) اتخاذ شده باشد.

V2.1 مستندات اعتبارسنجی و منطق کسب و کار (Logic Business and Validation Documentation)

مستندات اعتبارسنجی و منطق تجاری باید به روشنی محدوده‌های منطق کسب و کار، قوانین اعتبارسنجی و سازگاری مناسب بین داده‌های ترکیبی شده (از نظر مفهومی و متنی) را تعریف کنند، تا مشخص باشد چه مواردی باید در برنامه پیاده‌سازی شوند.

#	Description	Level
2.1.1	قوانین اعتبارسنجی ورودی‌های برنامه (مندرجه در مستندات برنامه) باید چگونگی بررسی صحت داده‌ها را مطابق با ساختار مورد انتظار مشخص نماید. این ساختار می‌تواند شامل فرمت‌های رایج داده مانند شماره کارت اعتباری، آدرس ایمیل، شماره تلفن یا یک قالب داده‌ای داخلی باشد.	1
2.1.2	قوانین اعتبارسنجی ورودی‌های برنامه (مندرجه در مستندات برنامه) باید چگونگی بررسی سازگاری مناسب بین داده‌های ترکیبی شده (از نظر مفهومی و متنی) را مشخص نماید؛ مانند تطبیق کدپستی با نام محله.	2
2.1.3	انتظارات مربوط به محدوده‌ها و اعتبارسنجی‌های منطق کسب و کار مستند شده باشند؛ این مستندات باید شامل محدوده‌ها در سطح هر کاربر و محدوده‌های عمومی در سطح برنامه باشند.	2

V2.2 اعتبارسنجی ورودی (Input Validation)

کنترل‌های مؤثر در اعتبارسنجی ورودی، انتظارات کسب و کار یا اعمال کردی مرتبط با نوع داده‌ای که برنامه باید دریافت کند را اعمال می‌کنند. این کار منجر به کیفیتی بهتر داده‌ها و کاهش دامنه حمله (Attack Surface) می‌شود. با این حال، اعتبارسنجی ورودی جایگزین استفاده از encoding، پارامتری سازی یا پاک‌سازی داده‌ها هنگام استفاده از داده در سایر بخش‌های برنامه یا هنگام نام‌اش در خروجی نمی‌باشد.

در این زمینه، «ورودی» می‌تواند از منابع متنوعی دریافت شود، از جمله: فیلدهای فرم، HTML، درخواست‌های REST، پارامترهای URL، فیلدهای هدر کوکی، HTTP، فایلهای ذخیره شده در دیسک، پایگاه‌های داده، API خارجی.

یک کنترل منطق کسب و کار می‌تواند بررسی کند که مقدار ورودی، عددی کمتر از 100 است. یک انتظار عمل کردی می‌تواند بررسی کند که یک عدد (که تعینی نمی‌کند) یک حلقه چند بار اجرا شود تا پردازش بیش از حد و در نتیجه وضعیتی انکار سرویس (DoS) اتفاق نیفتد از یک آستانه‌ی خاص کمتر باشد.

با این‌که اعتبار سنجی بر پای Schema الزام صریح ندارد، ولی این روش می‌تواند مؤثرترین راه کار برای پوشش کامل اعتبار سنجی در های API HTTP یا سایر واسطه‌های که از JSON یا XML استفاده می‌کنند باشد.

نکات مهم درباره اعتبار سنجی بر اساس Schema:

- “نسخه منتشر شده” از مشخصات Schema JSON، Validation، برای استفاده در محیط تولید مناسب تلقی می‌شود، نه لزوماً پایدار (Stable). هنگام استفاده از این نوع اعتبار سنجی، باید اطمینان حاصل شود که هیچ مغایرتی با الزامات زیر وجود ندارد.
- در صورت نیاز پس از رسمی و نه‌ای شدن استاندارد، تمامی کتابخانه‌های مورد استفاده برای Schema JSON باید Validation به صورت مستمر پایش و به‌روزرسانی شوند.
- اعتبار سنجی DTD نباید مورد استفاده قرار گیرد و ارزیابی DTD در فری‌مورک‌ها باید غیرفعال شود، تا از حملات XXE مرتبط با DTD جلوگیری شود.

#	Description	Level
2.2.1	ورودی‌ها باید به منظور اعمال انتظارات کسب و کاری اعمال کردی مرتبط با آن‌ها اعتبار سنجی شوند. این اعتبار سنجی باید ی‌کی از این دو روش زیر را داشته باشد، استفاده از اعتبار سنجی مثبت (Positive) (Validation مثبتی بر فهرست مجاز از مقادیر، الگوها یا بازه‌های قابل قبول و تطبیق با ساختار مورد انتظار و محدودیت‌های منطقی، بر اساس قوانین از پیش تعریف شده. برای سطح 1، (L1) تمرکز می‌تواند بر ورودی‌های باشد که در تصمیم‌گیری‌های خاص تجاری یا امنیتی استفاده می‌شوند. برای سطح 2 و بالاتر، این اعتبار سنجی باید شامل تمام ورودی‌ها باشد.	1
2.2.2	برنامه به‌گونه‌ای طراحی شود که اعتبار سنجی ورودی را در لایه قابل اعتمادی از سرویس (Trusted Service) (Layer) اعمال کند. هرچند اعتبار سنجی سمت کلاینت (Client-side) (Validation) باعث به‌بود تجربه کاربری می‌شود و توصیه می‌گردد، اما نباید به عنوان ی‌ک کنترل امنیتی ات‌کا شود.	1
2.2.3	برنامه باید اطمینان حاصل نماید که ترکیب داده‌های مربوطه با توجه به قوانین از پیش تعریف شده مغ‌قول و منطقی باشد.	2

V2.3 امنیتی منطق کسب و کار (Business Logic) Security

این بخش به الزامات کلیدی می‌پردازد که هدف آن‌ها اطمینان از پیاده‌سازی صحیح منطق کسب و کاری برنامه است و برنامه در برابر حملات مربوط به کسب و کار، مصون است.

#	Description	Level
2.3.1	اطمینان حاصل شود که برنامه فقط جریانه‌های منطق تجاری را برای همان کاربر و طبق ترتیب گام به گام از پیش تعریف شده و مورد انتظار پردازش می‌کند و امکان رد کردن مراحل (Skipping) (Steps) وجود ندارد.	1
2.3.2	اطمینان حاصل شود که محدوده‌های منطق تجاری مطابق با مستندات برنامه پیاده‌سازی شده‌اند، تا از سوءاستفاده از نقص‌های منطقی جلوگیری شود.	2
2.3.3	تراکنش‌های سطح منطق کسب و کار (business logic) باید به گونه‌ای استفاده شوند که ای‌کی عملیات منطق کسب و کار به‌طور کامل با موفقیت انجام شود، یا در صورت بروز خطا، به حالت صحیح قبلی بازگرد (rollback) شود.	2
2.3.4	در منطق کسب و کار (business) باید (locking) از مکانیسم‌های قفل‌گذاری (locking) استفاده شده باشد تا اطمینان حاصل شود که منابع محدود (مانند صندوق‌های سی‌ان‌ام یا بازه‌های زمانی تحویل)، قابل رزرو دوباره (double-booked) از طریق دست‌کاری منطق برنامه نباشند.	2
2.3.5	اطمینان حاصل شود که جریانه‌های منطق کسب و کاری با ارزش بالایی‌ها تایید چند کاربره (Multi-user Approval) هستند تا از اقدامات غیرمجاز یا تصادفی جلوگیری شود. این موارد می‌تواند شامل - اما نه محدود به - انتقال‌های مالی بزرگ، تایید قراردادها، دسترسی به اطلاعات طبقه‌بندی شده، یا غیره‌های سازمانی‌ای‌ها در محیط‌های تولید صنعتی باشد.	3

V2.4 مقابله با خودکارسازی (Anti-automation)

این بخش شامل کنترل‌های ضد خودکارسازی است تا تعاملات انسانی‌مانند الزامی شود و از درخواست‌های خودکار بی‌شخص جلوگیری گردد.

#	Description	Level
2.4.1	کنترل‌های ضد خودکارسازی (Anti-Automation) در سیستم پیاده‌سازی شده باشند تا از سوءاستفاده‌ای‌ها مانع فرایوانی بی‌شخص از حدت وابع برنامه جلوگیری شود؛ چرا که این نوع حملات می‌توانند منجر به استخراج غیرمجاز داده‌ها، تولید داده‌های زائد، مصرف کامل سهمیه‌ها، عبور از محدوده‌های نرخ (Rate Limit)، ایجاد اختلال در سرویس (DoS) یا استفاده بی‌شخص از منابع گران‌قیمت شوند.	2

#	Description	Level
2.4.2	بررسی کنی د که چری انه ای منطق کسب و کار (Business Logic (Flows به گونه ای طراحی شده باشن د که نی ازم ند زمان بندی واقعی انسان باشن د، و از ارسال سری ع و غی ربطی عی تراکنش ها (Transaction Submissions) چلوگی ری کنن د.	3

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنی د:

- Testing Validation Input Guide: Testing Security Web OWASP
- Testing Logic Business Guide: Testing Security Web OWASP
- چلوگی ری از خود کارسازی (Anti-automation) را می توان به روش های مختلف ی انجام داد، از جمله با اس ت فاده از Applications Web to Threats Automated OWASP
- Sheet Cheat Validation Input OWASP
- Schema JSON

V3 امنیت رابط کاربری وب (Web Security) Frontend

هدف کنترلی

این دسته از الزامات بر روی محافظت در برابر حملات ی تمرکز دارد که از طریق رابط وب (Web Frontend) برنامه اجرا می شوند. این الزامات برای راه کارهای ی از نوع ماشینی-به-ماشینی (Machine-to-Machine) کاربردی ندارند.

V3.1 مستند سازی امنیت رابط کاربری وب (Web Security Frontend Documentation)

این بخش ویژگی های امنیتی مرورگر را که باید در مستندات برنامه مشخص شوند، توضیح می دهد.

#	Description	Level
3.1.1	در مستندات برنامه باید ویژگی‌های امنیتی مربوط به مرورگری که برنامه را اجرا می‌کند مشخص شده باشد (نظری است فاده از HTTPS، پشتیبانی از Security Content Strict HTTP (HSTS)، اجرای Policy (CSP) و سایر مکانیسم‌های امنیتی مرتبط با HTTP). همچنین در مستندات باید مشخص شود مرورگر در صورت عدم دسترسی به این ویژگی‌ها چه رفتاری باید داشته باشد. (برای مثال: نمایش هشدار به کاربر یا ممنوعیت از دسترسی به برنامه).	3

V3.2 تفسیر ناخواسته محتوا (Unintended Content Interpretation)

پردازش محتوا یا اعمال کرد مربوط به آن در یکی شرایط زمینه‌ای نادرست می‌تواند منجر به اجرای نامی محتوا یا مخرب برای کاربر شود.

#	Description	Level
3.2.1	کنترل‌های امنیتی متناسب با مرورگرها در هنگام پردازش پاسخ‌های HTTP به منظور جلوگیری از پردازش محتوا یا اعمال کرد مربوط به آن در شرایط زمینه‌ای نادرست باید بکار گرفته شود. (برای مثال، زمانی که یک API یک فایل بارگزاری شده توسط کاربر یا هر منبع دیگری به طور مستقیم درخواس می‌شود). کنترل‌های ممکن می‌توانند شامل این موارد باشند: عدم پردازش محتوا و مگر در صورت وجود هدرهای متناسب با درخواس‌های HTTP (مانند Sec-Fetch-*) به منظور اطمینان از صحیح بودن پردازش محتوا یا اعمال کرد مربوط به . است فاده از عباتر sandbox در هدر Content-Security-Policy یا است فاده از نوع attachment در هدر Content-Disposition .	1
3.2.2	اطمینان حاصل شود که محتوای که برای نمایش به صورت متن طراحی شده نه برای پردازش به صورت HTML، با است فاده از توابع امن نظری CreateTextNode یا textContent به منظور جلوگیری از اجرای ناخواسته محتوای مانند HTML یا JavaScript پردازش شود.	1
3.2.3	برنامه هنگام است فاده از اجاواس کریپت سمت کاربر (Client-Side)، JavaScript با است فاده از اعلام متغیرها به صورت صریح، انجام بررسی نوع به صورت سخت‌گیرانه، اجتناب از ذخیره‌سازی متغیرهای سراسری بر روی شیء document، و پیاده‌سازی ایزولاسیون فضای نام (namespace)، (isolation از وقوع Clobbering DOM جلوگیری می‌کند).	3

V3.3 پی‌کربندی کوکی (Cookie Setup)

این بخش الزامات مربوط به پی‌کربندی ایمن کوکی‌های حساس را تشریح می‌کند تا سطح بالاتری از اطمینان نسبت به اینکه آن‌ها توسط خود برنامه ایجاد شده‌اند فراهم شود و همچنین

از نشانه‌های امنیتی نام‌های ناشناس به‌شماره‌های گوناگون.

#	Description	Level
3.3.1	اطمینان حاصل شود که کوکی‌ها دارای ویژگی 'Secure' هستند. در صورتی که از پیشوند 'Host-' برای نام کوکی استفاده نمی‌شود، باید پیشوند Secure- در نام کوکی لحاظ شده باشد.	1
3.3.2	اطمینان حاصل شود که مقدار ویژگی SameSite برای هر کوکی، متناسب با هدف آن کوکی تنظیم شده باشد تا میزان آسیب‌پذیری در برابر حملات فوری به سمت کلمات کلیدی نظیر CSRF کاهش یابد.	2
3.3.3	اطمینان حاصل شود که کوکی‌ها از پیشوند Host- در نام خود استفاده می‌کنند، مگر در مواردی که کوکی به‌طور مشخص برای اشتراک‌گذاری بین چند دامنه (Hosts) طراحی شده باشد.	2
3.3.4	اطمینان حاصل شود که اگر مقدار کوکی (مثلاً توکن نشانه‌ها) برای اسکرپت‌های سمت کلمات کلیدی در دسترس باشد، ویژگی HttpOnly باید برای آن کوکی فعال شده باشد، و همچنین مقدار کوکی (مثلاً همان توکن نشانه‌ها) تنها از طریق هدر Set-Cookie به کلمات کلیدی ارسال شود.	2
3.3.5	اطمینان حاصل شود که حداکثر مجوع نام و مقدار کوکی از ۴۰۹۶ بایت تجاوز نکند. کوکی‌هایی که بیش از این مقدار باشند توسط مرورگر ذخیره نخواهند شد، و بنابراین در درخواست‌های ارسال نمی‌شوند؛ موضوعی که ممکن است باعث شود عمل‌کرده‌ای از برنامه که به آن کوکی وابسته‌اند، غیرفعال یا مختل شوند.	3

V3.4 هدرهای امنیتی (Browser Security Mechanism Headers)

این بخش مشخص می‌کند که کدام هدرهای امنیتی باید در پاسخ‌های HTTP تنظیم شوند، تا هنگام پردازش پاسخ توسط مرورگر، ویژگی‌ها و محدودیت‌های امنیتی مرورگر فعال گردند.

#	Description	Level
3.4.1	اطمینان حاصل شود که در تمامی پاسخ‌های HTTP، هدر Strict-Transport-Security وجود دارد تا سیاست (HTTP HSTS) Transport Strict (Security) اعمال شود. حداقل مقدار max-age باید برابر با یک سال باشد. برای برنامه‌های سطح 2 (L2) و بالاتر، این سیاست باید برای تمام زیردامنه‌ها (subdomains) نیز اعمال شود.	1

#	Description	Level
3.4.2	اطمینیان حاصل شود که در Access-Control-Allow-Origin مربوط به (CORS) دارای یک مقدار ثابت و مشخص توسط برنامه باشد یا اگر در Origin در درخواست مورد استفاده قرار می گیرد باید با فهرستی از مبدهای مورد اعتماد (Allowlist) اعتبار سنجی شود. در صورتی که استفاده از مقدار * در Access-Control-Allow-Origin تنظیم شده باشد، اطمینیان حاصل شود که هیچ اطلاعات حساسی در پاسخ قرار ندارد.	1
3.4.3	اطمینیان حاصل شود که تمامی پاسخ های HTTP شامل هدر Content-Security-Policy باشند تا ریسک اجرای اسکریپت های مخرب (مانند JavaScript) مخرب کاهش یابد. دستوره های 'none' object-src و 'none' base-uri الزامی هستند. برای برنامه های سطح 3، (L3) باید سیاست CSP به صورت اختصاصی برای هر پاسخ (Per-response Policy) و مبتنی بر nonce یا hash تعریف شده باشد.	2
3.4.4	اطمینیان حاصل شود که تمامی پاسخ ها شامل هدر X-Content-Type-Options: nosniff باشند، این (هدر) به مرورگرها دستور می دهد که برای پاسخ دریافتی از Sniffing Content یا حدس زدن نوع محتوا (MIME Type) (Guessing) استفاده نکنند و آن ها را ملزم می کند که نوع محتوای واقعی را دقیقاً مطابق مقدار هدر Content-Type در درخواست/پاسخ در نظر بگیرند. برای مثال، پاسخ به یک درخواست برای یک فایل استایل (CSS) تنها در صورتی پذیرفته می شود که مقدار Content-Type در پاسخ 'text/css' باشد. این کار همچنین امکان استفاده از قابلیت مسدودسازی خواندن منابع از مبدا متفاوت Blocking Read Cross-Origin یا (CORB) توسط مرورگر را فراهم می سازد.	2
3.4.5	برنامه باید (وبسایت یا اپلیکیشن) یک "Referrer Policy" (سیاست ارجاع) تنظیم کرده باشد تا از نشأت اطلاعات حساس از طریق هدر HTTP به نام Referer به سرویس های مشخص ثالث جلوگیری شود. این کار را میتوان از طریق هدر پاسخ HTTP به نام Referrer-Policy یا با استفاده از ویژگی های در عناصر HTML انجام داد. اطلاعات حساس ممکن است شامل مسیر (Path) و داده های کوئری (Query) در آدرس URL باشد، و در مورد برنامه های داخلی و غیر عمومی، حتی نام میزبان (hostname) نیز میتواند جزو اطلاعات حساس محسوب شود.	2
3.4.6	برنامه باید از frame-ancestors در هدر Content-Security-Policy (CSP) برای تمام پاسخ های HTTP استفاده کند، تا به صورت پیش فرض، اجازهی نمایش سایت در قالب iframe داده نشود، و فقط در صورت نیاز، نمایش منابع خاص (specific resources) در iframe مجاز باشد. به این نکته توجه کنید که هدر X-Frame-Options با این که هنوز توسط برخی مرورگرها پشتیبانی می شود، اما قدیمی (obsolete) است و نباید به آن تکیه کرد.	2
3.4.7	در Content-Security-Policy (CSP) باید شامل یک آدرس برای گزارش (Report violation) باشد.	3

#	Description	Level
3.4.8	تمام پاس‌خ‌های HTTP که باعث رندر (نمایش) می‌شوند مانند (پاس‌خ‌های با نوع محتوای text/html) باید شامل هدر Cross-Origin-Opener-Policy با دست‌ور same-origin یا same-origin-allow-popups باشند (بنا به نیازی). این کار از حملات چلوگری می‌کند که از دست‌رسی مشترک به اشیای Window سوءاستفاده می‌کنند، مانند حملات tabnabbing و counting. frame	3

V3.5 تفکیک مبدا توسط مرورگر (Separation) Origin

وقتی یک درخواست برای انجام عمل‌کرد حساس در سمت سرور پذیرفته می‌شود، برنامه باید اطمینان حاصل کند که این درخواست واقعاً از طرف خود برنامه یا یک منبع مورد اعتماد ارسال شده و توسط مهاجم جعل نشده است.

عمل‌کرد حساس در این زمینه می‌تواند شامل مواردی مانند ارسال فرم‌ها برای کاربران تأیید شده یا تأیید نشده (مثلاً درخواست احراز هویت)، عملیات تغییری وضعیت (state-changing operations)، عمل‌کردهایی که منابع زیادی مصرف می‌کنند (مانند خروجی گرفتن از داده‌ها) باشد.

مکانی‌زم‌های کلیدی برای محافظت در برابر چنین حملاتی شامل سیاست‌های امنیتی مرورگرها مانند Policy Origin Same برای جاوااسکریپت و همچنین منطق SameSite برای کوکی‌ها هستند. یکی دی‌گر از روش‌های رایج برای محافظت، مکانی‌زم CORS preflight است. این مکانی‌زم به‌ویژه برای اندپوینت‌هایی که قرار است از یک مبدأ متفاوت فراخوانی شوند حیاتی است، اما حتی برای اندپوینت‌هایی که نباید از مبدأهای دی‌گرفراخوانی شوند نیز می‌تواند به‌عنوان یک لایه محافظتی مؤثر در برابر جعل درخواست Forgery Request عمل کند.

#	Description	Level
3.5.1	برنامه باید از مکانی‌زم Preflight در CORS برای چلوگری از درخواست‌های cross-origin به عمل‌کردهای حساس استفاده نمی‌کند، این درخواست‌ها به‌گونه‌ای اعتبارسنجی شوند که فقط از مبدأ اصلی برنامه ارسال شده باشند. این اعتبارسنجی می‌تواند با استفاده از توکن‌های anti-forgery (tokens) یا الزام به ارسال هدرهای HTTP اضافی انجام شود که در لیست هدرهای مجاز CORS (CORS-safelisted) قرار ندارند. این اقدام برای محافظت در برابر حملات جعل درخواست از طریق مرورگر، که با نام Request (Cross-Site CSRF Forgery) شناخته می‌شوند، ضروری است.	1
3.5.2	اگر برنامه برای چلوگری از استفاده غیرمجاز cross-origin از عمل‌کردهای حساس، به مکانی‌زم Preflight در CORS متکی است، باید امکان فراخوانی این عمل‌کردها با درخواست‌هایی که باعث ایجاد Preflight نمی‌شوند، وجود نداشته باشد. برای تحقق این هدف ممکن است لازم باشد مقدار هدرهای Origin و Content-Type بررسی شوند یا از هدر اضافی استفاده شود که در لیست هدرهای مجاز CORS قرار ندارد.	1

#	Description	Level
3.5.3	برنامه باید درخواست‌های HTTP که به عمل کرده‌ای حساس از متدهای مناسب مانند POST، PUT، PATCH یا DELETE استفاده می‌کنند، و نه از متدهای که در استاندارد HTTP به عنوان "ایمن" تعریف شده‌اند، مانند HEAD GET یا OPTIONS. به عنوان جایگزینی مکتوم، می‌توان از بررسی سخت‌گیرانه هدیه‌ای *Sec-Fetch کرد تا اطمینان حاصل شود که درخواست از منبع نامناسبی مثل بارگذاری تصویر، ناوبری صفحه یا فراخوانی cross-origin ارسال نشده است.	1
3.5.4	برنامه‌های مجزا باید روی نام‌های دامنه (hostname) متفاوت میزبانی شده‌اند تا از محدودیت‌های ارائه‌شده توسط (Same-Origin Policy) بهره‌مند شوند. این شامل کنترل تعامل اسناد یا اسکرپت‌های بارگذاری شده از یک مبدأ با منابع مبدأ دیگر و اعمال محدودیت بر اساس دامنه برای کوکی‌ها می‌شود.	2
3.5.5	برنامه باید پیام‌های دریافت‌شده از طریق رابط postMessage در صورتی که مبدأ پیام مورداعتقاد نباشد یا ساختار پیام نامعتبر باشد، نادیده گرفته می‌شوند.	2
3.5.6	هیچ‌کدام از برنامه‌های قابل‌لیت JSONP استفاده نشود تا از حملات Cross-Site Inclusion Script (XSSI) جلوگیری شود.	3
3.5.7	داده‌هایی که نیازی به احراز هویت دارند، در پاس‌های مربوط به منابع اسکرپتی (مانند فایل‌های JavaScript) نباید قرار بگیری‌ند تا از حملات XSS جلوگیری شود.	3
3.5.8	اطمینان حاصل کنید که منابع احراز هویت‌شده (مانند تصاویر، ویدیوها، اسکرپت‌ها و سایر اسناد) فقط در مواقعی که هدف آن است، قابل بارگذاری یا جاسازی توسط کاربر باشند. این کار را می‌توان با اعتبارسنجی دقیق هدیه‌ای HTTP مانند *Sec-Fetch انجام داد تا اطمینان حاصل شود که درخواست از یک منبع غیرمجاز ارسال نشده است، یا با تنظیم هدیه Cross-Origin-Resource-Policy برای محدود کردن دسترسی مرورگر به محتوای.	3

V3.6 ی‌ک‌پارچگی منابع خارجی (External Resource Integrity)

این بخش، راهنمای‌هایی برای میزبانی ایمن محتوا در سایت‌های شخص ثالث (Third-party) ارائه می‌دهد.

#	Description	Level
3.6.1	منابع سمت کلاینت مانند کتابخانه‌های جاوااسکریپت، فایلهای CSS، فایلهای فونت‌های وب، تنه در صورتی به صورت خارجی (مثلاً از طریق شبکه تحویل محتوا - CDN) می‌زبانی شوند که آن منابع: استاتیکی (تغییرناپذیر) باشند، نسخه‌گذاری شده (versioned) باشند و یک پارچگی آنها با استفاده از SRI Integrity Subresource بررسی شود. اگر این موارد امکان‌پذیر نیست، باید برای هر منبع، یک تصدیق امنیتی مستند وجود داشته باشد که توجیه کند چرا این منبع به این شکل بارگذاری می‌شود.	3

V3.7 سایر ملاحظات امنیتی مرورگر (Other Security Browser Considerations)

این بخش شامل مجموعه‌ای از کنترل‌های امنیتی و ویژگی‌های امنیتی مدرن مرورگر است که برای افزایش امنیت در سمت کلاینت (مرورگر کاربر) توصیه می‌شوند.

#	Description	Level
3.7.1	برنامه صرفاً باید از فناوری‌های سمت کلاینتی استفاده کند که همچنان پشتیبانی می‌شوند و از نظر امنیتی قابل اعتماد هستند. مثال‌هایی از فناوری‌هایی که این الزام را برآورده نمی‌کنند عبارتند از: Plugins، Flash، Shockwave، ActiveX، Silverlight، NACL، NSAPI و Applet جاوا در سمت کلاینت	2
3.7.2	برنامه فقط زمانی کاربر را به دامنه‌های hostname دی‌گر که تحت کنترل برنامه نیستند (به صورت خودکار هدایت می‌کند، که آن مقصد در یک فهرست مجاز (allowlist) قرار داشته باشد).	2
3.7.3	برنامه باید هنگام ری‌دایرکت کاربر به یک URL که خارج از کنترل برنامه است، یک پیام هشدار نشان می‌دهد و امکان لغو آن را به کاربر می‌دهد.	3
3.7.4	باید دامنه اصلی برنامه، مثلاً (site.tld) به فهرست preload عمومی HSTS اضافه شده باشد.	3
3.7.5	اگر مرورگر کاربر از ویژگی‌های امنیتی مورد انتظار برنامه پشتیبانی نمی‌کند، برنامه رفتار مناسبی داشته باشد، مثلاً هشدار دادن به کاربر یا مسدود کردن دسترسی.	3

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

• details prefix __Host- Set-Cookie

- Sheet Cheat Policy Security Content OWASP
- Project Headers Secure OWASP
- Sheet Cheat Prevention Forgery Request Cross-Site OWASP
- form submission List Preload Browser HSTS
- Sheet Cheat Prevention Clobbering DOM OWASP

Service Web and API V4

هدف

برخی ملاحظات امنیتی صرفاً برای برنامه‌های که API را برای استفاده توسط مرورگرهای وب یا مصارف دی‌گر (معمولاً مبتنی بر XML, JSON GraphQL, می باشند) ارائه می‌دهند، مطرح می‌شوند. این فصل شامل پی‌کربندی‌ها و مکانیسم‌های امنیتی مرتبطی است که باید به این نوع برنامه‌ها اعمال شوند.

توجه کنید نگارنده‌های عمومی امنیتی مانند احراز هویت، مدیریت نشست و اعتبارسنجی ورودی‌ها که در فصل‌های دی‌گر مطرح شده‌اند، در مورد API نیز صدق می‌کنند. بنابراین این فصل نباید به صورت جداگانه یا خارج از زمینه کلی استناد دارد، بررسی یا آزموده شود.

Security Service Web Generic V4.1

این بخش به ملاحظات کلی امنیتی سرویس‌های وب می‌پردازد و در نتیجه بر رعایت استانداردهای اولیه و اصول به‌داشتی امنیتی در طراحی و پیاده‌سازی سرویس‌های وب تأکید دارد.

#	Description	Level
4.1.1	اطمینان حاصل شود که در هر پاسخ HTTP که شامل Body می‌باشد، هدر Content-Type وجود داشته باشد که نوع آن، با محتوای پاسخ درستی از سرور مطابقت داشته باشد. این هدر باید شامل پارامتر charset نیز باشد تا رمزگذاری کاراکتر ایمن (مانند UTF-8 یا ISO-8859-1) را مشخص کند. این مقادیر باید مطابق با استاندارد Types Media IANA باشند؛ برای مثال، انواعی مانند، "text/" و "+xml" را در بر می‌گیرند.	1
4.1.2	اطمینان حاصل شود که تنها نقاط پایانی که برای استفاده کاربر نه‌ای از طریق مرورگر طراحی شده‌اند (User-Facing Endpoints) به صورت خودکار از HTTP به HTTPS هدایت شوند، سایر سرویس‌ها یا نقاط پایانی نباید (ری‌دایرکت شفاف، Transparent Redirect) داشته باشند. این اقدام برای جلوگیری از وضعیتی است که در آن یک کلاینت به اشتباه درخواست‌های رمزنگاری نشده (HTTP) ارسال می‌کند، اما چون این درخواست‌ها به طور خودکار به HTTPS ری‌دایرکت می‌شوند، نشست‌های داده‌ای حساس پنهان خواهد ماند.	2

#	Description	Level
4.1.3	اطمینیان حاصل شود که هدرهای HTTP که در برنامه استفاده می شود و توسط یک لایه واسطه، مانند balancer load، یک پروکسی وب یا یک سرور backend-for-frontend تنظیم شده است نمی تواند توسط کاربر نهایی بازنویسی شود. هدرهای نمونه می کنند است شامل X-Real-IP، X-Forwarded-* یا X-User-ID باشند.	2
4.1.4	اطمینیان حاصل شود که فقط متدهای HTTP که صریحاً توسط برنامه یا API آن پشتیبانی می شوند (از جمله OPTIONS در درخواست های preflight) می توانند مورد استفاده قرار گیرند و متدهای استفاده نشده مسدود شوند.	3
4.1.5	اطمینیان حاصل شود که برای درخواست های تراکنش های که بسیاری از سطح هستند یا از چندین سامانه عبور می کنند، از امضای دیجیتالی در سطح پیام استفاده شود تا اطمینیان امنیتی بیشتری علاوه بر محافظت های سطح انتقال (مانند TLS) فراهم گردد.	3

Validation Structure Message HTTP V4.2

این بخش به صورت دقیقی مشخص می کند که ساختار و هدرهای درخواست HTTP باید چه گونه اعتبارسنجی شوند تا از حملاتی مانند Request HTTP، response Smuggling، injection header splitting و service of denial از طریق یک پیام طولانی HTTP جلوگیری شود.

این الزامات برای پردازش و تولید کلی پیام های HTTP اهمیت دارند، اما به ویژه هنگام تبدیلی پیام های HTTP بین نسخه های مختلف HTTP مهم هستند.

#	Description	Level
4.2.1	اطمینیان حاصل شود که تمامی اجزای برنامه (از جمله لود بالانسرها، فایروال ها و سرورهای اپلیکیشن) مرزهای پیام های HTTP ورودی را با استفاده از مکانیزم مناسب نسخه های HTTP تعریف می کنند تا از حملات HTTP smuggling request جلوگیری شود. در HTTP/1.x اگر در Transfer-Encoding وجود داشته باشد، در Content-Length باید مطابق RFC 2616 نادیده گرفته شود. در مقابل، هنگام استفاده از HTTP/2 یا HTTP/3، اگر در Content-Length وجود داشته باشد، گزینده باید اطمینیان حاصل کند که این مقادیر با طول فریم های DATA سازگار می باشد.	2
4.2.2	اطمینیان حاصل شود که هنگام تولید پیام های HTTP، در Content-Length با طول محتوای که توسط چارچوب بندی پروتکل HTTP تعریف می شود، تداخل نداشته باشد تا از حملات smuggling request جلوگیری شود.	3
4.2.3	اطمینیان حاصل شود برنامه، پیام های HTTP/2 یا HTTP/3 با هدرهای connection-specific مانند Transfer-Encoding را به منظور جلوگیری از حملات response splitting یا injection header ارسال یا قبول نمی کند.	3

#	Description	Level
4.2.4	اطمینان حاصل شود برنامه تنه درخواست های HTTP/3 and HTTP/2 که هدرها و مقادیر آن شامل هیچ توالی CR (\r), LF (\n), CRLF (\n\r) نباشد را می پذیری تا از حملات injection header جلوگیری کنند.	3
4.2.5	اطمینان حاصل کنی که اگر برنامه (Frontend or Backend) اقدام به ساخت و ارسال درخواست های HTTP می کند از مکانیزم های sanitization ، validation و سایر مکانیزم ها استفاده کند تا از ایجاد URI ها (مانند فراهوانی API) یا فیلدهای در درخواست HTTP (مانند Authorization یا Cookie که بیش از حد مجاز، طولانی هستند، جلوگیری شود که منجر به عدم پذیری درخواست توسط مولفه دریافت کننده (هر بخشی از زیرساخت یا سامانه که درخواست HTTP را دریافت و پردازش می کند) شود. عدم رعایت این موضوع می تواند باعث وقوع حمله Service of Denial شود، ارسال یک درخواست بیش از حد طولانی (مانند یک فیلد هدر کوکی طولانی) باعث می شود سرور همواره با وضعیت خطا پاسخ دهد.	3

GraphQL V4.3

GraphQL به طور فزاینده ای در حال تبدیل شدن به روشی رایج برای توسعه ی کلاینت های داده محور است، که به صورت مستقیم به چندین سرورس مختلف در backend وابسته نیستند. این بخش به ملایحظات امنیتی ویژه مربوط به استفاده از GraphQL می پردازد.

#	Description	Level
4.3.1	اطمینان حاصل شود یکی از مکانیزم ها برای جلوگیری از حملات انکار سرورس (DoS) در لایه GraphQL یا لایه داده استفاده شده است. این مکانیزم ها عبارتند از استفاده از فهرست مجاز کوئری ها (Query Allowlist)، محدودسازی عمق کوئری ها (Depth)، محدودسازی، (Limiting) تعداد یا حجم داده (Limiting) و تحلیل هزینه کوئری (Query Cost Analysis). این اقدامات، به منظور مقابله با درخواست های پیچیده، تو در تو یا کوئری های پرهزینه که می توانند موجب مصرف بیش از حد منابع و اختلال در سرورس دهی شوند ضروری است.	2
4.3.2	اطمینان حاصل شود که قابلیت Introspection در GraphQL در محیط تولید (Production) غیرفعال شده است، مگر API مبتنی بر GraphQL به صورت صریح برای استفاده توسط اشخاص یا سامانه های ثالث (Third Parties) طراحی شده باشد.	2

WebSocket V4.4

WebSocket یی که پروتکل ارتباطی است که امکان ارتباط دوطرفه همزمان را از طریق یک اتصال TCP فراهم می کند. این پروتکل در سال ۲۰۱۱ توسط IETF در قالب RFC 6455 استاندارد سازی شد.

اگرچه WebSocket برای کار کردن از طریق پورت‌های رایج HTTP یعنی پورت 443 (HTTPS) و پورت 80 (HTTP) طراحی شده، اما پروتکل‌های مجزا از HTTP محسوب می‌شوند.

این بخش به الزامات کلیدی امنیتی می‌پردازد که برای جلوگیری از حملاتی طراحی شده‌اند که از طریق این کانال ارتباطی بلادرنگ، به‌ویژه در حوزه‌های امنیتی ارتباطات و مدیریت نشست انجام می‌گیرند.

#	Description	Level
4.4.1	اطمینان حاصل شود که تمام ارتباطات WebSocket از طریق پروتکل (TLS WSS) برقرار می‌شوند.	1
4.4.2	اطمینان حاصل شود که در هنگام Handshake اولیه‌ی HTTP برای WebSocket، مقدار Origin بررسی شود و در صورتی که با لیستی از مبداها (مجاز (allowed origins) برای برنامه مطابقت داشته باشد، اتصال پذیرفته می‌شود.	2
4.4.3	اطمینان حاصل شود که در صورتی که امکان استفاده از مکانیسم استناد دارد مدیریت نشست برنامه وجود نداشته باشد، از توکن‌های اختصای (Duplicated Tokens) برای این منظور استفاده شود که با الزامات امنیتی مدیریت نشست به‌طور کامل مطابقت داشته باشد.	2
4.4.4	اطمینان حاصل شود که توکن‌های اختصای مدیریت نشست WebSocket در هنگام انتقال از یک نشست HTTPS موجود به کانال WebSocket، از طریق نشست HTTPS احراز هویت شده قبلی، دریافت یا اعتبارسنجی می‌شوند.	2

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- Sheet Cheat Security REST OWASP
- Apollo and graphql.org from Authorization GraphQL on Resources
- Testing GraphQL Guide: Testing Security Web OWASP
- WebSockets Testing Guide: Testing Security Web OWASP

Handling File V5

هدف

استفاده از فایل‌ها می‌تواند انواع مختلف‌تری از ریسک‌ها را برای برنامه ایجاد کند، از جمله حملات انکار سرویس (DOS)، دسترسی غیرمجاز به اطلاعات، پخش فضا‌ی ذخیره‌سازی. این فصل شامل مجموعه‌ای از الزامات امنیتی برای مقابله با این تهدیدات است.

Documentation Handling File V5.1

این بخش شامل الزاماتی است که باین می کند ویژگی های مورد انتظار برای فایل های که توسط برنامه پذیرفته می شوند بایست مستندسازی شوند. این مستندسازی پیشنیازی ضروری برای طراحی و بررسی کنترل های امنیتی مرتبط با بارگذاری فایل ها است.

#	Description	Level
5.1.1	مستندات نوع فایل های مجاز، پسوندهای مورد انتظار، حداکثر اندازه هر فایل (شامل اندازه پس از استخراج) را برای هر قابلیت بارگذاری مشخص کرده باشد. همچنین، مستندات توضیح می دهند چگونه فایل ها برای دانلود و پردازش توسط کاربران نهایی ایمن سازی می شوند؛ برای مثال، رفتار برنامه هنگام شناسایی یک فایل مخرب چگونه خواهد بود.	2

Content and Upload File V5.2

قابلیت بارگذاری فایل یکی از منابع اصلی ورود فایل های غیرقابل اعتماد به سامانه می باشد. این بخش الزامات لازم را برای اطمینان از اینکه وجود، حجم یا محتوای این فایل ها نمیتواند به برنامه آسیب برساند، تشریح می کند.

#	Description	Level
5.2.1	برنامه تنها فایل های با حجم مشخص، که قادر به پردازش آن است را می پذیرد تا از کاهش عملکرد یا حملات انکار سرویس (DoS) جلوگیری شود.	1
5.2.2	هنگام پذیرش فایل توسط برنامه، چه به صورت مستقیم و چه به صورتی که آرشیو مانند فایل zip پسوند فایل با پسوند مورد انتظار مطابقت داشته باشد و اعتبارسنجی محتوای فایل صورت گیرد تا تایید شود محتوای فایل با نوع تعریف شده توسط پسوند همخوانی دارد. این فرآیند می تواند شامل موارد بررسی بایتهای ابتدای (magic bytes) فایل، انجام بازنویسی تصاویر برای حذف ساختارها و اطلاعات ناخواسته و استفاده از کتابخانه های تخصصی برای اعتبارسنجی کامل محتوای فایل باشد اما محدود به آنها نیست. در سطح L1، کافیه است این بررسی ها فقط بر روی فایل های که در فرآیندهای تجاری یا امنیتی خاص استفاده می شوند متمرکز باشد. در سطح L2 و بالاتر، این فرآیند بایست برای تمامی فایل های ورودی برنامه اجرا شود.	1
5.2.3	برنامه بایست پیش از استخراج محتوای فایل های فشرده (مانند docx، gz، zip، odt) آنها را از نظر حداکثر اندازه مجاز پس از استخراج و همچنین حداکثر تعداد فایل های درون آن بررسی می کند.	2

#	Description	Level
5.2.4	برنامه فضای مشخصی برای اندازه فایل و تعداد فایل‌های مجاز برای هر کاربر اعمال می‌کند تا از این طریق، یک کاربر نتواند با بارگذاری تعداد زیادی فایل یا فایل‌های بیش از حد بزرگ، فضای ذخیره‌سازی را پر کند.	3
5.2.5	برنامه اجازه بارگذاری فایل‌های فشرده که حاوی لینک‌های نمادین (symlinks) هستند را نمی‌دهد، مگر این‌که این رفتار به طور مشخص مورد نیازی باشد. در چنین شرایطی باید یک فهرست مجاز (allowlist) از مسیرها یا فایل‌های که می‌تواند برای آن‌ها لینک نمادین ایجاد شود، اعمال و اعتبارسنجی گردد.	3
5.2.6	برنامه تصاویر بارگذاری شده با اندازه پیکسل‌های بزرگتر از حد مجاز را رد کند تا از حملات attacks flood pixel جلوگیری شود.	3

Storage File V5.3

این بخش شامل الزامات جلوگیری از اجرای غیرمجاز فایل‌ها پس از بارگذاری، شناسایی محتوای خطرناک و جلوگیری از استفاده از داده‌های غیرمعتبر برای کنترل مکان ذخیره‌سازی فایل‌ها است.

#	Description	Level
5.3.1	فایل‌هایی که از ورودی‌های غیرمعتبر بارگذاری یا تولید می‌شوند و در پوشه‌های عمومی قرار می‌گیرند، هنگام دسترسی مستقیم از طریق یک درخت‌واست HTTP، به عنوان کالجرای سمت سرور اجرا نشوند.	1
5.3.2	هنگام ایجاد مسیرهای فایل برای عملیات مختلف، به جای استفاده از نام فایل‌های ارسال شده توسط کاربر، از داده‌های تولید شده داخلی یا منابع مورد اعتماد استفاده شود. و در صورتی که استفاده از نام فایل‌ها یا متادیتای ارسال شده توسط کاربر ضروری باشد، می‌بایست validation و Sanitization اعمال شود. این کار برای جلوگیری از حملات Path Request Server-Side (RFI) Inclusion (LFI, File Remote or Local Traversal، Forgery (SSRF) می‌باشد.	1
5.3.3	پردازش فایل‌های سمت سرور، مانند استخراج فایل‌های فشرده، اطلاعات مسیریارائه شده توسط کاربر را نادیده می‌گیرد تا از آسیب‌پذیری‌های مانند slip zip جلوگیری شود.	3

Download File V5.4

این بخش شامل الزامات برای کاهش ریسک‌ها هنگام ارائه فایل‌ها برای دانلود است، از جمله حملات Path Traversal و حملات Injection می‌باشد. همچنین اطمینان حاصل می‌شود که فایل‌ها حاوی محتوای خطرناک

نمایش ندهد.

#	Description	Level
5.4.1	برنامه نام فایل ارسال شده توسط کاربر را چه در قالب JSON، JSONP یا پارامتر URL، اعتبارسنجی می کند یا نادیده می گیرد، و در پاسخ HTTP، نام فایل را به طور صریح در فیلد Content-Disposition هدر مشخص می نماید.	2
5.4.2	نام های فایل های ارائه شده (مانند فیلدهای هدر پاسخ HTTP یا پیوست های ایمیل)، به درستی encoded یا sanitized شده اند (مطابق RFC 6266 تا ساختار سند حفظ شده و از حملات تزریق چلوگری شود).	2
5.4.3	فایل های دریافت شده از منابع غیرقابل اعتماد توسط آن تی ویروس اسکن شود تا از ارائه محتوای مخرب شناخته شده چلوگری شود.	2

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- Sheet Cheat Upload File OWASP
- read file arbitrary for symlinks using of Example
- Wikipedia from Bytes "Magic of Explanation"

Authentication V6

Objective Control

احراز هویت فرآیند تأیید یا اثبات اصلیت یک فرد یا دستگاه می باشد. این فرآیند شامل بررسی صحت ادعای ارائه شده توسط یک شخص یا یک دستگاه بوده به گونه ای که در برابر جعل هویت مقاوم باشد و از بازیابی یا رهگیری گفرواژه ها چلوگری کند.

استاندارد SP NIST 800-63 یک استاندارد مدرن و مبتنی بر شواهد است که برای سازمان های سراسر جهان ارزشمند محسوب می شود و برای سازمان های دولتی ایالات متحده و نهادهای که با این سازمان ها تعامل دارند، اهمیت ویژه ای دارد.

اگرچه بسیاری از الزامات مطرح شده در این فصل بر اساس بخش دوم این استاندارد (با عنوان SP NIST 800-63B تحت عنوان «راهنمای هویت دیجیتالی - احراز هویت و مدیریت چرخه عمر») بنا شده اند، تمرکز این فصل بر تهی دات رایج و وضع فهای پرتکار در احراز هویت می باشد. این فصل تلاش برای پوشش جامع تمامی نکات استاندارد نمی کند. در مواردی که رعایت کامل استاندارد SP NIST 800-63 ضروری باشد، لطفاً به متن کامل این استاندارد مراجعه فرمایید.

همچنین، ممکن است اصطلاحات به کاررفته در SP NIST 800-63 با اصطلاحات عمومی تر تفاوت داشت داشته باشد؛ به همین دلیل، این فصل عمدتاً از واژگان رایج تر و قابل فهم تر استفاده می کند تا وضوح مطلب افزایش یابد.

یکی از ویژگی‌های رایج در سامانه‌های پیشرفته تر، تطبیق پذیری مراحل احراز هویت با توجه به عوامل مختلف ریسک است. این ویژگی در فصل Authorization مورد بررسی قرار می‌گیرد، چرا که این مکانیزم‌ها باید در تصمیم‌گیری‌های مربوط به Authorization نیز لحاظ شوند.

Documentation Authentication V6.1

این بخش شامل الزامات مربوط به مستندسازی احراز هویت است که باید برای یک برنامه کاربردی نگهداری شود. این مستندسازی برای پیاده‌سازی و ارزیابی نحوه پی‌کربندی کنترل‌های مرتبط با احراز هویت از اهمیت بالایی برخوردار است.

#	Description	Level
6.1.1	مستندات برنامه، نحوه استفاده از کنترل‌های مانند محدودیت نرخ (Rate Limit)، مقابله با خودکارسازی (Anti-Automation) و پاسخ تطبیقی (Adaptive Response) را برای دفاع در برابر حملات مانند Stuffing Credential و حملات Force Brute Password به وضوح تعریف کرده باشد. مستندات باید به وضوح نحوه پی‌کربندی این کنترل‌ها را تشریح کرده و نشان دهند که چگونه از قفل شدن مخرب حساب‌های کاربری جلوگیری می‌شود.	1
6.1.2	یک لیست از کلمات خاص مرتبط با زمینه سازمانی (Context-Specific) مستندسازی شود تا از به کارگیری آن‌ها در رمزهای عبور جلوگیری شود. این فهرست می‌تواند شامل نام سازمان، نام محصول، شناسه سامانه، نام کد پروژه، نام دپارتمان یا نقش‌های سازمانی و موارد مشابه باشد.	2
6.1.3	در صورت وجود مسیری‌های متعدد احراز هویت در برنامه کاربردی، تمامی این مسیری‌ها به همراه کنترل‌های امنیتی و سطح قدرت احراز هویت مورد نیاز، مستندسازی شده‌اند. لازم است این کنترل‌ها به صورت یکسان و سازگار در تمامی مسیری‌های احراز هویت اجرا شوند.	2

Security Password V6.2

کلمات عبور، که توسط SP NIST 800-63 به عنوان “رمزهای به خاطر سپرده شده” (Memorized Secrets) شناخته می‌شوند، شامل کلمات عبور، عبارتهای عبور، کدهای PIN، الگوهای بازگشایی (unlock Patterns)، انتخاب تصویر صحیح (مثلاً انتخاب بچه گربه صحیح از میان چند تصویر) هستند. این‌ها معمولاً به عنوان “چیزی که شما می‌دانید” در نظر گرفته می‌شوند و اغلب به عنوان یکی از مکانیزم احراز هویت که عملی است فاده می‌شوند.

بر این اساس، این بخش شامل الزامات و توصیف‌هایی برای ایجاد و مدیریت امن کلمات عبور می‌باشد. بیشترین الزامات در سطح L1 مطرح شده‌اند، چرا که در این سطح نقش کلیدی دارند. از سطح L2 به بعد، استفاده از احراز هویت چندعاملی الزامی می‌باشد، که در آن گفروازه می‌تواند یکی از عوامل تشکیلی شده باشد.

الزامات این بخش عمدتاً به بند § 5.1.1.2 از راهنمای NIST's Guidance می‌باشد.

#	Description	Level
6.2.1	کلمات عبور تنظیم شده توسط کاربر حداقل ۸ کاراکتر طول داشته باشد. هرچند حداقل ۱۵ کاراکتر به شدت توصیه می شود.	1
6.2.2	کاربران قادر به تغیری گفرواژه خود باشند.	1
6.2.3	هنگام تغیری کلمه عبور، گفرواژه فعلی و گفرواژه جدید توسط کاربر تکمیل شود.	1
6.2.4	کلمات عبوری که در حین ثبت نام حساب کاربری یا تغیری کلمه عبور ارسال می شوند باید یک فهرست در دسترس از حداقل ۳۰۰۰ کلمه عبور رایج (که) با سیاست های کلمه عبور برنامه (مانند حداقل طول) سازگاری دارند بررسی شوند.	1
6.2.5	کلمات عبور با هر ترکیبی قابل استفاده باشند، بدون اینکه قوانینی برای محدود کردن نوع کاراکترهای مجاز وجود داشته باشد. نباید هیچ گونه الزامی برای تعداد حداقل کاراکترهای حروف بزرگ یا کوچک، اعداد یا کاراکترهای خاص وجود داشته باشد.	1
6.2.6	فیلدهای ورودی کلمه عبور از نوع "type=password" استفاده می کنند تا ورودی ها پنهان شوند. برنامه ها می توانند به کاربر این امکان را بدهند که موقتاً کلمه عبور کامل یا آخرین کاراکتر تایپ شده را مشاهده کند.	1
6.2.7	عمل کرد "چسباندن"، (paste) ابزارهای کمکی گفرواژه مرورگر و مدیریت کننده های گفرواژه خارجی مجاز باشند.	1
6.2.8	برنامه کلمه عبور کاربر را دقیقاً همانطور که از کاربر دریافت شده است، بدون هیچ گونه تغیری را می ماند کوتاه سازی (truncation) یا تبدیلی حروف بزرگ و کوچک (case transformation)، بررسی کند.	1
6.2.9	استفاده از گفرواژه های با حداقل طول ۶۴ کاراکتر مجاز باشد.	2
6.2.10	کلمه عبور کاربر تا زمانی که مشخص شود که به خطر افتاده است یا کاربر آن را تغیری دهد، معتبر باقی بماند. برنامه نباید الزام به تغیری دوره ای گفرواژه را داشته باشد.	2
6.2.11	فهرست مستند شده ای از کلمات خاص مرتبط با زمینه سازمانی برای جلوگیری از ایجاد کلمات عبور آسان قابل حدس، مورد استفاده قرار گیرد.	2
6.2.12	کلمات عبور وارد شده در حین ثبت نام حساب کاربری یا تغیری کلمه عبور با مجموعه ای از کلمات عبور افشا شده بررسی شوند.	2

Security Authentication General V6.3

این بخش شامل الزامات عمومی برای امنیت مکانی زمینه ای احراز هویت و همچنین تعین انتظارات مربوط به سطوح مختلف امنیتی می باشد. در سطح L2، سازمان ها باید الزاماً از احراز هویت چندعاملی (MFA) استفاده کنند. در سطح L3، استفاده از احراز هویت مبتنی بر سخت افزار الزامی می باشد

که باید درونی یک محیط اجرای مورد اعتماد و قابل گواهی (Trusted Environment Execution (TEE) - TEE انجام شود. این می تواند شامل کلیدهای عبور وابسته به دستگاه (Device-Bound Passkeys)، احرازکننده های با سطح اطمینان بالا مطابق با استاندارد eIDAS (سطح LoA)، احرازکننده های دارای سطح اطمینان AAL3 طبق استاندارد NIST، یا مکانیسم های معادل دی گرباشد.

اگرچه این الزامات در مقایسه با بسیاری از چارچوب های دی گر، رویکردی سخت گیری رانه نسبت به MFA دارند، اما این سخت گیری برای ارتقای سطح امنیت کاربران حیاتی می باشد. هرگونه تلاش برای کاهش این الزامات بایستی یک برنامه شفاف و دقیقی برای کاهش ریسک های مرتبط با احراز هویت همراه باشد، به طوری که راهنمای ها و پژوهش های ارائه شده توسط NIST نیز در آن لحاظ شده باشد.

توجه داشته باشید که در زمان انتشار این سند، استاندارد SP NIST 63-800 استفاده از ای میل را به عنوان یکی که مکانیسم احراز هویت قابل قبول نمی داند).copy archived.

الزامات این بخش به بخش های مختلف از راهنمای NIST's Guidance مربوط می شوند، از جمله بندهای § 4.2.1، § 4.3.1، § 5.2.2 و § 6.1.2.

#	Description	Level
6.3.1	کنترل های لازم برای جلوگیری از حملاتی مانند Stuffing Credential و force brute password طبق مستندات امنیتی برنامه پیاده سازی شوند.	1
6.3.2	حساب های کاربری پیش فرض (مانند "admin"، "root" یا "sa") در برنامه وجود نداشته باشند یا غیرفعال شده باشند.	1
6.3.3	برای دسترسی به برنامه، از یکی که مکانیسم احراز هویت چندعاملی (MFA) یا ترکیبی از مکانیسم های احراز هویت تک عاملی استفاده شود. برای سطح L3، حداقل یکی از عوامل احراز هویت بایست مبتنی بر سخت افزار باشد در برابر حملات فیشینگ، مقاومت در برابر نفوذ و جعل هویت را فراهم کند، درحالی که با اقدام آگاهانه کاربر (مانند فشردن دکمه روی کلیدی سخت افزاری FIDO یا تلفن همراه) قصد کاربر برای احراز هویت را نیز تأیید نماید. هرگونه تسهیل یا تعدیل در این الزام، مستلزم ارائه دلایل مستند و مجوعه ای جامع از کنترل های جبرانی است.	2
6.3.4	تأیید کنیدی که در صورت وجود مسیری های متعدد احراز هویت در برنامه، هیچ مسیری احراز هویت مستند سازی نشده ای وجود نداشته باشد و کنترل های امنیتی و سطح قدرت احراز هویت در تمامی مسیرها به صورت یکسان و سازگار اعمال شوند.	2
6.3.5	کاربران از تلاش های مشکوک برای احراز هویت (موفق یا ناموفق) مطلع شوند. این اطلاع رسانی می تواند شامل تلاش برای احراز هویت از موقعیتی مکانی یا کلاینت غیرمعمول، احراز هویت ناقص (تنها موفقیتی یکی از عوامل)، تلاش برای احراز هویت پس از یک دوره طولانی عدم فعالیت، یا احراز هویت موفق پس از چندین تلاش ناموفق باشد.	3
6.3.6	ایمیل به عنوان مکانیسم احراز هویت، چه به صورت تک عاملی و چه به عنوان یکی از عوامل احراز هویت چندعاملی، استفاده نمی شود.	3

#	Description	Level
6.3.7	کاربران پس از به روزرسانی اطلاعات احراز هویت خود، مانند بازنشانی گفروازه یا تغیری نام کاربری یا آدرس ایمیل، مطلع شوند.	3
6.3.8	با توجه به پیام‌های خفا، کدهای پاسخ HTTP یا تفاوت در زمان پاسخ‌دهی، نتوان کاربران معتبر را از طریق تلاش‌های ناموفق احراز هویت شناسایی کرد. همچنین، عمل کردهای ثبت نام و بازیابی گفروازه نیز باید از این نوع افشای اطلاعات محافظت شده باشند.	3

Recovery and Lifecycle Factor Authentication V6.4

عوامل احراز هویت می‌توانند شامل کلمه عبور، توکن‌های نرم‌افزاری، توکن‌های سخت‌افزاری و دستگاه‌های بی‌سیمتری باشند. مدیریتی امن چرخه عمر این مکانیزم‌ها برای حفظ امنیتی برنامه اهمیت بسیاری دارد و این بخش شامل الزامات مرتبط با این موضوع می‌باشد. الزامات این بخش عمدتاً مربوط به بندهای § 5.1.1.2 و § 6.1.2.3 از راهنمای Guidance NIST's می‌باشد.

#	Description	Level
6.4.1	کلمه عبور اولیه یا کدهای فعال‌سازی تولید شده توسط سیستم، به صورت تصادفی و ایمن تولید شوند، با سیستمی است فعلی رمز عبور مطابقت دارند، و پس از مدت زمان کوتاهی یا پس از اولین استفاده منقضی می‌شوند. این رمزهای اولیه نباید به عنوان رمز عبور بلندمدت مورد استفاده قرار گیرند.	1
6.4.2	راهنمای کلمه عبور یا احراز هویت مبتنی بر دانش (که به عنوان "سوالات امنیتی" شناخته می‌شوند) در سیستم وجود ندارد.	1
6.4.3	فرآیند امنیتی برای بازنشانی کلمه عبور فراموش شده پیاده‌سازی شده است که هیچ‌یک از مکانیزم‌های احراز هویت چندعاملی فعال را دور نزنند.	2
6.4.4	در صورت دست رفتن یکی از عوامل احراز هویت چندعاملی، فرآیند اثبات هویت با همان سطحی که در زمان ثبت نام انجام شده است، اجرا شود.	2
6.4.5	دست‌ورالعمل‌های تمدید مکانیزم‌های احراز هویت دارای تاریخ انقضا برای کاربران به موقع ارسال می‌شود تا پیش از انقضای مکانیزم فعلی فرصت کافی برای تمدید وجود داشته باشد، و در صورت لزوم، یادآورهای خودکار نیز تنظیم و ارسال شده باشند.	3
6.4.6	کاربران ادمن می‌توانند فرآیند بازنشانی کلمه عبور را برای کاربران عادی آغاز کنند، اما اجازه تغیری مستقیم یا انتخاب رمز عبور برای کاربران را ندارند. این کنترل برای جلوگیری از دسترسی ادمن به رمز عبور کاربران طراحی شده است.	3

requirements authentication Multi-factor General V6.5

این بخش راهنمای کلی ارائه می‌دهد که برای انواع مختلف مکانیسم‌های احراز هویت چندعاملی (MFA) قابل استفاده است.

مکانیسم‌های بررسی شده شامل موارد زیر هستند:

- Secrets Lookup (فرایند یافتن یا بازیابی اطلاعات محرمانه)
- رم‌های عبوری یکبار مصرف مبتنی بر زمان (TOTPs)
- مکانیسم‌های خارج از باند (Out-of-Band Mechanisms)

Secret Lookup که‌های هستند که به صورت از پیش تولید شده ارائه می‌شوند و مشابه با که‌های مجوز تراکنش (TAN)، که‌های بازیابی حساب‌های شبکه‌های اجتماعی یا شبکه‌های از مقادیر تصادفی می‌باشند. این نوع مکانیسم به عنوان “چیزی که در اختیار دارید” در نظر گرفته می‌شود، زیرا که‌ها عمداً غیرقابل حفظ کردن هستند و نیازی به ذخیره‌سازی ندارند.

رم‌های یکبار مصرف چندعاملی (Multi-factor) (TOTPs) مشابه رم‌های یکبار مصرف تکعاملی هستند، اما برای ایجاد رمز نهایی (OTP) نیازی به وارد کردن کد PIN معتبر، بازکردن بابی‌ومتری، وارد کردن USB یا چفت‌سازی NFC، یا مقدار اضافی دی‌گری (مانند ماشین حساب‌های امضای تراکنش) دارند.

جزئیات مربوط به مکانیسم‌های خارج از باند (Out-of-Band) در بخش بعدی ارائه خواهد شد.

الزامات موجود در این بخش‌ها عمدتاً مربوط به بندهای § 5.1.2، § 5.1.3، § 5.1.4.2، § 5.1.5.2، § 5.2.1 و § 5.2.3 از راهنمای NIST's Guidance می‌باشند.

#	Description	Level
6.5.1	Secrets Lookup درخواس‌تها یا که‌های احراز هویت خارج از باند (Out-of-Band) و رم‌های یکبار مصرف مبتنی بر زمان (TOTP) تنه‌ای یکبار قابل استفاده باشند و پس از استفاده موفق، مجدداً قابل استفاده نباشند.	2
6.5.2	در صورت ذخیره‌سازی Secrets Lookup در سمت Backend برنامه، اگر مقدار آن‌روپی کمتر از ۱۱۲ بیت باشند (معدل ۱۹ کاراکتر تصادفی الفبای-عددی یا ۳۴ رقم تصادفی)، باید بایک الگوریتم تایید شده برای ذخیره رمز عبور و با استفاده از یک Salt تصادفی ۳۲ بیتی هش شوند. در صورتی که آن‌روپی کد ۱۱۲ بیت یا بیشتر باشد، می‌توان از یک تابع هش استاندارد برای ذخیره‌سازی استفاده کرد.	2
6.5.3	Secrets Lookup که‌های احراز هویت خارج از باند (out of band) و تولید رم‌های یکبار مصرف مبتنی بر زمان (TOTP) همگی با استفاده از یک تولیدکننده عدد شبه تصادفی رمزنگاری شده (CSPRNG) تولید شده باشند تا از تولید مقادیر قابل پیش‌بینی جلوگیری شود.	2

#	Description	Level
6.5.4	Secrets Lookup و کدهای احراز هویت خارج از باند (out of band) حداقل دارای ۲۰ بیت آنتروپی باشند (مع مولاً ۴ کاراکتر تصادفی الفبای-عدد دی ۱۰۶ رقم تصادفی برای رسیدن به این میزبان آنتروپی کافی است).	2
6.5.5	درخواست‌ها، کدهای توکن‌های احراز هویت خارج از باند (Out-of-Band) و همچنین رم‌های عبوری که بار مصرف مبتنی بر زمان (TOTP) دارای طول عمر مشخص باشند. حداکثر طول عمر برای درخواست‌های خارج از باند باید ۱۰ دقیقه و برای TOTP حداکثر ۳۰ ثانیه باشد.	2
6.5.6	امکان لغو (Revoke) هر عامل احراز هویت، از جمله دست‌گاه‌های فیزیکی، در صورت سرقت یا مفقود شدن وجود داشته باشد.	3
6.5.7	مکانیسم‌های احراز هویت بی‌ومتری فقط به عنوان عامل دوم و در کنار یکی از عوامل «چیزی که در اختیار داری» یا «چیزی که کاربر می‌داند» مورد استفاده قرار می‌گیرند.	3
6.5.8	اعتبارسنجی رم‌های عبوری که بار مصرف مبتنی بر زمان (TOTP) بر اساس زمان از یک سرویس قابل اعتماد تمام‌انجام می‌شود و نه بر اساس زمان ارائه‌شده توسط کلاینت یا منابع غیرقابل اعتماد.	3

mechanisms authentication Out-of-Band V6.6

این نوع احراز هویت معمولاً شامل ارتباط سرور احراز هویت با یکی از دستگاه‌های فیزیکی از طریق یک کانال ثانویه امن است. به عنوان مثال، ارسال Notifications Push به دستگاه‌های تلفن همراه. این نوع از مکانیسم احراز هویت به عنوان «چیزی که در اختیار داری» در نظر گرفته می‌شود.

مکانیسم‌های ناامن احراز هویت خارج از باند نظیر ایمیل و VOIP مجاز نیستند. همچنین، احراز هویت از طریق خطوط تلفن عمومی (PSTN) و پیام‌ک (SMS) در حال حاضر توسط NIST به عنوان مکانیسم‌های «محدود شده» ("restricted" mechanisms authentication) دست‌بندی شده‌اند و باید به به تدریج با مکانیسم‌های مانده رمزی که بار مصرف مبتنی بر زمان (مکانیسم‌های رمزنگاری شده یا روش‌های مشابه امن‌تر جایگزین شوند. استاندارد SP NIST 800-63B در بند § 5.1.3.3 توصیه می‌کند که در صورت ضرورت پشتیبانی از احراز هویت خارج از باند تلفنی یا پیام‌کی، باید ریس‌ک‌های مانده تعویض دست‌گاه، تعویض سیستم کارت، انتقال شماره یا رفتارهای غیرعادی دیگر مدنظر قرار گیرد. هرچند این بخش از چارچوب ASVS الزام‌صریحی برای رعایت این موارد ندارد، اما عدم توجه به این ملاحظات امنیتی در یک اپلیکیشن حساس با سطح امنیتی L2 یا L3 باید به عنوان یک غلایت هشدار جدی تلقی شود.

توجه داشته باشید که NIST اخیراً راهنمایی‌هایی ارائه کرده است که استفاده از Notifications Push را نامطلوب می‌داند. هرچند این بخش از ASVS به این موضوع اشاره نکرده است، آگاهی از خطرات ناشی از حمله Bombing Push اهمیت بالایی دارد.

#	Description	Level
6.6.1	مکانی‌زم‌های احراز هویت مبتنی بر شبکه تلفن عمومی (PSTN) برای ارسال رم‌زهای یک‌بار مصرف (OTP) از طریق تماس تلفنی یا پیام‌ک‌ت‌ها در صورتی که شماره تلفن قبلاً اعتبارسنجی شده باشد، یا روش‌های جایگزین قوی‌تر (رم‌زهای عبوری یک‌بار مصرف مبتنی بر زمان) در دسترس باشند ارائه می‌شوند و سرویس‌دهنده باید کاربران را از ریس‌ک‌های امنیتی این روش‌ها مطلع سازد. برای اپلی‌کیشن‌های سطح L3، استفاده از تماس تلفنی و پیام‌ک به عنوان گزین‌ه، مجاز نیست.	2
6.6.2	درخواست‌ه، که‌ها یا توکن‌های احراز هویت خارج از بانده (Out-of-Band) به درخواست احراز هویت اصلی که برای آن تولید شده‌اند متصل باشند و قابل استفاده برای درخواست‌های قبلی یا بعدی نباشند.	2
6.6.3	مکانی‌زم‌های احراز هویت خارج از بانده (Out-of-Band) مبتنی بر کده در برابر حملات force brute از طریق اعمال کنترل محدودیت نرخ (Rate Limiting) محافظت شوند. همچنین، استفاده از کده با حداقل ۶۴ بیت آنتروپی مدنظر (Entropy) قرار گیرد.	2
6.6.4	در صورت استفاده از Notifications Push جهت احراز هویت چندعاملی، محدودیت نرخ (Rate Limiting) به منظور جلوگیری از حملات Bombing Push اعمال شده باشد. استفاده از تطبیق عدد (Number Matching) نیز می‌تواند به کاهش این ریس‌ک کمک کند.	3

mechanism authentication Cryptographic V6.7

مکانی‌زم‌های احراز هویت رمزنگاری شده شامل کارت‌های هوشمند یا کلیدهای FIDO هستند، که در آن‌ها کاربر باید دست‌گاه رمزنگاری را به رایانه متصل کرده یا با آن چفت کند تا فرآیند احراز هویت تکمیل شود. در این روش، سرور احراز هویت یک عدد تصادفی (Nonce) چالشی به دست‌گاه یا نرم‌افزار رمزنگاری ارسال می‌کند و دست‌گاه یا نرم‌افزار بر اساس کلید رمزنگاری securely ذخیره شده، پاسخی را محاسبه می‌کند. الزامات این بخش راهنمای‌های خاص پیاده‌سازی این مکانی‌زم‌ها را ارائه می‌دهد و راهنمای‌های مربوط به الگوریتم‌های رمزنگاری در فصل «رمزنگاری» آورده شده است.

در مواردی که کلیدهای مشترک یا کلیدهای محرمانه برای احراز هویت رمزنگاری شده مورد استفاده قرار می‌گیرند، لازم است این کلیدها به همان روشی که سایر اسرار سیستم مدیریت می‌شوند، نگهداری شوند. همان‌طور که در بخش «Secret» «Management» از فصل «Configuration» مستندسازی شده است.

الزامات این بخش عمده‌تاً مربوط به بند § 5.1.7.2 از راهنمای Guidance NIST's می‌باشد.

#	Description	Level
6.7.1	گواهی‌نامه‌های مورد استفاده برای تأیید ادعاهای احراز هویت رمزنگاری شده به گونه‌ای ذخیره شوند که از تغیری در دست‌کاری آن‌ها محافظت شود.	3
6.7.2	مقدار چالش (Challenge) (Nonce) حداقل ۶۴ بیت طول داشته باشد و از نظر آماری یکتا باشد یا حداقل در طول عمر دست‌گاه رمزنگاری منحصر به فرد باقی بماند.	3

Provider Identity an with Authentication V6.8

ارائه‌دهندگان هویت (Providers (Identity یا (IdPs) هویت فدرال شده برای کاربران فراهم می‌کنند. کاربران اغلب بیش از یک هویت با چندین ارائه‌دهنده هویت دارند، مانند هویت سازمانی که از طریق Identity Ping Okta، AD، Azure یا Google مدیریته می‌شود، یا هویت مصرف‌کننده که از طریق شبکه‌های اجتماعی مانند Google، Facebook، Twitter یا WeChat ایجاد می‌شود. این فهرست تأیید یا حمایت از این شرکت‌ها یا خدمات نیست، بلکه صرفاً تشویقی برای توسعه‌دهندگان است تا واقعیتهای وجود چندین هویت برای بسیاری از کاربران را در نظر بگیرند. سازمان‌ها باید با توجه به پروفایل ریسک و قدرت اثبات هویت ارائه‌دهنده هویت، به ادغام با هویت‌های موجود کاربران فکر کنند. به عنوان مثال، سازمان‌های دولتی به احاطه مال زیاد هویت شبکه‌های اجتماعی را به عنوان روش ورود به سیستم‌های حساس قبول نمی‌کنند، زیرا ایجاد هویت‌های جعلی یا موقت در این فضا ساده است، در حالی که شرکت‌های فعال در حوزه بازی‌های موبایل ممکن است برای افزایش تعداد بازی‌کنان فعال خود نیازی به ادغام با پلتفرم‌های اصلی شبکه‌های اجتماعی داشته باشند.

استفاده امن از ارائه‌دهندگان هویت خارجی نیازمند پی‌کردنی و تأیید دقیقی است تا از جعل هویت یا ادعاهای ساختگی جلوگیری شود. این بخش شامل الزامات لازم برای مقابله با این ریسک‌ها می‌باشد.

#	Description	Level
6.8.1	اگر برنامه از چند ارائه‌دهنده هویت (IdP) پشتیبانی می‌کند، امکان جعل هویت کاربر از طریق یک ارائه‌دهنده هویت دی‌گروچود نداشته باشد (برای مثال، با استفاده از شناسه کاربری یکسان). ممکن است اسم استاندارد برای جلوگیری از این تهدید، این است که برنامه، کاربران را با استفاده از ترکیب شناسه IdP (به عنوان فضای نام یا namespace) و شناسه کاربر در آن، IdP ثبت و شناسایی کند.	2
6.8.2	وجود ویکی‌پارچگی امضاهای دیجیتالی بر روی تصدیق‌های (تقاضاهای ادعاهای) احراز هویت برای مثال بر روی (توکن‌های JWT یا SAML assertions) همواره اعتبارسنجی می‌شود و هرگونه تصدیقی که فاقد امضا یا دارای امضای نامعتبر باشد، رد می‌گردد.	2

#	Description	Level
6.8.3	(تقاضاها یا ادعاهای) تصدیقه‌های SAML به صورت منحصر به فرد پردازش شده و تنها یکی بار در بازه‌ی اعتبار خود استفاده می‌شوند تا از حملات بازپخش (Replay Attacks) جلوگیری گردد.	2
6.8.4	اگر برنامه‌ای از یکی از ارائه‌دهنده‌های هویت (IdP) جداگانه استفاده می‌کند و انتظار قدرت، روش‌های تازگی احراز هویت خاصی را برای عمل کرده‌ای خاص دارد، برنامه این موضوع را با استفاده از اطلاعات برگردانده شده توسط IdP تأیید می‌کند. به عنوان مثال، اگر از OIDC استفاده شود، این کار می‌تواند با اعتبارسنجی ادعاهای (Claims) موجود در توکن شناسایی (ID Token) مانند «acr» و «auth_time» (در صورت وجود) انجام شود. اگر ارائه‌دهنده‌ی هویت این اطلاعات را ارائه نکند، برنامه باید یک روش جایگزین مستندسازی شده داشته باشد که فرض می‌کند حداقل سطح احراز هویت استفاده شده است (مثلاً احراز هویت تک‌عاملی با نام کاربری و گفواژه).	2

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- Guidelines Identity Digital - 800-63 SP NIST •
- Management Lifecycle and Authentication - 800-63B SP NIST •
- FAQ 800-63 SP NIST •
- Authentication for Testing Guide: Testing Security Web OWASP •
- Sheet Cheat Storage Password OWASP •
- Sheet Cheat Password Forgot OWASP •
- Sheet Cheat Questions Security Using and Choosing OWASP •
- Matching "Number on Guidance CISA •
- Alliance FIDO the on Details •

Management Session V7

هدف

مکانیسم‌های مدیریتی نشست (Session Management Mechanisms) به برنامه‌ها این امکان را می‌دهند که تعاملات کاربر و دستگاه را در طول زمان، حتی هنگام استفاده از پروتکل‌های stateless (مانند HTTP) با یک گرم‌رتب بطن کنند. برنامه‌های مدرن ممکن است از چندین توکن نشست (Session To-ken) با ویژگی‌ها و اهداف متفاوت استفاده کنند. یکی سیستم مدیریتی نشست ایمن، سیستمی است که از به دست آوردن، استفاده یا سوءاستفاده مهاجم از نشست قربانی جلوگیری می‌کند. برنامه‌های نشست را نگهداری می‌کنند باید اطمینان حاصل کنند که الزامات سطح بالای زیر در مدیریتی نشست رعایت شده‌اند:

- نشسته‌ها برای هر کاربر (شخص) منحصر به فرد هستند و نمی‌توان آنها را حدس زد یا به اشتراک گذاشت.
- نشسته‌ها زمانی که دی‌گرمورد نیازی نیستند، باطل می‌شوند و در دوره‌ای عدم فعالیت، منقضی می‌گردند.

بسیاری از الزامات مطرح‌شده در این فصل به کنترل‌های انتخاب‌شده از دست‌ورالعمل‌های هویتی دیجیتالی SP NIST 800-63 مربوط می‌شوند و بر تهدیدات رایج و ضعف‌های متداول در احراز هویت که معمولاً مورد سوءاستفاده قرار می‌گیرند، تمرکز دارند.

توجه داشته باشید که الزامات مربوط به چزئیات پیاده‌سازی خاص برخی از مکانیسم‌های مدیریتی نشسته را می‌توان در منابع دی‌گری یافت:

- HTTP Cookie ها یکی از مکانیسم‌های امن سازی توکن‌های نشسته هستند. الزامات امنیتی خاصی برای کوکی‌ها در بخش "Web Frontend Security" قرار داده شده است.
- توکن‌های Self-contained به عنوان روشی برای نگه‌داری نشسته‌ها بطور مکرر مورد استفاده قرار می‌گیرند. الزامات امنیتی خاصی در بخش "Self-contained Tokens" قرار داده شده است.

Documentation Management Session V7.1

هیچ الگویی واحدی که برای تمامی برنامه‌های کاربردی مناسب باشد، وجود ندارد. بنابراین، تعریف مرزها و محدوده‌های عمومی که برای تمامی موارد مناسب باشد غیرممکن است. یکی از تحلیل‌های ک با تصمیمات امنیتی هستند شده مرتبط با مدیریت نشسته باید به عنوان یکی از برای پیاده‌سازی و ارزیابی انجام شود. این امر تضمین می‌کند که سیستم مدیریت نشسته با الزامات خاص آن برنامه کاربردی طراحی شده است.

فارغ از اینکه از مکانیسم نشسته stateful یا stateless استفاده شود، تحلیل باید به صورت کامل و مستند انجام شود تا نشان دهد راه‌حل انتخاب‌شده توانایی برآورده کردن تمامی الزامات امنیتی مرتبط را دارد. همچنین باید تعامل با هرگونه مکانیسم ورود یک‌پارچه (Single Sign-On) یا SSO که در حال استفاده است، مورد توجه قرار گیرد.

#	Description	Level
7.1.1	بازه عدم فعالیت نشسته کاربر و بیشینه طول عمر نشسته قبل از احراز هویت مجدد، در ترکیب با سایر کنترل‌ها مستند و متناسب سازی شود. مستند سازی انجام شده باید شامل توجیه برای هر انحراف (نقض) از الزامات احراز هویت مجدد SP NIST 800-63B باشد.	2
7.1.2	در مستند باید تعداد نشسته همزمان مجاز برای یکی کاربر و همچنین رفتار و احوال مناسب برای زمانی که این تعداد به حداکثر رسید مشخص شود.	2

#	Description	Level
7.1.3	تمام سیستم‌های که نشسته‌ای کاربر را به عنوان بخشی از یک اکوسیستم مدیریت هویت فدرال (مانند سیستم‌های SSO) ایجاد و مدیریت می‌کنند، باید در کنار سایر کنترلهای مربوط به طول عمر نشسته، خاتمه آنها و هرگونه شرایطی که نیازی به احراز هویت مجدد دارد، مستند شوند.	2

Security Management Session Fundamental V7.2

این بخش الزامات موردنیاز نشسته‌ای امن را از طریق بررسی امنیتی تولید و اعتبارسنجی توکن‌ها احصا می‌نماید.

#	Description	Level
7.2.1	برنامه کاربردی باید تمام بررسی‌های توکن نشسته را با استفاده از یک سرویس backend مطمئن انجام دهد.	1
7.2.2	برنامه کاربردی که از هر یک از توکن‌های Self-Contained یا reference که به صورت پویا برای مدیریت نشسته تولید شده‌اند استفاده کند اما از آن‌ها استفاده نکند.	1
7.2.3	در صورتی که برای نمایش نشسته کاربر از توکن‌های reference استفاده می‌شود، این توکن‌ها منحصر به فرد بوده و با استفاده از یک تولیدکننده عدد شبه تصادفی امن رمزنگاری (CSPRNG) که دارای انتروپی حداقل 128 بیتی است تولید شوند.	1
7.2.4	برنامه کاربردی در هنگام احراز هویت کاربری که توکن نشسته جدیدی تولید نماید که شامل احراز هویت مجدد و ابطال توکن نشسته کنونی است.	1

Timeout Session V7.3

مکانیزم‌های انقضای نشسته به منظور به حداقل رساندن فرصت برای hijacking session و سایر روش‌های سواستفاده از نشسته‌ها بکار می‌روند.

#	Description	Level
7.3.1	بر اساس تحلیل ریسک و تصمیمات امنیتی مستند شده، پس از گذشت مهلت زمانی عدم فعالیت، کاربر مجبور به احراز هویت مجدد گردد.	2

#	Description	Level
7.3.2	بر اساس تحلیل ریسک و تصمیمات امنیتی مستند شده، پس از گذشت بیشینه زمان مطلق طول عمر، کاربر مجبور به احراز هویت مجدد گردد.	2

Termination Session V7.4

ابطال نشست ممکن است توسط خود برنامه کاربردی و یا در صورتی که ممکنی‌زم SSO برای مدیریت نشست مورد استفاده قرار گرفته است، توسط SSO به جای برنامه صورت پذیرد. ممکن است لازم باشد هنگام بررسی الزامات این بخش، تصمیم گرفته شود که آیا ارائه‌دهنده SSO در محدوده این الزامات قرارگیری‌خواهد، زیرا برخی از این SSO ممکن است توسط ارائه‌دهنده کنترل شوند.

ابطال نشست باید منجر به الزام کاربر به احراز هویت مجدد و در سراسر برنامه، ورودی‌کپارچه (Federated) Login و سایر بخش‌ها اثر بخش باشد.

برای ممکنی‌زم‌های نشست، stateful ابطال عموم‌شامل نام‌تبررسی نشست در است‌backend در مواردی که توکن‌ها self-contained هستند، معیارهای بیش‌تری به منظور ابطال یا بلاک کردن این توکن‌ها مورد نیاز هستند؛ درغیراین صورت این توکن‌ها تا زمان منقضی شدن بصورت معتبر باقی می‌مانند.

#	Description	Level
7.4.1	در مواقعی که فرآیند خاتمه نشست کاربر فعال می‌شود (مانند خروج کاربر یا انقضای نشست)، برنامه اجازه‌هیچ فعالیتی را به آن نشست نمی‌دهد. برای tokens Reference یا session، stateful این به معنی نام‌تبررسی اطلاعات نشست در برنامه Backend می‌باشد. برنامه‌های که از توکن‌های Self-contained استفاده می‌کنند نیاز به راه‌حلهای نظیر نگه‌داری فهرستی از توکن‌های خاتمه‌یافته، عدم پذیرش توکن‌های که پیش از یک تاریخی و زمان مشخص برای هر کاربر تولد شده‌اند، یا چرخش کلید امضای مخصوص هر کاربر دارند.	1
7.4.2	برنامه کاربردی تمام نشست‌های فعال را در زمانی که حساب کاربری غیرفعال یا حذف می‌شود را ابطال نماید. (مانند کارمندی که شرکت را ترک می‌کند)	1
7.4.3	برنامه کاربردی قابلیت ارائه گزینۀ ابطال تمام نشست‌های فعال دی‌گرا پس از یک تغیری موفقیت‌آمیزی حذف‌ریک از عوامل احراز هویت (نظیر تغیری پس‌ورد از طریق ریست یا بازنشانی و در صورت وجود به روزرسانی تنظیمات MFA) را داشته باشد.	2
7.4.4	تمامی صفحاتی که نی‌ازمند احراز هویت هستند دارای دسترسی‌آسان و مشخص به فرآیند خروج باشند.	2

#	Description	Level
7.4.5	مدیران برنامه کاربردی قادر به ابطال نشست‌های فعال برای یک شخص یا تمامی کاربران باشند.	2

Abuse Session Against Defenses V7.5

این بخش الزاماتی را برای کاهش ریسک‌های ناشی از نشست‌های فعال که می‌توانند از طریق وکتورهای که متکی بر وجود نشست‌های کاربر فعال و قابلیت‌های آنها هستند سرقت و یا سوءاستفاده قرارگیری را ارایه می‌دهد. بطور مثال، با استفاده از اجرای محتوای آلوده مرورگر احراز هویت شده قربانی را مجبور کنی که یک عملی را با استفاده از نشست کاربر قربانی انجام دهد.

توجه داشته باشید که راه‌نمای سطح محور موجود در فصل «احراز هویت» باید هنگام در نظر گرفتن الزامات این بخش مدنظر قرار گیرد.

#	Description	Level
7.5.1	برنامه کاربردی قبل از اعطای مجوز تغیری ویژگی‌های حساس حساب کاربری که بر روی احراز هویت تاثیر دارد نظیر آدرس ایمیل، شماره تلفن، تنظیمات MFA یا اطلاعات دیگری که در بازگردانی حساب کاربری مورد استفاده قرار می‌گیرند، به احراز هویت مجدد کامل نیازی دارد.	2
7.5.2	حساب‌های کاربری قادر به دیدن (پس از احراز هویت مجدد با حداقل یک عامل) و ابطال یک یا همه نشست‌های فعال باشند.	2
7.5.3	برنامه کاربردی قبل از اجرای تراکنش‌های عملی‌های حساس نیازی ندارد احراز هویت مجدد با حداقل یک عامل یا صحت‌سنجی ثانویه باشد.	3

Re-authentication Federated V7.6

این بخش مربوط به کسانی است که کد طرف‌ات‌کا (RP) یا ارایه دهنده هویت (IdP) را می‌نویسند. این الزامات از 800-63C SP NIST برای هویت‌اشتراکی و ادعاه‌گرفته شده است.

#	Description	Level
7.6.1	طول عمر و ابطال نشست بین RP و idp طبق سند- که نیازی ندارد احراز هویت مجدد در زمان مورد نیازی از ماندن وقتی که بی‌شینه زمان بین رخداد احراز هویت idp رسیده باشد- انجام شود.	2

#	Description	Level
7.6.2	ساخت نشسته‌ها نیازی به رضایت کاربر یا یک اقدام صریح دارد که بدین صورت از ایجاد نشسته‌های جدید برنامه بدون تعامل کاربر جلوگیری می‌شود.	2

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- Testing Management Session Guide: Testing Security Web OWASP
- Sheet Cheat Management Session OWASP

Authorization V8

هدف

مجوز دسترسی، تضمین می‌کند که دسترسی‌ها فقط در اختیاریار مصرف کنندگان مجاز (کاربران، سرورها و سایر کلاینت‌ها) قرار گیرد. به منظور اعمال اصل حداقل دسترسی (POLP)، برنامه‌های کاربردی مورد تایید باید این الزامات سطح بالا را احصاء نمایند:

- قوانین صدور مجوزها که شامل عوامل دخیل در تصمیم‌گیری و عوامل محیطی است را مستند سازی نمایند.
- دسترسی مصرف کنندگان به منابع باید بر اساس حقوق تعریف شده آنها مجازش‌مده شود.

Documentation Authorization V8.1

مستند سازی جامع مجوزها به منظور اطمینان از اعمال پیوسته، قابلیت ممیزی (Auditable) و همسود بودن تصمیمات امنیتی با سیاست‌های سازمانی ضروری است و همچنین با شفاف سازی و اجرای کردن الزامات امنیتی برای توسعه دهندگان، مدیران سیستم و آزمون‌گرهای ریسک دسترسی‌غیر مجاز را کاهش می‌دهد.

#	Description	Level
8.1.1	مستند اعطای مجوز باید شامل قوانینی به منظور محدودسازی دسترسی در سطح عمل‌کرد و دسترسی مبتنی بر داده بر اساس مجوزهای کاربران و ویژگی‌های منابع باشد.	1

#	Description	Level
8.1.2	مستند اعطای مجوز بای د قوانینی را برای مح دودیتهای دسترسی در سطح Field بر اساس مجوزه ای کاربران و ویژگی های منابع تعریف نماید (هر دو دسترسی خواندن و نوشتن). توجه داشته باشید که این قوانین ممکن است به مقادیری گرو ویژگی های شیء داده ی مربوطه (Object) Data (Relevant) بستگی داشته باشند، مانند حالت (state) یا وضعیت (status).	2
8.1.3	مستند برنامه کاربردی بای د ویژگی های محیطی و زمینه ای (شامل و نه محدود به زمان روز، مکان، آدرس IP یا دست گاه) که در برنامه کاربردی به منظور اتخاذ تصمیمات امنیتی (نظری آن دست از تصمیماتی که مربوط به احراز هویت و اعطای مجوز می باشد) را مشخص کند.	3
8.1.4	مستندات احراز هویت و اعطای مجوز، علاوه بر اعطای مجوز در سطح Field، سطح عمل کرد و داده های خاص، چگونگی استفاده از عوامل محیطی و زمینه ای در اتخاذ تصمیمات را مشخص نماید.	3

Design Authorization General V8.2

پیاده سازی کنترل های اعطای مجوز جزئی در سطح عمل کرد، داده و فیلدها، تضمین می کند که مصرف کنندگان فقط قادر به دسترسی به آن چیزی هستند که صریحاً به آنها دسترسی داده شده است.

#	Description	Level
8.2.1	برنامه کاربردی بای د از محدود بودن دسترسی های در سطح عمل کرد به مشترکینی که دارای دسترسی صریح هستند اطمینان حاصل نماید.	1
8.2.2	برنامه کاربردی بای د از محدود بودن دسترسی های در سطح داده به مشترکینی که دارای دسترسی صریح به اقلام داده ای خاص هستند اطمینان حاصل نماید تا از این طریقی خطر آسیب پذیری IDOR و level object broken authorization را کاهش دهد.	1
8.2.3	برنامه کاربردی دسترسی در سطح Field را با مجوزه ای صریح به Field ها برای مصرف کنندگان جهت کاهش خطر level property object broken authorization (BOPLA) محدود کرده باشد.	2
8.2.4	کنترل های امنیتی تطبیقی که بر اساس ویژگی های محیطی و زمینه ای کاربر (مانند زمان روز، مکان، آدرس IP یا دست گاه) هستند، برای تصمیم گیری های احراز هویت (Authentication) و مجوزدهی (Authorization) طبق مستندات برنامه پیاده سازی شده اند. این کنترل ها بای د هم در زمان تلاش کاربر برای شروع نشست (Session) جدید و هم در طول نشست فعال، اعمال شوند.	3

Authorization Level Operation V8.3

اجرای فوری تغیریات در مجوزه‌ای دسترسی در لایه مناسب از معماری برنامه کاربردی به منظور جلوگیری از اقدامات غیرمجاز بویژه در محیط‌های پویا ضروری است.

#	Description	Level
8.3.1	برنامه کاربردی قوانین مجوزدهی را در لایه سرویس معتبر اعمال نموده و بر آن دسته از کنترل‌های که مصرف کنندگان نامعتبر قادر به دست‌کاری آن‌ها هستند متکی نباشد. (نظریه اواس کریپت سمت کاربر)	1
8.3.2	در مواقعی که امکان اعمال فوری تغیریات وجود ندارد (مثلاً وقتی که سیستم برای تصمیم‌گیری در مجوزدهی از توکن‌های Self-Contained JWT استفاده می‌کند که اطلاعات را داخل خود ذخیره کرده و تا زمان انقضای تغیری نمی‌کنند)، باید کنترل‌های اجرایی وجود داشته باشد. این کنترل‌ها باید بتوانند زمانی که کاربر عملی را انجام می‌دهد که دی‌گر مجاز به آن نیست، هشدار بدهند و تغیری را برگردانند. با این حال، توجه کنید که این راه‌کار جایگزین نمی‌تواند جلوی نشت اطلاعات را بگیرد.	3
8.3.3	دسترسی به یک object بر اساس مجوزه‌ای کاربری شخص درخواست کننده (مثال: مصرف کننده) باشد نه بر اساس مجوزه‌ای واسطه‌ای سرویسی که به نمایندگی از او عمل می‌کند. بطور مثال، در صورتی که یک مصرف کننده یک وب سرویس را با استفاده از یک توکن self-contained برای احراز هویت فراخوانی می‌کند و سپس آن سرویس اطلاعات را از یک سرویس دیگر درخواست می‌کند، سرویس دوم باید همان توکن مصرف کننده را استفاده کند، نه یک توکن سرویس-به-سرویس (Machine-to-Machine) (token) که متعلق به سرویس اول است، تا تصمیمات مجوزدهی دقیقاً بر اساس مجوزه‌ای کاربر گرفته شوند.	3

Considerations Authorization Other V8.4

ملاحظات اضافی برای مجوزدهی، بویژه برای درگاه‌های مدیریتی و محیط‌های چند کاربره (multi-tenants) وجود دارد که برای جلوگیری از دسترسی غیرمجاز مفید هستند.

#	Description	Level
8.4.1	برنامه‌های کاربردی چند کاربره (multi-tenants) از کنترل‌های cross-tenant به منظور تضمین این امر که عملیات‌های مصرف کنندگان هرگز کاربرانی (tenants) را که مجوز تعامل با آن‌ها را ندارند را تحت تأثیر قرار نمی‌دهد.	2

#	Description	Level
8.4.2	دست‌رسی به درگاه‌های پی‌کربندی شامل چندین لایه امنیتی از قبیل تأیید مداوم هویت مصرف‌کننده، ارزیابی وضعیت امنیتی دستگاه و تحلیل ریسک زمینه‌ای باشد تا از این طریق اطمینان حاصل شود که موقعیت شبکه‌ای یا نقاط پایانی معتبر تنه‌عوامل برای مجوزدهی نیستند اگرچه که این عوامل احتیاطی دسترسی غیرمجاز را کاهش می‌دهند.	3

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- Authorization Guide: Testing Security Web OWASP •
- Sheet Cheat Authorization OWASP •

Tokens Self-contained V9

هدف

مفهوم توکن self-contained در RFC 6749 OAuth2 اصلی مربوط به سال 2012 ذکر شده است. این مفهوم به توکنی اشاره دارد که شامل داده یا Claim است که یک سرویس گیرنده برای تصمیم‌گیری-های امنیتی به آن‌ها متکی خواهد بود. از این رو باید آن‌ها را از توکن‌های ساده‌ای که فقط حاوی یک شناسه هستند که سرویس گیرنده از آن‌ها برای جستجوی داده بصورت محلی استفاده می-کند متمایز دانست. رایج‌ترین مثال‌ها از توکن‌های self-contained، Tokens Web JSON و SAML assertion هستند.

کاربرد توکن‌های self-contained محتای خارج از حوزه OAuth و OIDC پس‌افراگیری شده است. در همین زمان امنیتی این مکانیزم بر قابلیت اعتبارسنجی یکپارچه‌گی توکن و تضمین این مساله که این توکن برای یک زمینه خاص معتبر است استوار است. موانع زیادی (چالش‌ها) در این فرآیند وجود دارد و این بخش جزئیات ویژه‌ای از این مکانیزم‌ها را ارائه می‌دهد که برنامه‌ها باید به منظور جلوگیری از این چالش‌ها، ارائه دهند.

integrity and source Token V9.1

این بخش شامل الزاماتی است که تضمین می‌کند که توکن توسط بخش مطمئن (مورد اعتبار) تولید شده است و مورد دست‌کاری قرار نگرفته است.

#	Description	Level
9.1.1	قبل از پذیرش محتوای توکن‌ها، آن‌ها را با استفاده از امضای دیجیتالی MAC به منظور حفاظت در مقابله دستکاری‌ها مورد اعتبارسنجی قرار داده شود.	1
9.1.2	فقط الگوریتم‌های موجود در لیست مجاز قادر به استفاده در ساخت و صحت‌سنجی توکن‌های self-contained در یک زمینه مشخص باشند. این لیست مجاز باید در حالت ایده آل شامل هم الگوریتم‌های متقارن و هم نامتقارن باشد و نباید شامل الگوریتم‌های NONE باشد. در صورتی که هر دو الگوریتم‌های متقارن و نامتقارن مورد نیاز باشند کنترله‌ای بی‌شتری باید به منظور جلوگیری از سردرگمی کلیدی (key Confusion) بکار گرفته شود.	1
9.1.3	به منظور جلوگیری از مشخص نمودن منابع غیر مطمئن و کلیدهای متوسط مهاجمین، موارد کلیدی که در اعتبارسنجی توکن‌های self-contained مورد استفاده قرار می‌گیرند، از منابع از پیش پی‌کرده شده مطمئنی که برای صادرکننده توکن است تعریف می‌شود. برای JWTs و سایر ساختارهای JWS، هدرهای نظیر 'x5u'، 'jku' و 'jwk' باید بایک لیست مجاز از منابع مطمئن مورد اعتبارسنجی قرار گیرند.	1

content Token V9.2

قبل از اتخاذ تصمیمات امنیتی براساس محتوای توکن، self-contained ضروری است ابتدا بررسی شود که آیا توکن در دوره معتبر خود و برای هدفی که توسط سرویس‌گیرنده درجهت کاربرد آن در نظر گرفته و ارائه شده است. این موضوع به منظور جلوگیری از استفاده متقابل ناامن بین سرویس‌های مختلف با انواع توکن‌های مختلف از صادرکننده مشابه است. الزامات خاص برای OAuth و OIDC در این بخش مورد پوشش قرار گرفته شده‌اند.

#	Description	Level
9.2.1	اگری که بازه زمانی معتبر در داده‌های توکن وجود داشته باشد، توکن و محتوای آن تنها در صورتی پذیرفته می‌شوند که زمان بررسی در این بازه زمانی معتبر باشد. به عنوان مثال، برای JWT، ادعاهای 'exp' و 'nbf' باید تایید شوند.	1
9.2.2	سرویس‌دهی که یک توکن دریافت می‌کند قبل از پذیرش محتوای آن، از لحاظ درست‌ی نوع بررسی نموده و برای هدف مشخص شده در نظر می‌گیرد. بطور مثال، فقط توکن‌های access می‌توانند به منظور تصمیمات مجوزی و فقط توکن‌های ID می‌توانند به منظور اثبات هویت کاربر مورد پذیرش قرار گیرند.	2

#	Description	Level
9.2.3	یک سرویس فقط توکن‌های را بپذیرد که به منظور استفاده توسط آن سرویس هستند (audience). برای JWT ها، این امر می‌تواند از طریق اعتبارسنجی ادعای 'aud' با یک لیست مجاز تعریف شده در سرویس صورت پذیرد.	2
9.2.4	اگر صادرکننده توکن از کلید خصوصی مشابهی برای های audience متفاوت استفاده می‌کند، توکن‌های تولید شده دارای محدودیت مخاطبی باشند که بطور منحصر به فردی مخاطبین مورد نظر را مشخص می‌کند. این امر از استفاده مجدد توکن توسط مخاطب ناخواسته جلوگیری می‌کند. در صورتی که شناسه مخاطب به صورت پویا مشخص شده باشد، صادرکننده توکن باید این مخاطبین را به منظور اطمینان از عدم جعل هویت مخاطب مورد اعتبارسنجی قرار دهد.	2

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

• OWASP JSON Web Token Cheat Sheet

OIDC and OAuth V10

هدف

OAuth2 (که در این فصل با عنوان OAuth از آن یاد می‌شود) یک چارچوب استاندارد در صنعت برای واگذاری مجوز دسترسی (delegated authorization) است. برای مثال، با استفاده از OAuth یک برنامه کاربری (Client) می‌تواند به نمایندگی از کاربر، به API (منابع سرور) دسترسی پیدا کند، مشروط بر این‌که کاربر، دسترسی مورد نیاز را به آن برنامه اعطا کرده باشد.

به تنهایی، OAuth برای احراز هویت کاربر طراحی نشده است. چارچوب OpenID Connect (OIDC) با افزودن یک لایه هویت کاربر بر روی OAuth، آن را گسترش می‌دهد. امکاناتی که OIDC مانند اطلاعات استاندارد هویت کاربر، ورودی یکپارچه (Single Sign-On - SSO) و مدیریت نشست (Session Management) را پشتیبانی می‌کند. از آنجا که OIDC یک افزونه OAuth محسوب می‌شود، الزامات مربوط به OAuth در این فصل، شامل OIDC نیز می‌گردد.

نقش‌های زیر در OAuth تعریف شده‌اند:

- مشتری (OAuth Client) برنامه‌ای است که تلاش می‌کند به منابع سرور دسترسی پیدا کند (برای مثال، با فراخوانی یک API به کمک توکن دسترسی صادر شده) مشتری OAuth اغلب یک برنامه سمت سرور (Server-side Application) است.

- یک Client Confidential نوعی OAuth Client است که توانایی حفظ محرمانگی اعتبارنامه‌های (Credentials) را دارد که برای احراز هویت خود در برابر سرور مجوز (Authorization Server) استفاده می‌کند.

- ی یک Client Public توانایی حفظ محرمانگی اعتبارنامه‌ها (Credentials) برای احراز هویت در برابر Server Authorization را ندارد. بنابراین، به جای احراز هویت خود (برای مثال با استفاده از پارامترهای client_id و client_secret صرفاً خودش را شناسایی می‌کند) (با استفاده از پارامتر client_id)

- Server Resource OAuth (RS) سروری است که از طریق API منابع را در اختیاریار Clients OAuth قرار می‌دهد.

- Server Authorization OAuth (AS) ی یک برنامه‌ی سرور است که به Clients OAuth توکن‌های دسترسی (Access Tokens) صادر می‌کند. این توکن‌ها به Clients OAuth امکان می‌دهند تا به منابع موجود در RS دسترسی پیدا کنند؛ یا به نمایندگان از کاربر نهایی (End-User) یا به نمایندگان از خود Client. OAuth معمولاً AS ی یک برنامه‌ی مجزا است، اما در صورت لزوم می‌تواند در ی یک RS مناسب نیز ادغام شود.

- Owner Resource (RO) همان کاربر نهایی (End-User) است که به Clients OAuth اجازه می‌دهد تا به نمایندگان از او، دسترسی محدودی به منابع موجود روی Server Resource داشته باشند. Resource Owner با تعامل با Authorization Server، رضایت خود را برای این واگذاری مجوز دسترسی (Del-egated Authorization) اعلام می‌کند.

نقش‌های زیر در OIDC تعریف شده‌اند:

- Party Relying (RP) برنامه‌ی کاربری (Client Application) است که از طریق Provider OpenID درخواست احراز هویت کاربر نهایی (End-User) را می‌دهد. این نقش در واقع همان نقش ی یک Client OAuth را بر عهده دارد.

- Provider OpenID (OP) ی یک OAuth AS است که توانایی احراز هویت کاربر نهایی (End-User) را دارد و اطلاعات هویتی (OIDC Claims) را در اختیاریار RP قرار می‌دهد. OP می‌تواند همان Provider Identity (IdP) باشد، اما در سناریوهای فدراسیونی (Federated OP Scenarios)، Provider Identity (جایی که کاربر نهایی احراز هویت می‌شود) ممکن است دو برنامه‌ی سرور جداگانه باشند.

OAuth و OIDC در ابتدا برای برنامه‌های سوم شخص (Third-Party Applications) طراحی شده بودند. امروزه، این پروتکل‌ها به‌طور گسترده در برنامه‌های اول شخص (First-Party Applications) نیز مورد استفاده قرار می‌گیرند. با این حال، هنگام استفاده در سناریوهای اول شخص مانند احراز هویت (Authentication) و مدیریت نشست (Session Management) این پروتکل‌ها پیچیدگی‌های را به همراه می‌آورند که ممکن است چالش‌های امنیتی جدیدی ایجاد کنند.

OAuth و OIDC را می‌توان در انواع مختلف برنامه‌ها به کار برد، اما تمرکز در ASVS و همچنین الزامات مطرح شده در این فصل، بر روی برنامه‌های وب (Web Applications) و رابط‌های برنامه‌نویسی کاربردی (APIs) است.

از آنجا که OAuth و OIDC را می‌توان به عنوان لایه‌ای منطقه‌ی بر بستر فناوری‌های وب در نظر گرفت، الزامات عمومی مطرح شده در فصل‌های دی‌گرم‌واره در اینجا نیز کاربرد دارند. بنابراین، این فصل را نمی‌توان به‌صورت جدا از بقیه‌ی محت‌وا در نظر گرفت.

این فصل به به‌ترین شیوه‌های جاری (Current Best Practices) برای OAuth2 و OIDC می‌پردازد که با مشخصات منتشر شده در نشانی‌های <https://openid.net/developers/specs/> و <https://oauth.net/2/> هم‌سو هستند. حتی اگرچه RFCها به بلوغ رسیدگی می‌شوند، اما به‌طور مداوم به‌روزرسانی می‌شوند.

بنابراین، هنگام به کارگیری الزامات این فصل، همسویی با آخرین نسخه‌ها بسیاری را همراهی دارد. برای جزئیات بیشتر به بخش مراجع (References) مراجعه کنید.

با توجه به پیچیدگی این حوزه، برای داشتن یک راه کار امن مبتنی بر OAuth یا OIDC بسیاری از ضروری است که از Authorization های Server شناخته شده و استاندارد در صنعت استفاده شود و پی کربندی‌های امنیتی توصیف شده به کار گرفته شوند.

اصطلاحات به کاررفته در این فصل با OAuth RFC و مشخصات OIDC همسو هستند. با این حال، توجه داشته باشید که اصطلاحات OIDC تنها در مواردی استفاده می‌شوند که مربوط به الزامات خاص OIDC باشند؛ درغیراین صورت، از اصطلاحات OAuth استفاده خواهد شد. در زمینه‌ی OAuth و OIDC اصطلاح توکن (Token) در این فصل اشاره دارد به:

- Access Tokens، که فقط بایده توسط RS مصرف شوند و می‌توانند ی‌ای Tokens Reference باشند که با استفاده از Introspection اعتهبارسنجی می‌شوند، یا Tokens Self-contained باشند که با استفاده از کلید رمزنگاری (Material Key) اعتهبارسنجی می‌شوند.
- Refresh Tokens، که فقط بایده توسط Authorization ای Server مصرف شوند که توکن را صادر کرده است.
- ID Tokens، که فقط بایده توسط Client ای مصرف شوند که جریانه از احراز مجوز (Authorization Flow) را آغاز کرده است.

سطوح ریسک برای برخی از الزامات این فصل به این بستگی دارد که آیا Client ی Confidential است یا به عنوان ی Client Public در نظر گرفته می‌شود. از آنجا که استفاده از احراز هویت قوی Client بسیاری از برده‌های حمله را کاهش می‌دهد، برخی از الزامات مهم کن است در هنگام استفاده از ی Client Confidential برای برنامه‌های L1 آسان‌تر در نظر گرفته شوند.

Security OIDC and OAuth Generic V10.1

این بخش الزامات معماری عمومی را پوشش می‌دهد که برای همه‌ی برنامه‌های که از OAuth یا OIDC استفاده می‌کنند، اعمال می‌شوند.

#	Description	Level
10.1.1	توکن‌ها بایده فقط به مؤلفه‌های ارسال شوند که به‌طور دقیقی به آنها نی‌از دارند. برای مثال، هنگام استفاده از الگوی Backend-for-Frontend برای برنامه‌های JavaScript مبتنی بر مرورگر، Access Tokens و Refresh Tokens بایده فقط برای Backend قابل دسترس باشند.	2

#	Description	Level
10.1.2	Client باید فقط مقادیری را از Server Authorization بپذیرد (مانند Token ID یا Code Authorization (که این مقادیر حاصل از یک Flow Authorization باشند که توسط همان نشست (Session) و تراکنش Agent User آغاز شده است. این موضوع مستلزم آن است که های Secret تولیدی شده توسط کلاینت مانند code_verifier، در PKCE یا مقادیر state و nonce در OIDC قابل حدس زدن نباشند، مخصوصاً همان تراکنش باشند، و به صورت امن هم به کلاینت و هم به نشست (Session) مرورگری که تراکنش در آن شروع شده، متصل شده باشند.	2

Client OAuth V10.2

این الزامات مسئولیت‌های را که بر عهده برنامه‌های کلاینت OAuth است مشخص می‌کنند. کلاینت می‌تواند مثلاً یک بک‌اند وب‌سرور باشد (که اغلب به عنوان Frontend For Backend یا BFF عمل می‌کند) یا یک سرویس بک‌اند یکپارچه‌ساز، یا یک برنامه فرانت‌اند تک صفحه‌ای (SPA) یا همان برنامه مبتنی بر مرورگر باشد.

به طور کلی، Clients Backend به عنوان Confidential Clients و Clients Frontend و Clients Public به عنوان Public Clients در نظر گرفته می‌شوند. با این حال، برنامه‌های Native که روی دستگاه کاربر نه‌ای اجرا می‌شوند می‌توانند در صورتی که از OAuth Dynamic Client Registration استفاده کنند، به عنوان Confidential در نظر گرفته شوند.

#	Description	Level
10.2.1	در Code، Flow، Client OAuth باید در برابر حملات Browser-based Forgery Request که معمولاً با عنوان Cross-Site Forgery Request (CSRF) شناخته می‌شوند و درخواست‌های توکن را ایجاد می‌کنند، محافظت داشته باشد. این محافظت باید با استفاده از قابلیت PKCE یا با بررسی پارامتر state که در درخواست احراز مجوز ارسال شده است.	2
10.2.2	اگر Client OAuth بتواند با بیش از یک Server Authorization تعامل داشته باشد، باید دارای سازوکاری برای دفاع در برابر Attacks Mix-up باشد. برای مثال، می‌تواند الزام کند که Server Authorization مقدر پارامتر 'iss' را برگرداند و این مقدار هم در Response Authorization و هم در Response Token اعتبارسنجی شود.	2
10.2.3	Client OAuth باید فقط Scopes (یا سایر پارامترهای مجوزدهی) مورد نیاز را در درخواست به Server Authorization درخواست کند.	3

Server Resource OAuth V10.3

در زمی نهی ASVS و این فصل، Server Resource یی که API است. برای فراهم کردن دسترسی ایمن، Resource Server باید:

- Token Access را مطابق با فرمت توکن و مشخصات پروتکل مربوطه اعانتبارسنجی کند، برای مثال با استفاده از JWT-validation یا OAuth Token Introspection.
- اگر Token Access معتبر باشد، تصمیمات مجوزدهی (Authorization) (Decisions) را براساس اطلاعات موجود در توکن و دسترسی‌های اعطا شده اعمال نماید. برای مثال، Server Resource باید بررسی کند که آیا Client (که به نمایندگی از RO عمل می‌کند) اجازه دسترسی به Resource درخواسی را دارد یا خیر.

بنابراین، الزامات ذکر شده در اینجا مختص OAuth یا OIDC هستند و باید پس از اعانتبارسنجی توکن و پیش از اعمال تصمیمات مجوزدهی براساس اطلاعات موجود در توکن انجام شوند.

#	Description	Level
10.3.1	Server Resource باید فقط Access های Token را بپذیرد که برای استفاده با همان سرویس در نظر گرفته شده‌اند (Audience). Audience در یی که Token Access ساختاریافته (مانند مقدار aud در JWT) درج شده باشد، یا از طریقی Token Introspection Endpoint بررسی شود.	2
10.3.2	Server Resource باید تصمیمات مجوزدهی (Authorization) (Decisions) را براساس Claims موجود در Token Access که واگذاری مجوز (Delegated Authorization) را تعریف می‌کنند، اعمال کند. اگرهای Claims مانند scope و sub و authorization_details موجود باشند، باید جزو فرایند تصمیم‌گیری قرار گیرند.	2
10.3.3	اگر برای گرفتن یی که تصمیم‌کنترل دسترسی، لازم است یی که کاربر یی که تا از داخل Token Access (چه JWT و چه پاسخ Introspection Token) شناسایی شود، سرور منبع (Resource) (Server) کاربر را از روی های Claim شناسایی کند که قابل اختصاص دادن به کاربران دیگرنیستند. معمولاً ایی‌نی‌عنی است‌فاده از ترکیب دو Claim یی‌عنی 'iss' (صادرکننده توکن) و 'sub' (شناسه یی‌کتای کاربر)	2
10.3.4	اگر Server Resource نی‌ازمند قدرت، روش یا تازگی خاص احراز هویت باشد، بررسی کند که Token Access ارائه شده این محدودیت‌ها را برآورده می‌کند. برای مثال، در صورت وجود، از های Claims OIDC مانند amr و auth_time به ترتیب استفاده شود.	2
10.3.5	تأیید کنی‌د که Server Resource از استفاده از Access های Token سرقت شده یی Replay توکن‌ها (توسط افراد غیرمجاز) جلوگیری کند، با الزام استفاده از Access Sender-Constrained Tokens، TLS Mutual Authentication یا OAuth2 (DPoP). Possession of Proof of Demonstration	3

Server Authorization OAuth V10.4

این الزامات، مسئولیت‌های مربوط به Authorization OAuth، Servers شامل Providers OpenID را تشریح می‌کنند.

برای Client، Authentication، روش 'self_signed_tls_client_auth' مجاز است، همراه با پیش‌نیازهای که در بخش 2.2 از RFC 8705 مشخص شده‌اند.

#	Description	Level
10.4.1	سرور احراز هویت باید آدرس‌های بازگشتی (Redirect URI) را بر اساس فهرست مجاز (Allowlist) از URI‌های پیش‌ثبت‌شده و مخصوص هر کلاینت، با استفاده از مقایسه دقیق رشته‌های String (Exact Comparison) اعتراف‌سنجی می‌کند.	1
10.4.2	اگر Server Authorization کد احراز مجوز (Authorization Code) را در Authorization Response برمی‌گرداند، این کد فقط یکتا برای درخواست توکن باید قابل استفاده باشد. برای دومین درخواست معتبر با یک Authorization Code که قبلاً برای صدور Token Access استفاده شده است، Authorization Server باید درخواست توکن را رد کند و هر توکن صادرشده‌ی مرتبط با آن Code Authorization را باطل نماید.	1
10.4.3	تأیید کنی‌د که Code Authorization دارای مدت زمان معتبر بودن محدود باشد. حداکثر طول عمر می‌تواند برای برنامه‌های کاربردی در سطوح L1 و L2 تا ۱۰ دقیقه و برای برنامه‌های کاربردی در سطح L3 تا ۱ دقیقه باشد.	1
10.4.4	برای یک Client مشخص، Server Authorization فقط استفاده از های Grant را مجاز بداند که آن Client به استفاده از آنها نیازی دارد. توجه داشته باشید که های Grant (Implicit 'token' Grant) و (Resource 'password' Credentials Password Owner Flow) دی‌گرن باید استفاده شوند.	1
10.4.5	Server Authorization باید حملات Replay Token Refresh را برای Client Public کاهش دهد، ترجیحاً با استفاده از Refresh Sender-Constrained Tokens، Refresh Tokens Access Certificate-Bound (DPoP) یا Refresh Tokens Mutual TLS (mTLS). برای برنامه‌های L1 و L2 می‌توان از Refresh Token استفاده کرد. اگر Refresh Token Rotation استفاده شود، Server Authorization باید پس از استفاده، Token Refresh را باطل کند و اگر Refresh Token که قبلاً استفاده و باطل شده ارائه شود، تمام Refresh های Token مربوط به آن مجوز را ابطال نماید.	1

#	Description	Level
10.4.6	اگر از Grant Code استفاده شود، Server Authorization باید حمله‌ها را با الزام به استفاده از Code for Key Proof Exchange (PKCE) کاهش دهد. برای Authorization Request، Server Authorization باید یک مقدار معتبر برای 'code_challenge' الزام کند و نباید مقدار 'code_challenge_method' از نوع 'plain' را بپذیرد. برای Request، Token باید اعتبارسنجی پارامتر 'code_verifier' الزامی باشد.	2
10.4.7	اگر Server Authorization از Registration Client Dynamic Unauthenticated پشتیبانی می‌کند، ریسک مربوط به Client های Application مخرب را باید کاهش دهد. این کار باید شامل اعتبارسنجی Metadata Client (مانند هر Registered، URI، اطمی‌نان از رضایت کاربر، و هشدار به کاربر پیش از پردازش یک Request Authorization با یک Application Client غیرقابل اعتماد باشد.	2
10.4.8	Token Refresh باید دارای یک انقضای مطلق (Absolute Expiration) باشد، حتی اگر از Token Refresh Sliding Expiration استفاده شود.	2
10.4.9	Token Refresh و Token Access Reference باید توسط کاربر مجاز از طریق Server Authorization Interface User قابل ابطال باشد، تا ریسک های مخرب یا توکن‌های سرقت شده کاهش یابد.	2
10.4.10	Client Confidential باید برای درخواست‌های Backchannel بین Client و Server، Authorization، Token، Requests، Authorization Pushed Requests (PAR) و Requests، Revocation Token احراز هویت شود.	2
10.4.11	پی‌کر بندی Server Authorization باید فقط Scopes موردنیاز را به OAuth Client اختصاص دهد.	2
10.4.12	برای یک Client مشخص، Server Authorization فقط مقدار 'response_mode' را که این Client نیاز دارد، مجاز بداند. برای مثال، با اعتبارسنجی این مقدار توسط Server Authorization در برابر مقادیر مورد انتظار با استفاده از (JAR). Request Authorization Pushed (PAR) یا Request Authorization JWT-secured (JAR).	3
10.4.13	'code' Type Grant همواره همراه با (PAR) Requests Authorization Pushed باید استفاده شود.	3
10.4.14	Server Authorization باید فقط توکن‌های دسترسی از نوع وابسته به ارسال کننده (Proof-of-Possession) صادر می‌کند؛ چه به صورت توکن‌های دسترسی مبتنی بر گواهی از طریق TLS دوسویه (mTLS) و چه به صورت توکن‌های دسترسی وابسته به (Demonstration DPoP). of Proof of Possession).	3

#	Description	Level
10.4.15	برای ی ک کلاینت سمت سرور (که روی دست گاه کاربر نه ایی اجرا نمیشود)، سرور احراز هویت اطمینان حاصل می کند که مقدار پارامتر authorization_details از بک اند کلاینت ارسال شده و کاربر در آن دست کاری نکرده است. به عنوان مثال، با الزام به است فاده از درخواست احراز هویت پوشش ده (PAR) یا درخواست احراز هویت ایمن شده با (JAR). JWT	3
10.4.16	کلاینت بای د از نوع Client Confidential باشد و سرور احراز هویت است فاده از روش های قدرتمند احراز هویت کلاینت - مبتنی بر رمزنگاری کلیدی عمومی و مقاوم در برابر حملات attacks replay - را الزامی کند؛ مانند TLS دوسویه (tls_client_auth ، self_signed_tls_client_auth یا JWT مبتنی بر کلیدی خصوصی (private_key_jwt).	3

Client OIDC V10.5

از آنجا که Party Relying در OIDC نقش ی ک کلاینت OAuth را هم ایفا می کند، الزامات بخش OAuth «Client نیز در اینجا قابل اعمال هستند.

همچنین توجه داشته باشید که بخش Authentication «Identity an with Provider» در فصل «Authenticat- tion» نیز شامل الزامات عمومی مرتبط و مهمی است.

#	Description	Level
10.5.1	Client به عنوان Relying بای د Party در برابر حملات بازپخش (Replay) توکن ID محافظت شده است. برای مثال، با بررسی اینکه مقدار claim مربوط به nonce در Token ID دقیقاً با مقدار nonce ای که در درخواست احراز هویت به Provider OpenID ارسال شده، ی کسان باشد (که در OAuth 2 به آن authorization request ارسال ی به server authorization گفته می شود).	2
10.5.2	Client بای د کاربر را به صورت ی کت از Claims Token ID شناسایی کند، معمولاً از 'sub' Claim، که قابل انتساب مجدد به کاربران دی گرنیست (در محدوده ی همان Identity Provider).	2
10.5.3	Client بای د تلاش های ی ک Server Authorization مخرب برای جا زدن خود به جای ی ک Server Authorization دی گراز طریق متادیتای Server Authorization را رد می کند. کلاینت بای د متادیتای Server Authorization را رد کند اگر مقدار URL issuer موجود در متادیتا دقیقاً با URL issuer از پیش پی کربندی شده و مورد انتظار کلاینت ی کسان نباشد.	2
10.5.4	Client بای د اعتبارسنجی کند Token ID برای است فاده توسط همان Client در نظر گرفته شده است، (Audience) با بررسی اینکه 'aud' Claim در توکن برابر با مقدار 'client_id' مربوط به Client باشد.	2

#	Description	Level
10.5.5	هنگام استفاده از OpenID Connect، Back-Channel Logout، Party Relying Logout از Service of Denial از طریق Logout Forced و Confusion Cross-JWT در جریان Logout باید چالوگری کند. Client باید بررسی کند که Token Logout به درستی با مقدار typ آن برابر با logout+jwt باشد، شامل Claim 'event' با نام عرض و صحیح باشد و شامل Claim 'nonce' نباشد. همچنین توضیح می‌شود Expiration کوتاه باشد (مثلاً ۲ دقیقه).	2

Provider OpenID V10.6

از آنجا که OpenID Provider نقش Server Authorization در OAuth را نیز ایفا می‌کنند، الزامات بخش Authorization «OAuth» در این جا نیز قابل اعمال هستند. توجه شود در صورتی که از Flow Token ID و نه Flow Code استفاده شود، Access Token صادر نمی‌شود و در نتیجه بسوی از الزامات مربوط به OAuth Server Authorization قابل اعمال نخواهند بود.

#	Description	Level
10.6.1	Provider OpenID باید فقط مقادیر 'code'، 'ciba'، 'id_token' یا 'id_token' code را برای Mode Response مجاز بداند. توجه داشته باشید که 'code' نسبت به 'id_token' (که Flow Hybrid OIDC است) ترجیح داده می‌شود و 'token' (هر Implicit Flow) نباید استفاده شود.	2
10.6.2	Provider OpenID از Service of Denial از طریق Logout Forced چالوگری کند. این کار با دریافت تأیید صریح از کاربر نه‌ای یا در صورت وجود، اعتراف سنجی پارامترها در Request Logout (که توسط Party Relying آغاز شده است)، مانند 'id_token_hint' انجام می‌شود.	2

Management Consent V10.7

این الزامات، فرآیند تأیید رضایت (Consent) کاربر توسط Server Authorization را پوشش می‌دهند. در صورت نبود تأیید صریح رضایت کاربر، یک مهاجم می‌تواند از طریق جعل (Spoofing) یا مهندسی اجتماعی (Social Engineering) به جای کاربر مجوزها (Permissions) را به دست آورد.

#	Description	Level
10.7.1	Server Authorization بای د ت ض م ی ن ک ن د ک ا ر ب ر ب ر ا ی ه ر د ر خ و ا س ت م چ و ز (Authorization Request) رضایت خود را اعلام کرده است. در صورتی که هویت کلاینت قابل اطمینان نباشد، Server Authorization بای د در همه موارد کاربرد را به صورت صریح برای اعلام رضایت (Consent) راهنمایی و از او تأیید بگیری.	2
10.7.2	زمانی که Server Authorization از کاربرد درخواست رضایت (Consent) می کند بای د اطلاعات ک افی و ش ف ا ف ی د ر ب ا ر ه آ ن چ ه ک ا ر ب ر د ر ح ا ل ت ا ی د آ ن ا س ت ا ر ا ی ه د ه د. در صورت لزوم، این اطلاعات بای د شامل این موارد باشد: ماهیت مجوزهای درخواستی (معمولاً بر اساس server resource scope و جزئیات مجوز در Rich Requests Authorization - RAR)، هویت برنامه یا اپلیکیشن دریافت کننده مجوز و مدت اعتبار این مجوزها.	2
10.7.3	کاربر بای د بتواند رضایتهای (Consents) را که از طریق Authorization Server صادر کرده است، مشاهده، ویرایش و لغو (Revoke) کند.	2

References

برای اطلاعات بیشتر درباره OAuth، لطفاً مشاهده کنید:

• oauth.net

• OWASP OAuth 2.0 Cheat Sheet

برای الزامات مرتبط با OAuth در ASVS، از های RFC منتشر شده و در وضعیت پیش نویس زیر استفاده می شود:

- Framework Authorization 2.0 OAuth The RFC6749
- Usage Token Bearer Framework: Authorization 2.0 OAuth The RFC6750
- Considerations Security and Model Threat 2.0 OAuth RFC6819
- Clients Public OAuth by Exchange Code for Key Proof RFC7636
- Protocol Registration Client Dynamic 2.0 OAuth RFC7591
- Grant Authorization Device 2.0 OAuth RFC8628
- 2.0 OAuth for Indicators Resource RFC8707
- Tokens Access 2.0 OAuth for Profile (JWT) Token Web JSON RFC9068
- Requests Authorization Pushed 2.0 OAuth RFC9126
- Identification Issuer Server Authorization 2.0 OAuth RFC9207
- Requests Authorization Rich 2.0 OAuth RFC9396
- (DPoP) Possession of Proof Demonstrating 2.0 OAuth RFC9449
- Security 2.0 OAuth for Practice Current Best RFC9700
- Applications Browser-Based for 2.0 OAuth draft
- Framework Authorization 2.1 OAuth The draft

برای اطلاعات بیشتر در مورد Connect، OpenID لطفاً به بخش زیر مراجعه کنید:

- 1.0 Core Connect OpenID
- Profile Security 2.0 FAPI

Cryptography V11

هدف

هدف این فصل تعریف به‌ترین رویه‌ها برای استفاده عمومی از رمزنگاری است. همچنین به دنبال آن است که درک بنیادی از اصول رمزنگاری ایجاد کرده و تغیری به سمت رویکردهای مقاوم‌تر و مدرن‌تر را ترغیب کند. این فصل این موارد را تشویق (پیشنهاد) می‌کند:

- پیاده‌سازی سیستم‌های رمزنگاری مقاوم که به‌طور ایمن دچار خطا شوند، با تهدیدات در حال تکامل سازگار شوند و آینده‌نگر باشند.
- به کارگیری سازوکارهای رمزنگاری که هم ایمن باشند و هم با به‌ترین رویه‌های صنعت هم‌ساز باشند.
- حفظ یک سیستم مدیریتی کلیدهای رمزنگاری امن، همراه با کنترل‌های دسترسی مناسب و قابلیت ممیزی.
- ارزیابی منظم چشم‌انداز رمزنگاری به‌منظور شناسایی مخاطرات جدید و تطبیق الگوریتم‌ها بر اساس آن‌ها.
- کشف و مدیریت موارد استفاده‌ی رمزنگاری در طول چرخه‌ی عمر برنامه، به‌منظور اطمینان از ثبات و حفاظت تمام‌دایره‌ای رمزنگاری.

علاوه بر تشریح اصول عمومی و به‌ترین شیوه‌ها، این سند اطلاعات فنی عمیق‌تری درباره الزامات در پیوست C است‌انداده‌ی رمزنگاری ارائه می‌دهد. شامل الگوریتم‌ها و حالت‌هایی که برای اهداف الزامات این فصل «تأیید شده» محسوب می‌شوند.

الزامات استفاده‌کننده از رمزنگاری برای حل مسائل دی‌گر، مانند مدیریتی secrets یا امنیتی ارتباطات، در بخش‌های دی‌گراست‌اند دارد بررسی خواهند شد.

Documentation and Inventory Cryptographic V11.1

برنامه‌ها باید با معماری رمزنگاری قدرت‌مند طراحی شوند تا از دارایی‌های داده مطابق با طبقه‌بندی آن‌ها محافظت شود. رمزگذاری هم‌چیزی که کاری پرهزینه و بی‌هوده است؛ درحالی‌که رمزگذاری نکردن هیچ‌چیزی، از نظر قانونی سهل‌انگاری محسوب می‌شود. بابت تعادلی برقرار شود، معمولاً در مرحله معماری یا طراحی سطح بالا، اس‌پرینتهای طراحی یا آزمای‌ش‌های معماری (architectural spikes). طراحی رمزنگاری به صورت لحظه‌ای یا اضافه کردن آن به سیستم پس از طراحی، به مراتب پرهزینه‌تر و دشوارتر از لحاظ امنیتی خواهد بود نسبت به حالتی که از ابتدا در طراحی لحاظ شده باشد.

مهم است که اطمینان حاصل شود تمام دایره‌ی رمزنگاری به‌طور منظم شناسایی، فهرست‌بندی و ارزیابی شوند. لطفاً برای کسب اطلاعات بیشتر درباره نحوه انجام این کار به پیوست مراجعه کنید.

نیاز به آینده‌نگری در برابر ظهور اجتناب‌ناپذیر رایانش کوانتومی برای مقاوم‌سازی سیستم‌های رمزنگاری نیز حیاتی است. رمزنگاری پساکوانتومی Cryptography (Post-Quantum) یا PQC به الگوریتم‌های رمزنگاری‌ای اشاره دارد که برای مقاوم‌ت در برابر حملات رایانه‌های کوانتومی طراحی شده‌اند؛ رایانه‌هایی که انتظار می‌رود الگوریتم‌های پرکاربردی همچون RSA و رمزنگاری منحنی بیضوی (ECC) را بشکنند.

لطفاً برای دریافت راهنمایی‌های کنونی درباره‌ی ابته‌دایات و استانداردهای تأییدی‌شده در زمینه رمزنگاری پساکوانتومی، (PQC) به ضمیمه مراجعه کنید.

#	Description	Level
11.1.1	اطمینان حاصل کنید که یک سیستم مستند برای مدیریت کلیدهای رمزنگاری و چرخه عمر کلیدهای رمزنگاری وجود دارد که از یک استاندارد مدیریت کلید مانند SP NIST 800-57 پیروی می‌کند. این باید شامل اطمینان از این موضوع باشد که کلیدها بیش از حد به اشتراک گذاشته نشوند (برای مثال، در مورد Secrets مشترک با بیش از دو موجودیت و در مورد کلیدهای خصوصی با بیش از یک موجودیت).	2
11.1.2	فهرست اجزاء رمزنگاری باید تهیه و نگهداری شود و به صورت منظم به روزرسانی گردد و شامل تمام کلیدهای رمزنگاری، الگوریتم‌ها و گواهی‌های دیجیتالی مورد استفاده در برنامه باشد. این فهرست همچنین باید مشخص کند که کلیدها در کدام بخش‌های سیستم مجاز یا غیرمجاز به استفاده هستند و چه نوع داده‌ای می‌توانند یا نمی‌توانند با این کلیدها محافظت شوند.	2
11.1.3	مکانیسم‌های کشف رمزنگاری برای شناسایی تمام موارد استفاده از رمزنگاری در سیستم باید استفاده شود؛ از جمله رمزنگاری (encryption)، هش کردن (hashing) و عملیات امضای دیجیتالی (signing).	3
11.1.4	فهرست اجزاء رمزنگاری باید تهیه شود. این فهرست باید شامل یک برنامه مستند باشد که مسیری مهاجرت به استانداردهای جدید رمزنگاری (مانند رمزنگاری پساکوانتومی) را مشخص می‌کند تا امکان واکنش مناسبت به تهدیدات آینده فراهم شود.	3

Implementation Cryptography Secure V11.2

این بخش الزامات مربوط به انتخاب، پیاده‌سازی و مدیریت مستمر الگوریتم‌های اصلی رمزنگاری برای یک برنامه را تعریف می‌کند. هدف این است که تنها ابته‌دایات رمزنگاری (primitives) قدرت‌مند و مورد پذیرش صنعت، مطابق با استانداردهای فعلی (مانند NIST و ISO/IEC) و به تری‌ن رویه‌ها به کار گرفته شوند. سازمان‌ها باید اطمینان حاصل کنند که هر جزء رمزنگاری بر اساس شواهد داوری شده توسط متخصصان (peer-reviewed) و آزمایش‌های عملی امنیتی انتخاب شده است.

#	Description	Level
11.2.1	برای عملیات رمزنگاری بای از پیاده‌سازی‌های مورد تأیید (شامل کت‌ابخانه‌ها و پیاده‌سازی‌های سخت‌افزاری) استفاده شود.	2
11.2.2	برنامه با انعطاف‌پذیری رمزنگاری (crypto agility) بای طراحی شده باشد، به‌طوری که الگوریتم‌های تولید اعداد تصادفی، رمزنگاری احراز هویت شده، MAC یا هش، طول کلیده‌ها، تعداد دوره‌ها، رمزه‌ها و حالت‌ها بتوانند در هر زمان مجدداً پی‌کربندی، ارتقاء یا تعویض شوند تا در برابر شکست‌های رمزنگاری محافظت شوند. به همین ترتیب، بای امکان جای‌گزینی کلیده‌ها و رمزه‌های عبور و رمزگذاری مجدد داده‌های موجود داشته باشد. این امکان ارتقاء بدون وقفه به رمزنگاری پساکوانتومی (PQC) را فراهم می‌کند، زمانی که پیاده‌سازی‌های با اطمینان بالا از طرح‌های استانداردهای تأیید شده PQC به‌طور گسترده در دسترس قرار گیرند.	2
11.2.3	تمام بدویات رمزنگاری (cryptographic primitives) بر اساس الگوریتم، اندازه کلید و پی‌کربندی، حداقل ۱۲۸ بیت سطح امنیتی را فراهم می‌کنند (2128)، برای مثال، یک کلید ECC با طول ۲۵۶ بیت تقریباً ۱۲۸ بیت امنیتی ارائه می‌دهد، درحالی‌که برای رسیدن به همین سطح امنیتی، الگوریتم RSA به کلید ۳۰۷۲ بیت نیاز دارد.	2
11.2.4	تأیید کنی‌د که تمام عملیات رمزنگاری زمان ثابت (constant-time) باشند و هیچ عملیات «کوتاه‌شده» (short-circuit) در مقایسه‌ها، محاسبات یا بازگشت‌ها وجود نداشته باشد تا از نشت اطلاعات جلوگیری شود. (مهاجم می‌تواند از همین تفاوت‌های زمانی، از طریق Attack، Side-Channel Timing اطلاعات حساس را استخراج کند).	3
11.2.5	تأیید کنی‌د که تمام مازول‌های رمزنگاری به‌صورت امن دچار خطا (fail securely) شوند و خطاها به گونه‌ای مدیریتی شوند که ایجاد آس‌ب‌پذیری‌های مانده حملات Oracle Padding را ممکن نکنند.	3

Algorithms Encryption V11.3

الگوریتم‌های رمزنگاری احراز هویت شده (Authenticated Encryption) مبتنی بر AES و CHACHA20، زیربنای عمل‌کرده‌ای رمزنگاری مدرن را تشکیل می‌دهند.

#	Description	Level
11.3.1	از مدهای بلاک ناامن (مانند ECB) و روش‌های Padding ضعیف (مانند PKCS#1 v1.5) نباید استفاده شود.	1
11.3.2	تنه‌ها رمزه‌ها و مدهای تأیید شده، مانند AES با GCM بای استفاده شوند.	1

#	Description	Level
11.3.3	داده‌های رمزنگاری شده باید با استفاده از ترکیب‌های یک روش رمزنگاری احراز هویت شده تأیید شده یا با ترکیب‌های یک روش رمزنگاری تأیید شده با یک الگوریتم MAC تأیید شده در برابر تغیرات غیرمجاز محافظت شوند.	2
11.3.4	تأیید کنید که، «Vector Initialization Nonce» و سایر اعداد یکبار مصرف برای بیش از یک چگفت کلید رمزنگاری و غیره استفاده نشوند. روش تولید آنها باید متناسب با الگوریتم مورد استفاده باشد.	3
11.3.5	تأیید کنید که هر ترکیبی از یک الگوریتم رمزنگاری و یک الگوریتم MAC در حالت Encrypt-then-MAC عمل کند.	3

Functions Hash-based and Hashing V11.4

توابع هش در انواع گسترده‌ای از پروتکل‌های رمزنگاری استفاده می‌شوند، مانند امضای دیجیتال، HMAC، توابع مشتق‌سازی کلید (KDF)، تولید بیت تصادفی و ذخیره‌سازی پس‌ورد. امنیت سیستم رمزنگاری تنها به اندازه‌ی امنیت توابع هش پایه‌ای است که استفاده می‌شوند. این بخش الزامات استفاده از توابع هش امن در عملیات رمزنگاری را بیان می‌کند.

برای ذخیره‌سازی رمز عبور، علاوه بر پیوست رمزنگاری، OWASP Cheat Sheet نیز زمینه و راهنمای مفیدی ارائه می‌دهد.

#	Description	Level
11.4.1	تأیید کنید که تنها توابع هش تأیید شده برای موارد استفاده عمومی رمزنگاری، از جمله امضای دیجیتال، HMAC KDF و تولید بیت تصادفی استفاده شوند. توابع هش غیرمجاز، مانند MD5، نباید برای هیچ هدف رمزنگاری به کار گرفته شوند.	1
11.4.2	رمز عبورها با استفاده از یک تابع مشتق‌سازی کلید تأیید شده و پرهیز نه محاسباتی (که به آن «تابع هش رمز عبور» نیز گفته می‌شود) ذخیره شوند، و تنظیمات پارامترها بر اساس راهنمای‌های کنونی پی‌کر بندی شده باشد. این تنظیمات باید تعادل بین امنیت و عملکرد برقرار کنند تا حملات Brute-force برای سطح امنیتی مورد نیاز به‌طور کافی دشوار باشد.	2
11.4.3	توابع هش استفاده شده در امضای دیجیتال، باید به‌عنوان بخشی از احراز هویت داده‌ای یکپارچه‌گی داده، در برابر collision مقاوم بوده و طول بیت مناسب داشته باشند. اگر مقاومت در برابر collision لازم باشد، طول خروجی باید حداقل 256 بیت باشد. اگر تنها مقاومت در برابر حملات Second Pre-image لازم باشد، طول خروجی باید حداقل 128 بیت باشد.	2

#	Description	Level
11.4.4	برنامه بایدهنگام استخراج کلیدهای مخفی از پسوردها از توابع مشتق سازی کلید (KDF) تایید شده با پارامترهای Stretching Key استفاده می کند. پارامترهای مورد استفاده بایدهتعدادی بین امنیتی و عملکرد برقرار کنند تا از نفوذ حملات brute-force و به خطر افتادن کلید رمزنگاری تولید شده جلوگیری شود.	2

Values Random V11.5

تولید اعداد شبه تصادفی امن از نظر رمزنگاری (CSPRNG) بسیاری از دشواری است. به طور کلی، منابع خوب اینترنتی در یک سیستم اگر بیش از حد استفاده شوند، سریعیاً تخلیه می شوند، اما منابع با تصادفی بودن کمتر می توانند منجر به ایجاد کلیدها و secrets قابل پیش بینی شوند.

#	Description	Level
11.5.1	اعداد ورشته های تصادفی که قرار است قابل حدس نباشند، بایدهاستفاده از یک تولیدکننده اعداد شبه تصادفی امن رمزنگاری شده (CSPRNG) تولید شوند و حداقل 128 بیت اینترنتی داشته باشند. توجه داشته باشید که UUID این شرط را رعایت نمی کنند.	2
11.5.2	مکانیسم تولید اعداد تصادفی مورد استفاده به گونه ای طراحی شده باشد که حتی تحت بار سنگین نیز به طور امن عمل کند.	3

Cryptography Key Public V11.6

رمزنگاری کلید عمومی (Public Key Cryptography) زمانی استفاده می شود که امکان ایتمایل به اشتراک گذاری کلید مخفی (secret key) بین چندین طرف وجود نداشته باشد.

به عنوان بخشی از این موضوع، نیاز به مکانیسم های تبادل کلید تأیید شده وجود دارد، مانند Diffie-Hellman و Diffie-Hellman Curve Elliptic (ECDH)، تا اطمینان حاصل شود که سیستم رمزنگاری در برابر تهدیدات مدرن امن باقی می ماند. فصل "ارتباط امن" الزامات مربوط به TLS را ارائه می دهد، بنابراین الزامات این بخش برای مواقعی در نظر گرفته شده است که رمزنگاری کلید عمومی در موارد استفاده به غیر از TLS به کار گرفته شود.

#	Description	Level
11.6.1	فقط الگوریتم‌های رمزنگاری و مدهای عملیاتی تأیید شده برای تولید کلید و مقادیر اولیه، و همچنین برای تولید و تأیید امضاهای دیجیتالی بایست استفاده می‌شوند. الگوریتم‌های تولید کلید نباید کلیدهای ناامن تولید کنند که در برابر حملات شناخته شده آسیبی پذیر باشند؛ به عنوان مثال، کلیدهای RSA که در برابر factorization Fermat آسیبی پذیر هستند.	2
11.6.2	برای تبادل کلید از الگوریتم‌های رمزنگاری تأیید شده (مانند Diffie-Hellman) استفاده می‌شود و تمرکز بر این است که مکانیزم‌های تبادل کلید از پارامترهای امن بهره ببرند. این کار از حملات بر فرازی برقراری کلید جلوگیری می‌کند و مانع حملات مرد میانی (MITM) یا شکست‌های رمزنگاری می‌شود.	3

Cryptography Data In-Use V11.7

حفاظت از داده‌ها در حین پردازش اهمیت بالایی دارد. توصیه می‌شود از تکنیک‌های ماندن رمزنگاری کامل حافظه، رمزنگاری داده‌ها در حال انتقال و اطمینان از رمزنگاری سریعی داده‌ها پس از استفاده بهره گرفته شود.

#	Description	Level
11.7.1	رمزنگاری کامل حافظه (Full Memory Encryption) باید فعال باشد تا داده‌های حساس هنگام استفاده محافظت شوند و دسترسی کاربران یا فرآیندهای غیرومجاز به آن‌ها جلوگیری شود.	3
11.7.2	کاهش داده‌ها (data minimization) تضمین می‌کند که حداقل مقدار داده در حین پردازش افشا شود، و اطمینان حاصل شود که داده‌ها بلافاصله پس از استفاده یا در اولین فرصت ممکن رمزنگاری شوند.	3

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- Cryptography Weak for Testing Guide: Testing Security Web OWASP
- Sheet Cheat Storage Cryptographic OWASP
- 140-3 FIPS
- 800-57 SP NIST

Communication Secure V12

هدف

این فصل شامل الزامات مربوط به مکانیسم‌های خاصی است که باید برای محافظت از داده‌ها در حین انتقال برقرار باشند، هم بین کلاینت کاربر نه‌ای و سرور سبک‌اند و هم بین سرورهای داخلی و سبک‌اند.

مفاهیم کلی که در این فصل ترویج می‌شوند عبارتند از:

- اطمینان از اینکه ارتباطات به صورت خارجی رمزگذاری شده‌اند و در ایده‌آل حالت، به صورت داخلی نیز رمزگذاری شوند.
- پی‌کربندی مکانیسم‌های رمزگذاری با استفاده از آخرین دست‌ورال‌عمل‌ها، شامل الگوریتم‌ها و رمزهای ترجیحی.
- استفاده از گواهی‌نامه‌های امضا شده برای اطمینان از اینکه ارتباطات توسط افراد غیرمجاز رهگیری نمی‌شوند.

علاوه بر ارائه اصول کلی و به‌ترین شیوه‌ها، ASVS اطلاعات فنی عمیق‌تری درباره قدرت رمزنگاری نیز در ضمیمه C - است‌انداده‌ای رمزنگاری ارائه می‌دهد.

Guidance Security TLS General V12.1

این بخش راهنمای اولیه در مورد نحوه‌ی ایمن‌سازی ارتباطات TLS را ارائه می‌دهد. ابزاری به‌روز باید برای بازبینی پی‌کربندی TLS به‌صورت مستمر استفاده شوند.

اگرچه استفاده از TLS wildcard ذاتاً ناامن نیست، اما در صورتی که یک گواهی که در تمام می‌محیط‌های test، staging development و production به کار رفته است به خطر بی‌فتد، ممکن است وضعیت امنیتی تمام برنامه‌های استفاده‌کننده از آن گواهی نیز دچار آسیب شود. در صورت امکان، باید از حفاظت مناسب، مدیریتی صحیح و همچنین استفاده از گواهی‌های TLS مجزا در محیط‌های مختلف بهره‌گرفته شود.

#	Description	Level
12.1.1	جدیدترین نسخه‌های پیش‌نه‌ادی پروتکل TLS فعال باشند، مانند TLS 1.2 و TLS 1.3 نسخه‌ی جدیدترین پروتکل TLS باید گزینگی ترجیحی باشند.	1
12.1.2	تنها cipherهای suite پیش‌نه‌ادی فعال باشند و قوی‌ترین cipherها (L3) باید تنها از به‌عنوان گزینگی ترجیحی تنظیم شده باشند. برنامه‌های L3 باید تنها از cipherهای suite پشتیبانی کنند که Secrecy Forward را فراهم می‌کنند.	2
12.1.3	برنامه باید پیش از استفاده از گواهی برای احراز هویت یا مجوزدهی، اعتبار و مورد اعتماد بودن گواهی‌های کلاینت در mTLS را بررسی و تأیید کند.	2

#	Description	Level
12.1.4	لغو و صحت‌چک گواهی‌ها، مانند OCSP، باید فعال و به‌درستی پی‌کربندی شده باشد.	3
12.1.5	افشای متادیتای حواس، مانند Indication Name Server (SNI)، در طول فرآیند TLS Handshake چال‌و‌گری می‌شود.	3

Services Facing External with Communication HTTPS V12.2

تمام ترافیکی که HTTP به سمت سرورهای بیرون‌سازمانی (external-facing) که برنامه ارائه می‌دهد، به صورت رمزنگاری شده و با استفاده از گواهی‌های عمومی مورد اعتماد ارسال می‌شود.

#	Description	Level
12.2.1	برای تمام ارتباطات بین کلاینت و سرورهای مبتنی بر HTTP که در معرض دسترسی خارجی هستند، از TLS باید استفاده شود و هیچ‌گونه بازگشت (fallback) به ارتباطات ناامن یا بدون رمزنگاری انجام نمی‌گردد.	1
12.2.2	سرورهای که در معرض دسترسی خارجی قرار دارند از گواهی‌های TLS عمومی و مورد اعتماد استفاده می‌کنند.	1

Security Communication Service to Service General V12.3

ارتباطات سرور (چه داخلی و چه خارجی) تنها شامل HTTP نمی‌باشد. ارتباطات به واسطه‌ی بررسی‌ست‌ها نیز باید امن باشند، ترجیحاً با استفاده از TLS.

#	Description	Level
12.3.1	یک پروتکل رمزنگاری شده مانند TLS باید برای تمام ارتباطات ورودی و خروجی به و از برنامه استفاده شود، از جمله سیستم‌های مانیتورینگ، ابزارهای مدیریتی، دسترسی از راه دور و SSH، می‌انافزارها، پایگاه‌های داده، mainframes، سیستم‌های partner یا API‌های خارجی. سرور نباید به پروتکل‌های ناامن یا بدون رمزگذاری بازگردد.	2
12.3.2	کلاینت‌های TLS باید قبل از برقراری ارتباط با سرور، TLS، گواهی‌های دریافتی را اعتبارسنجی کنند.	2
12.3.3	TLS یا دیگر مکانیزم‌های مناسب رمزگذاری انتقال باید برای تمام ارتباطات بین سرورهای داخلی مبتنی بر HTTP درون برنامه استفاده شود و ارتباط به حالت‌های ناامن یا بدون رمزگذاری بازنگردد.	2

#	Description	Level
12.3.4	ارتباطات TLS بین سرویس‌های داخلی باید از گواهی‌های معتبر استفاده کنند. در صورتی که از گواهی‌های داخلی تولید شده یا Self-Signed استفاده شود، سرویس مصرف‌کننده باید به گواهی‌های پی‌کربندی شود که تنها به های CA داخلی مشخص و گواهی‌های Self-Signed خاص اعتماد کند.	2
12.3.5	سرویس‌هایی که درون یک سیستم بایگین‌گر ارتباط دارند (ارتباطات درون‌سرویس)، باید از احراز هویت قوی استفاده کنند تا هر نقطه انتهایی (endpoint) تأیید شود. روش‌های احراز هویت قوی، مانند احراز هویت کلاینت TLS، باید به کار گرفته شوند تا هویت تضمین شود (با استفاده از زیرساخت کلید عمومی (PKI) و مکانیزم‌هایی که در برابر حملات تکرار (replay) (attacks) مقاوم هستند). برای معماریهایی که در برابر حملات تکرار سرویس مشخص برای ساده‌سازی مدیریتی گواهی‌ها و افزایش امنیت در نظر گرفته شود	3

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- Sheet Cheat Security Layer Transport - OWASP
- guide configuration TLS Side Server Mozilla's
- configurations TLS good known generate to tool Mozilla's
- configuration TLS validate to Project OWASP - O-Saft

Configuration V13

هدف

پی‌کربندی پیش‌فرض برنامه برای استفاده در اینترنت باید امن باشد.

این فصل راهنمای‌هایی در مورد پی‌کربندی‌های مختلف لازم برای دستیابی به این هدف را ارائه می‌دهد، از جمله پی‌کربندی‌هایی که در طول توسعه، ساخت و استقرار اعمال می‌شوند. موضوعات پوشش داده شده شامل جلوگیری از نشت داده‌ها، مدیریتی امن ارتباط بین اجزا و حفاظت از secrets است.

Documentation Configuration V13.1

این بخش الزامات مستندسازی را برای نحوه ارتباط برنامه با سرویس‌های داخلی و خارجی، و همچنین تکنیک‌هایی برای جلوگیری از کاهش دسترسی به دلی‌ل عدم دسترسی به سرویس ارائه می‌دهد. همچنین به مستندسازی مرتبط با secrets نیز پرداخته می‌شود.

#	Description	Level
13.1.1	تمامی نیازه‌ای ارتباطی برنامه باید به‌طور کامل مستندسازی شده باشند. این مستندسازی شامل سرویس‌های خارجی که برنامه به آن‌ها وابسته است و همچنین مواردی که کاربر نه‌ای جهت اتصال به سرویس خارجی مشخص می‌کند باشد.	2
13.1.2	برای هر سرویسی که برنامه استفاده می‌کند، مستندسازی باید شامل حداکثر تعداد ارتباطات همزمان (مثلاً محدودیت‌های Connection Pool و نحوه رفت‌وآمد برنامه هنگام رسیدن به این حد باشد، از جمله هر مکانیسم بازیابی جایگزین، تا از بروز شرایط Service of Denial جلوگیری شود.	3
13.1.3	مستندات برنامه باید استراتژی‌های مدیریتی منابع را برای هر سیستمی سرویس خارجی که استفاده می‌کند مشخص کرده باشد (مثلاً پایگاه‌های داده، هندهای فایل، رشته‌ها، اتصالات HTTP این مستندسازی باید شامل این موارد باشد: رویه‌های آزادسازی منابع، تنظیمات تایم‌اوت، مدیریت خطاها، جایی که منطق Retry اعمال می‌شود، شامل محدودیت تعداد Retry، تأخیرها و الگوریتم‌های Back-off. برای عملیات همزمان درخواست-پاسخ HTTP، (Synchronous) مستندات باید تایم‌اوت کوتاه را الزام کند و یا Retry را غیرفعال کند یا به‌طور سخت‌گیرانه محدود کند تا از تأخیرهای زنجیره‌ای و دررفت منابع جلوگیری شود.	3
13.1.4	مستندسازی برنامه باید شامل های secret حیاتی برای امنیت برنامه و همچنین برنامه‌ای برای چرخش آن‌ها باشد (بر اساس مدل تهدید سازمان و نیازه‌ای کسب‌وکار).	3

Configuration Communication Backend V13.2

برنامه‌ها با چندین سرویس از جمله API، پایگاه‌های داده یا سایر اجزای تعامل دارند. این سرویس‌ها ممکن است نسبت به برنامه اجزای داخلی محسوب شوند اما در مکانیسم‌های کنترل دسترسی استاندارد برنامه گنجانده نشده باشند، یا کامل‌اً خارجی باشند. در صورت لزوم، برنامه برای تعامل امن با این اجزای پی‌کربندی شود و در صورت نیاز، این پی‌کربندی نیز محافظت گردد.

توجه: فصل "Secure Communication" راهنمای‌های برای رمزگذاری در حین انتقال را ارائه می‌دهد.

#	Description	Level
13.2.1	رتباطات بین اجزای بک اند برنامه که از مکانیسم استناد دارد نشست کاربران برنامه پشتیبانی نمی کنند، از جمله، APIهای افزایش داده های داده، بایدهای احراز هویت شده باشند. احراز هویت بایدهای احراز هویت سروس چداگانه، توکنهای کوتاه مدت یا احراز هویت مبتنی بر گواهی انجام شود و نبایدهای اعانت بارنامه های ثابت مانند رمز عبور، کلیدهای API یا حسابه های مشترک بایدهای دسترسی های سطح بالا استفاده شود.	2
13.2.2	ارتباطات بین اجزای بک اند، از جمله سروس های محلی یا سیستم عامل، APIهای افزایش داده های داده، بایدهای احراز هویت انجام شود که کمترین سطح دسترسی لازم را دارند.	2
13.2.3	اگر اعانت بارنامه های برای احراز هویت سروس استفاده می شود، اعانت بارنامه های که توسط مصرف کنندنده استفاده می شود، اعانت بارنامه پیش فرض نباشد (مثلاً root/root یا admin/admin).	2
13.2.4	ی allowlist برای تعین منابع یا سیستم های خارجی که برنامه مجاز به ارتباط با آنها است بایدهای وجود داشته باشد (مثلاً برای درخواست های خروجی، بارگذاری داده یا دسترسی به فایل). این allowlist میتواند در لایه برنامه، وب سرور، فایروال یا ترکیبی از لایه های مختلف پیاده سازی شود.	2
13.2.5	وب سرور یا سرور برنامه بایدهای allowlist از منابع یا سیستم های که سرور میتواند به آنها درخواست ارسال کند یا داده ها و فایل ها را بارگذاری کند پی کربندی شده باشد.	2
13.2.6	در مواقعی که برنامه به سروس های چداگانه متصل می شود، پی کربندی مستند سازی شده برای هر اتصال رعایت شود، از جمله حداکثر ارتباطات همزمان، رفتار هنگام رسیدن به حداکثر ارتباط مجاز، زمان های Timeout اتصال و استراتژی های Retry.	3

Management Secret V13.3

مدیریت secret ی که وظیفه پی کربندی حیاتی است تا از حفاظت داده های مورد استفاده در برنامه اطمینان حاصل شود. الزامات خاص مربوط به رمزنگاری در فصل "Cryptography" قابل دسترسی است، اما این بخش بر مدیریت و نحوه برخورد با secret تمرکز دارد.

#	Description	Level
13.3.1	یک راه کار مدیریّت secret، مانند Vault، Key برای ایجاد امن، ذخیره، کنترل دسترسی و از بین بردن secret بک اند است فاده شود. این secret می تواند شامل رمز عبور، داده های رمز نگاری، اطلاعات حساس مربوط به اتصال به پایگاه داده و سیست های مشخص ثالث، کلیدها و های seed توکن های زمان محور، سایر های secret داخلی و کلیدهای API باشد. Secret نباید در کد منبع برنامه یا در build artifacts قرار گیرد. برای برنامه های L3، این باید شامل راه کاری مبتنی بر سخت افزار مانند HSM باشد.	2
13.3.2	دسترسی به دارایی های secret بر اساس اصل حداقل دسترسی (Least Privilege) رعایت شود.	2
13.3.3	تمام عملیات رمز نگاری باید با استفاده از یک ماژول امنیتی ایزوله (مانند Vault یا Security Hardware Module) انجام شود تا مواد کلیدی به طور امن مدیریّت و محافظت شده و از افشای خارج از ماژول امنیتی جلوگیری شود.	3
13.3.4	secret بر اساس مستندسازی برنامه طوری پی کربندی شوند که منقضی شده و چرخش داده شوند.	3

Leakage Information Unintended V13.4

پی کربندی های production باید به گونه ای امن شوند که از افشای داده های غیرضروری جلوگیری شود. بسیاری از این مسائل به ندرت به عنوان ریسک های مهم ارزیابی می شوند، اما اغلب با سایر آسیب پذیری ها زنجیره ای می شوند. اگر این مسائل به صورت پیش فرض وجود نداشته باشند، سطح دشواری حمله به برنامه افزایش می یابد.

به عنوان مثال، مخفی کردن نسخه ای اجزای سرور باعث حذف نیازی به به روزرسانی تمام اجزای نمی شود، و غیرفعال کردن نمایش فهرست پوشه ها نیازی به استفاده از کنترلهای دسترسی یا نگه داری فایل ها دور از پوشه عمومی را از بین نمی برد، اما سطح دشواری حمله را افزایش می دهد.

#	Description	Level
13.4.1	برنامه باید به گونه ای مستقر شود که هیچ گونه متادیتای کنترل نسخه، از جمله پوشه های git، یا svn، در دسترس نباشد یا به شکلی باشد که این پوشه ها نه از بیرون و نه توسط خود برنامه قابل دسترسی نباشند.	1
13.4.2	حالت های Debug برای تمام اجزا در محیط های production باید غیرفعال شده باشند تا از افشای قابلهای دیباگ و نشات اطلاعات جلوگیری شود.	2
13.4.3	وب سرورها فهرست بندی دایرکتوری ها را به کاربر نهایی نباید نمایش دهند مگر این که به طور صریح مورد نظر باشد.	2
13.4.4	از متد TRACE HTTP در محیط های production نباید پشتیبانی شود تا از احتیاط نشات اطلاعات جلوگیری شود.	2

#	Description	Level
13.4.5	مستندسازی (مانند مستندات های API داخلی) و نقاط پایش (Monitoring Endpoints) در دسترس نباشند مگر این که به طور صریح مورد نظر باشد.	2
13.4.6	تأیید کنی که برنامه اطلاعات نسخه دقیقی اجزای backend را افشا نکند.	3
13.4.7	لایه وب باید به گونه ای پی کربندی شده باشد که فقط فایلهای با پسوندهای مشخص را سرو کند تا از افشای تصادفی اطلاعات، پی کربندی و کد منبع جلوگیری شود.	3

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

Testing Management Deployment and Configuration Guide: Testing Security Web OWASP •

Protection Data V14

هدف

برنامه ها نمی توانند تمام الگوهای استفاده و رفتارهای کاربران را پیش بینی کنند، بنابراین باید کنترل های را پیاده سازی کنند تا از دسترسی غیرمجاز به داده های حساس روی دست گاه های کاربر جلوگیری شود.

این فصل شامل الزاماتی است که چه داده های باید محافظت شوند، چگونه باید محافظت شوند، و همچنین مکانیسم های خاصی که باید پیاده سازی شوند یا اشتباهاتی که باید از آنها اجتناب شود.

یکی دیگراز مواردی که باید در حفاظت از داده ها در نظر گرفت، استخر ارجع مده، تغیری است فاده بی ش از حد از داده ها است. الزامات هر سیستم احتیاطاً بر اساس تفاوت خواهد بود، بنابراین تعینی این که چه چیزی «غیرعادی» محسوب می شود باید بر اساس مدل تهدید و ریسک کسب و کار انجام گیرد. از دیدگاه ASVS، شناسایی این مسائل در فصل «Security Error and Logging» Handling بررسی می شود و تعینی محدودیت ها در فصل «Validation» Business and «Logic» مطرح شده است.

Documentation Protection Data V14.1

یکی از پیش بینی های اصلی برای حفاظت از داده ها، دسته بندی داده های است که باید به عنوان داده حساس در نظر گرفته شوند. به احتیاط زیاد چند سطح متفاوت از حساسیت وجود خواهد داشت و برای هر سطح، کنترل های لازم جهت حفاظت از داده ها متفاوت خواهد بود.

مقررات و قوانین مخترلفی در حوزه حریم خصوصی وجود دارند که بر نحوه ذخیره سازی، استفاده و انتقال داده های شخصی حساس توسط برنامه ها تأثیر می گذارند. این بخش دیگرتلاشی برای

تکرار چنانچه قوانین یا مقرراتی در زمینه حفاظت از داده‌های خصوصی نمی‌کند، بلکه تمرکز آن بر ملاحظات فنی کلیدی برای حفاظت از داده‌های حساس است. لطفاً قوانین و مقررات محلی را بررسی کرده و در صورت نیاز با یکی که متخصیص حری خصوصی یا وکیل واجد شرایط مشورت کنید.

#	Description	Level
14.1.1	تمامی داده‌های حساس ایجاد شده و پردازش شده توسط برنامه باید شناسایی شده و بر اساس سطوح حفاظتی طبقه‌بندی شده‌اند. این موضوع شامل داده‌ای می‌شود که صرفاً کُدگذاری شده‌اند و به راحتی قابل بازیابی هستند، مانند رشته‌های Base64 یا محتوای متنی (plaintext) داخلی که JWT. سطوح حفاظتی باید الزامات قوانین و استانداردهای حفاظت از داده و حریم خصوصی که برنامه ملزم به رعایت آن‌هاست را در نظر بگیرند.	2
14.1.2	برای تمامی سطوح حفاظت از داده‌های حساس، مجموعه‌ای مستند از الزامات حفاظتی تعریف شده باشد. این الزامات باید شامل (اما محدود به آن نباشد) موارد زیر باشند: الزامات مرتبط با رمزنگاری کلی، بررسی صحت و یکپارچگی، دوره‌های نگهداری داده، نحوه ثبت داده‌ها در لاگ، کنترل‌های دسترسی به داده‌های حساس در لاگ‌ها، رمزنگاری در سطح پایگاه داده، الزامات مربوط به حریم خصوصی و فناوری‌های تقویت کننده حریم خصوصی مورد استفاده، و سایر الزامات مرتبط با محرمانگی.	2

Protection Data General V14.2

این بخش شامل مجموعه‌ای از الزامات عملی برای حفاظت از داده‌ها می‌باشد. بیشترین این الزامات به مسائلی خاصی مانند نشست ناخواسته داده‌ها مربوط می‌شوند، اما یکی الزام کلی نیست و وجود دارد که بر پیاده‌سازی کنترل‌های حفاظتی بر اساس سطح حفاظت موردنیاز برای هر داده تأکید دارد.

#	Description	Level
14.2.1	داده‌های حساس باید فقط در بدنه پیام HTTP یا فیلدهای هدر به سرور ارسال شوند، و این‌که URL و string query شامل اطلاعات حساس (مانند key API یا session token) نباشند.	1
14.2.2	برنامه باید از cache شدن داده‌های حساس در اجزای سرور مانند Balancer Load و Cache Application جلوگیری کند، یا اطمینان حاصل شود که داده‌ها پس از استفاده به‌طور ایمن پاک‌سازی می‌شوند.	2
14.2.3	داده‌های حساس تعریف شده به طرفه‌های غیرقابل اعتماد (مانند User Trackers) نباید ارسال شوند تا از جمع‌آوری ناخواسته داده‌ها خارج از کنترل برنامه جلوگیری شود.	2

#	Description	Level
14.2.4	کنترلهای مرتبط با داده‌های حساس شامل رمزنگاری، بررسی صحت و یکپارچگی، نگه‌داری، نحوه ثبت داده‌ها در لاگ، کنترلهای دسترسی به داده‌های حساس در لاگ‌ها، حریم خصوصی و فناوری‌های مربوط به حریم خصوصی، مطابقت با آنچه در مستندات برای سطح حفاظت مشخص هر داده تعریف شده است، پیاده‌سازی شوند.	2
14.2.5	مکانیسم‌های cache باید طوری پی‌کربندی شده باشند که تنها پاسخ‌های با نوع محتوای مورد انتظار برای آن منبع cache شوند و شامل محتوای حساس یا پویا نباشند. وب‌سرور باید هنگام دسترسی به فایل ناموجود، پاسخ 404 یا 302 برگرداند و نه فایل معتبر دیگری؛ تا از حملات Cache Web Deception جلوگیری شود.	3
14.2.6	برنامه باید تنها حداقل داده‌های حساس مورد نیاز برای عملکرد برنامه را بازگرداند. برای مثال، تنها برخی از ارقام شماره کارت اعتباری بازگردانده شود و نه شماره کامل. در صورتی که داده کامل لازم باشد، باید در رابط کاربری masked شود مگر اینکه کاربر به‌طور مشخص آن را مشاهده کند.	3
14.2.7	اطلاعات حساس باید تحت طبقه‌بندی دوره‌ای نگه‌داری داده قرار گرفته باشند، به گونه‌ای که داده‌های قدیمی یا غیرضروری به‌طور خودکار، در یک برنامه زمان‌بندی شده مشخص، یا براساس نیاز حذف شوند.	3
14.2.8	اطلاعات حساس باید از متادیتای فایل‌های ارسال شده توسط کاربر حذف شود، مگر اینکه ذخیره‌سازی آن با رضایت کاربر انجام شود.	3

Protection Data Client-side V14.3

این بخش شامل الزامات جلوگیری از نشت داده‌ها به روش‌های خاص در سمت کلاینت یا سمت User Agent برنامه است.

#	Description	Level
14.3.1	داده‌های احراز هویت شده از حافظه کلاینت، مانند DOM مرورگر، پس از خروج کلاینت یا Session پاک شوند. فیلدها در Clear-Site-Data در پاسخ مرورگر می‌تواند در این زمینه مفید باشد، اما کلاینت باید بتواند داده‌ها را حتی زمانی که اتصال به مرورگر در دسترس نیست در زمان پایان Session (خروج از آن) پاک کند.	1
14.3.2	برنامه باید هدرهای پاسخ، anti-cach HTTP، مناسب (مانند: Cache-Control: no-store) را تنظیم کند تا داده‌های حساس در مرورگرها cache نشوند.	2
14.3.3	برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:	2

References

also: see information, more For

- Consider the using Headers Security to website anti-caching and security check fields
- Documentation about anti-caching headers by Mozilla
- OWASP Secure Headers project
- OWASP Privacy Risks Project
- OWASP User Privacy Protection Cheat Sheet
- Australian Privacy Principle 11 - of personal information
- European Union General Data Protection Regulation (GDPR) overview
- European Union Data Protection Supervisor - Internet Privacy Engineering Network
- Information on the "Clear-Site-Data" header
- Deception Cache Web on paper White

Architecture and Coding Secure V15

Objective Control

بسیاری از الزامات ASVS یا به یک حوزه خاص امنیتی مانند احراز هویت یا مجزده‌ی وی به یک نوع خاص از کارکردهای نرم‌افزار مانند ثبت رخ داده‌ی مدیریت فایل مرتبط می‌باشند. این فصل الزامات کلی امنیتی که باید هنگام طراحی و توسعه نرم‌افزارها در نظر گرفته شوند را ارائه می‌دهد. این الزامات نه تنها بر معماری تمیزی و کیفیتی کد تمرکز دارند، بلکه شامل شیوه‌های خاص معماری و کنوایی نیز می‌شوند که برای امنیتی نرم‌افزار ضروری هستند.

Documentation Architecture and Coding Secure V15.1

بسیاری از الزامات مربوط به ایجاد یک معماری امن و قابل دفاع، به مستندسازی شفاف تصمیم‌گیری بستگی دارند که در مورد پیاده‌سازی کنترل‌های امنیتی مشخص و مؤلفه‌های استفاده‌شده در نرم‌افزار اتخاذ می‌شوند.

این بخش، الزامات مستندسازی را شرح می‌دهد، از جمله شناسایی مؤلفه‌های «عمل کرد خطرناک» در نظر گرفته می‌شوند یا به عنوان «مؤلفه‌های پرریسک» شناخته می‌شوند.

یک مؤلفه با «عمل کرد خطرناک» ممکن است یکی از مؤلفه توسعه یافته داخلی یا شخص ثالث باشد که عملیات‌های مانند deserialization داده‌های غیرقابل اعتماد، پردازش داده‌های خام یا باینری، اجرای کد به صورت پویا، یا دستکاری مستقیم حافظه را انجام می‌دهد. آس‌ب‌پذیری‌ها در این نوع عملیات‌ها ریسک بالایی برای به خطر انداختن برنامه و احتمالاً افشای زیرساخت زیربنای آن ایجاد می‌کنند.

یک «مؤلفه پرریسک» یکی که کتابخانه شخص ثالث است (یعنی توسعه یافته داخلی نیست) که کنترل‌های امنیتی در فرآیند توسعه یا عمل کرد آن وجود ندارد یا ضعیف پیاده‌سازی شده‌اند.

نمونه‌ها شامل مؤلفه‌های هستند که به‌خوبی نگه‌داری نمی‌شوند، پشتیبانی نمی‌شوند، در مرحله end-of-life قرار دارند، یا سابقه‌ای از آسی‌ب‌پذیری‌های جدی دارند.

این بخش همچنین برای اهمیت تعینی بازه‌های زمانی مناسب برای رسی‌دگی به آسی‌ب‌پذیری‌ها در مؤلفه‌های مشخص ثالث تأکید می‌کند.

#	Description	Level
15.1.1	مستندات برنامه باید بازه‌های زمانی جهت برطرف سازی آسی‌ب‌پذیری نسخه‌های مؤلفه مشخص ثالث (بر اساس ریسک) و همچنین به‌روزرسانی کتابخانه‌ها را به‌طور کلی تعریف کرده باشد تا ریسک ناشی از این مؤلفه‌ها به حداقل برسد.	1
15.1.2	یک کاتالوگ دارایی مانند Materials of Bill Software (SBOM) باید برای تمامی کتابخانه‌های مشخص ثالث مورد استفاده نگه‌داری شود. شامل تاییدی که مؤلفه‌ها از مخازن پیش‌ش تعینی نشده، قابل اعتماد و به‌طور مستمر نگه‌داری شده تهیه شده‌اند.	2
15.1.3	مستندات برنامه باید شامل عمل‌کرده‌ای که زمان‌بری یا پرمصرف (منابع سیستمی) هستند نیز باشد. این مورد باید شامل روش‌های برای جلوگیری از دست‌رفتن دسترس‌پذیری به دلیلی استفاده بیش از حد از این عمل‌کردها و همچنین جلوگیری از وضعیتی باشد که پردازش یک پاسخ طولانی‌تر از زمان انتظار مصرف‌کننده شود. تدابیر احتیاطی می‌تواند شامل پردازش غیروهم‌زمان، استفاده از صف‌ها و محدود کردن فرآیندهای موازی برای هر کاربر و هر برنامه باشد.	2
15.1.4	مستندات برنامه باید کتابخانه‌های مشخص ثالثی را که به‌عنوان «کامپوننت‌های پرخطر» شناخته می‌شوند، مشخص کرده باشد.	3
15.1.5	مستندات برنامه باید بخش‌های از برنامه را که به‌عنوان «عمل‌کرد خطرناک» استفاده می‌شود، مشخص کرده باشد.	3

Dependencies and Architecture Security V15.2

این بخش شامل الزامات مربوط به مدیریتی وابستگی‌ها و کامپوننت‌ها می‌باشد تا اجزای پرخطر، قدیمی یا ناامن به‌طور مؤثر شناسایی، کنترل و اداره شوند.

همچنین شامل استفاده از تکنیک‌های سطح معماری مانند containerization encapsulation، sandboxing، و ای‌زوله‌سازی شبکه می‌باشد تا تأثیر استفاده از «عملیات خطرناک» (Dangerous operations) یا «کامپوننت‌های پرخطر» (dangerous operations) (همان‌طور که در بخش قبلی تعریف شد) کاهش یابد و از کاهش دسترس به دلیلی استفاده بیش از حد از عمل‌کردهای پرمصرف منابع جلوگیری شود.

#	Description	Level
15.2.1	برنامه بای د تنه شامل کامپوننت‌های باشد که چارچوب‌های زمانی مستند شده برای به‌روزرسانی و رفع آسی‌ب‌پذیری‌ها را نقض نکرده باشند.	1
15.2.2	برنامه بای د بر اساس تصمیمات و استراتژی‌های امنیتی مستند سازی شده، تدابیری برای جلوگیری از کاهش دسترس‌پذیری ناشی از عمل‌کردهای که زمان‌بری یا پرمصرف منابع هستند، پیاده‌سازی کرده باشد.	2
15.2.3	محیط production تنه شامل عمل‌کردهای می‌باشد که برای اجرای برنامه ضروری هستند و هیچ قابلیت اضافی مانند کدهای تست، نمونه‌کدها و عمل‌کردهای توسعه‌ای در دسترس نمی‌باشد.	2
15.2.4	کامپوننت‌های مشخص ثالث و تمام وابستگی‌های انت‌قالی آنها از مخزن مورد انت‌ظار بای د دریافت شوند، چه داخلی و چه منبع خارجی، وریسک حمله‌ی confusion dependency مدیریت شود.	3
15.2.5	برنامه بای د محافظت‌های اضافی را در بخش‌های از برنامه که به‌عنوان «عمل‌کرد خطرناک» مستند شده‌اند یا از کتابخانه‌های مشخص ثالثی که به‌عنوان «کامپوننت‌های پرریسک» شناخته می‌شوند، پیاده‌سازی می‌کند. این می‌تواند شامل تکنیک‌های مانند encapsulation، sandboxing، containerization یا جداسازی در سطح شب‌که باشد تا حمله‌کنندگان پس از نفوذ به یکی که بخش از برنامه، نتوانند به بخش‌های دیگر برنامه منت‌قل شوند و زمان لازم برای پی‌ش‌روی آنها افزایش یابد.	3

Coding Defensive V15.3

این بخش انواع آسی‌ب‌پذیری‌ها را پوشش می‌دهد، از جمله pollution prototype juggling، type، و سایر مواردی که ناشی است‌فاده از الگوهای ناامن برنامه‌نویسی در یک زبان خاص هستند. برخی از این آسی‌ب‌پذیری‌ها ممکن است برای همه زبان‌ها مرتبط نباشند، در حالی که برخی دیگر راه‌حل‌های خاص زبان دارند یا به نحوه مدیریت ویژگی‌های مانند پارامترهای HTTP توسط یک زبان یا فری‌مورک خاص مربوط می‌شوند. همچنین، این بخش به خطر عدم اعت‌بارسنجی (رمزنگاری شده) به‌روزرسانی‌های برنامه نیز می‌پردازد.

این بخش همچنین خطرات مرتبط با استفاده از Objects برای نمایش داده‌ها و پ‌پذیرش و بازگرداندن آنها از طریق‌های API خارجی را مورد بررسی قرار می‌دهد. در این حالت، برنامه بای د اطمینان حاصل کند که فیلدهای داده‌ای که نباید قابل ویرایش باشند توسط ورودی کاربر تغیر نمی‌کنند (mass assignment) و API به‌صورت انت‌خابی مشخص کند که کدام فیلدهای داده بازگردانده شوند. هرچایی که دست‌رسی به فیلدها به مجوزه‌ای کاربر وابسته باشد، این موضوع بای د در چارچوب الزامات کنترول دست‌رسی در سطح فیلدها در فصل Authorization در نظر گرفته شود.

#	Description	Level
15.3.1	برنامه باید تنه‌های زیرمجموعه‌ای از فیلدهای موردنیاز از یک object data را بازگرداند. به عنوان مثال، نباید کل object data بازگردانده شود، زیرا برخی فیلدهای منفرد نباید برای کاربران قابل دسترسی باشند.	1
15.3.2	درجایی که بک‌اند برنامه به URL خارجی فرخوانی می‌فرستد، طوری باید پی‌کربندی شده باشد که تنه‌ها در صورت نیازی و منظور مشخص، از تغیری مسیره (redirects) پی‌روی کنند.	2
15.3.3	برنامه باید دارای تدابیری مقابله‌ای برای محافظت در برابر حملات mass assignment باشد، به طوری که فیلدهای مجاز برای هر کنترلر و اکشن محدود شده باشند؛ به عنوان مثال، نباید بتوان مقادیری برای فیلدهای یا به‌روزرسانی کرد که قرار نبوده بخش‌های آن اکشن باشد.	2
15.3.4	تمام اجزای proxying و middleware باید آدرس IP اصلی کاربر را به‌درستی منتقل کنند، با استفاده از فیلدهای داده‌معتبر که توسط کاربر نهایی قابل دست‌کاری نباشند، و برنامه و وب‌سرور از این مقدار صحیح برای ثبت لاگ و تصمیم‌گیری‌های امنیتی مانند limiting rate استفاده کنند، با در نظر گرفتن اینکه حتی آدرس IP اصلی هم ممکن است به دلیلی IP پویا، VPN‌ها یا فایروال‌های سازمانی قابل اعتماد نباشد.	2
15.3.5	برنامه باید به‌طور صریح اطمینان حاصل کند که variables از نوع صحیح هستند و عملیات‌های مقایسه و comparator به صورت دقیقی (strict) انجام می‌شوند. این اقدام برای جلوگیری از آسیبی‌پذیری‌های juggling type یا type confusion است که ممکن است به دلیلی فرضیات نادرست برنامه درباره نوع یک متغیر ایجاد شود.	2
15.3.6	کد JavaScript باید به گونه‌ای نوشته شده باشد که از pollution prototype جلوگیری کند، برای مثال با استفاده از Set() یا Map() به جای object literals.	2
15.3.7	برنامه باید دارای تدابیری محافظتی در برابر HTTP pollution parameter باشد، به‌ویژه اگر چارچوب برنامه تفاوت‌هایی بین منابع پارامترهای درخواست (رشته‌های query، پارامترهای body، کوکی‌ها یا فیلدهای درج‌قائل نشده).	2

Concurrency Safe V15.4

مسائلی هم‌زمانی مانند race condition، آسی‌پذیری‌های time-of-use to time-of-check (TOCTOU)، deadlock، starvation thread livelock، و هم‌گام‌سازی نادرست می‌توانند منجر به رفتار غیرقابل پیش‌بینی و خطرات امنیتی شوند. این بخش شامل تکنیک‌ها و استراتژی‌های مختل‌فی است که به کاهش این ریسک‌ها کمک می‌کنند.

#	Description	Level
15.4.1	objects shared در کد multi-threaded (مانند، cache، فایله، یا اشیائی که توسط threads multiple دسترسی پیدا می کنند) باید به صورت ایمن مورد استفاده قرار بگیرند، با استفاده از نوع های thread-safe و مکانیسم های همگام سازی مانند lock یا semaphore تا از condition race و تخریب داده جلوگیری شود.	3
15.4.2	بررسی وضعیتی که resource، مانند وجود آن یا دسترسی ها، و اقداماتی که به این وضعیت وابسته اند، به صورت یک عملیات واحد انجام شوند تا از condition race نوع time-of-use to time-of-check (TOCTOU) جلوگیری شود. برای مثال، بررسی اینکه یک فایل وجود دارد قبل از باز کردن آن، یا تأیید دسترسی که کاربر قبل از اعطای آن.	3
15.4.3	باید از قفلها (Locks) به طوری کنواخت استفاده شود تا از گریز کردن (Threads) جلوگیری شود، چه با انتظار برای یکدیگر چه با تلاش بی پایان، و همچنین منطق قفل گذاری باید در همان کدهای باقی بماند که مسئول مدیریت منابع است؛ این کار تضمین می کند که قفلها نمیتوانند به صورت ناخواسته یا خراب کارانه توسط کلاس ها یا کدهای بیرونی تغیری داده شوند.	3
15.4.4	تأیید کنید که سیاست های تخصیص resource از starvation thread جلوگیری می کنند، با اطمینان از دسترسی عادلانه به منابع، مانند استفاده از pools thread و اجازه دادن به threads با اولویت پائین تر تا در بازه زمانی منطقی پیشرفت کنند.	3

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- OWASP Pollution Prevention Cheat Sheet
- OWASP Mass Assignment Prevention Cheat Sheet
- OWASP CycloneDX Bill of Materials Specification
- OWASP Web Security Testing Guide: HTTP Parameter Pollution

Handling Error and Logging Security V16

هدف

لاگ های امنیتی با لاگ های خطای لاگ های عمل کرد متفاوت هستند و برای ثبت روی داده ای مرتبط با امنیتی استفاده می شوند؛ مانند تصمیحات مربوط به احراز هویت، کنترل دسترسی، و تلاش ها برای دور زدن کنترل های امنیتی (مانند اعتبارسنجی ورودی یا اعتبارسنجی منطق تجاری). هدف

آنها پشت‌بانی از تشخیص‌ص، پاسخ و بررسی است، از طریق ارائه داده‌های ساختاریافته و بایگ‌یفیت بالا برای ابزارهای تحلیلی مانند SIEM.

لاگ‌ها نباید شامل داده‌های شخصی حساس باشند مگر در مواردی که قانون الزام کند. هر داده‌ای که لاگ می‌شود باید به‌عنوان یک دارایی بسیار با ارزش محافظت شود. لاگ‌برداری نباید باعث به‌خطر افتادن حریم خصوصی یا امنیت سیستم شود. همچنین برنامه باید در شرایط به‌صورت ایمن عمل کند و از افشای غیرضروری اطلاعات یا ایجاد اختلال جلوگیری کند.

برای راهنمای دقیقی در مورد پیاده‌سازی، به Sheets Cheat OWASP در بخش منابع مراجعه کنید.

Documentation Logging Security V16.1

این بخش تضمین می‌کند که یک دارایی (Inventory) شفاف و کامل از لاگ‌گیری در سراسر stack برنامه وجود داشته باشد. این کار برای نظارت امنیتی مؤثر، پاسخ به حوادث و انطباق (Compliance) ضروری است.

#	Description	Level
16.1.1	فهرستی باید وجود داشته باشد که در آن ثبت وقایع (Logging) انجام شده در هر لایه از stack فناوری برنامه، روی داده‌ای ثبت شده، فرمت‌های لاگ، محل ذخیره‌سازی، نحوه استفاده، نحوه کنترل دسترسی به آنها و مدت زمان نگهداری لاگ‌ها مستند شده است.	2

Logging General V16.2

این بخش الزامات لازم را ارائه می‌دهد تا اطمینان حاصل شود که لاگ‌های امنیتی به‌صورت یکپارچه ساختاردهی شده‌اند و متادیتا مورد انتظار را در خود دارند. هدف این است که لاگ‌ها قابل‌خواندن توسط ماشین باشند و بتوان آنها را در سیستم‌ها و ابزارهای توزیع‌شده تحلیلی کرد.

به‌طور طبیعی، روی داده‌ای امنیتی اغلب شامل داده‌های حساس هستند. اگرچنین داده‌ای بدون ملأحظه در لاگ‌ها ثبت شود، خود لاگ‌ها به‌عنوان داده‌ای طبقه‌بندی شده محسوب می‌شوند و بنابراین مشمول الزامات رمزنگاری، سیاست‌های سخت‌گیرانه‌تر نگهداری و همچنین احتیاط افشا در طول ممیزی‌ها خواهند بود.

بنابراین، بسیار مهم است که فقط موارد ضروری در لاگ‌ها ثبت شوند و داده‌های لاگ با همان دقت و مراقبتی که برای ساینی‌های حساس به‌کار می‌رود، مورد توجه قرار گیرند.

الزامات زیر، نیازمندی‌های پای‌های مربوط به متادیتا در لاگ‌ها، هم‌گام‌سازی، فرمت و کنترل را مشخص می‌کنند.

#	Description	Level
16.2.1	هر ورودی لاگ باید شامل متادیتای لازم (مانند زمان، مکان، چه کسی و چه چیزی) باشد تا امکان بررسی دقیق خط زمانی هنگام وقوع یک رویداد فراهم شود.	2
16.2.2	منابع زمانی برای تمام مؤلفه‌های لاگ با هم همگام‌سازی شده‌اند و مهر زمان (timestamp) در متادیتای روی داده‌های امنیتی از UTC استفاده کنند یا شامل یک اختلال زمان (time zone (offset صریح باشد. استفاده از UTC توصیه می‌شود تا یک پارچگی در سراسر سیستم‌های توزیع شده تضمین شود و از بروز سردرگمی در زمان تغیری ساعت تابستانی جلوگیری گردد.	2
16.2.3	برنامه‌تنها لاگ‌ها را در فایل‌ها و سرویس‌های ذخیره‌ای ارسال می‌کنند که در فهرست مستند لاگ‌ها ثبت شده‌اند.	2
16.2.4	لاگ‌ها باید توسط پردازشگر لاگی که در حال استفاده است خوانده و همبسته شوند، و ترجیحاً با استفاده از یک فرمت لاگ مشترک این کار انجام گیرد.	2
16.2.5	هنگام لاگ‌گیری از داده‌های حساس، برنامه باید لاگ‌گیری را بر اساس سطح حفاظت داده اعمال کند. برای مثال، ممکن است لاگ کردن برخی داده‌ها مانند اطلاعات ورود یا جزئیات پرداخت مجاز نباشد. داده‌های دی‌گر، مانند token، session، تنها در صورتی باید لاگ شوند که به صورت hash یا masked، چه به‌طور کامل یا جزئی، ثبت شوند.	2

Events Security V16.3

این بخش الزامات مربوط به لاگ‌برداری از روی داده‌ای مرتبط با امنیت در داخل برنامه را تعریف می‌کند. ثبت این روی داده‌ها برای شناسایی رفتارهای مشکوک، پشتیبانی از تحقیقات، و انجام تعهدات مربوط به انطباق است.

این بخش انواع روی داده‌ای که باید لاگ شوند را مشخص می‌کند، اما سعی ندارد جزئیات کامل و جامعی ارائه دهد. هر برنامه دارای عوامل ریسک و زمینه عملیاتی منحصر به فرد خود است.

توجه داشته باشید ASVS در حالی شامل لاگ‌گیری روی داده‌ای امنیتی در محدوده خود می‌شود که Alerting و Correlation (مثلاً قوانین SIEM یا زیرساخت‌های مانیتورینگ) خارج از محدوده در نظر گرفته شده و توسط سیستم‌های عملیاتی و نظارتی مدیریت می‌شوند.

#	Description	Level
16.3.1	تمام عملیات احراز هویت باید لاگ‌گیری شوند، از جمله تلاش‌های موفق و ناموفق. همچنین باید متادیتای اضافی، مانند نوع احراز هویت یا عوامل استفاده شده، جمع‌آوری شود.	2

#	Description	Level
16.3.2	تلاش‌های ناموفق برای مجوزدهی (authorization) باید لاگ‌گیری شوند. برای برنامه‌های L3 این باید شامل لاگ‌گیری تمام تصمیمات مجوزدهی باشد، از جمله ثبت زمانی که داده‌های حساس دسترسی پیدا می‌کنند، بدون آنکه خود داده‌های حساس ثبت شوند.	2
16.3.3	برنامه باید روی داده‌ای امنیتی تعریف شده در مستندات (ابتدای بخش) را لاگ نماید و همچنین تلاش‌ها برای دور زدن کنترل‌های امنیتی، مانند اعتراف بارسنجی ورودی، منطق کسب و کار و ضدخودکارسازی، را نیز ثبت کند.	2
16.3.4	برنامه باید خطاهای غیرمنتظره و شکست‌های کنترل‌های امنیتی، مانند شکست‌های TLS در بک‌اند را لاگ کند.	2

Protection Log V16.4

لاگ‌ها به عنوان آثار ارزشمند forensic محسوب می‌شوند و باید محافظت شوند. اگر لاگ‌ها به سادگی قابل تعویض یا حذف باشند، یک پارچگی خود را از دست می‌دهند و برای تحقیقات حادثه یا پرونده‌های حقوقی غیرقابل اعتماد می‌شوند. همچنین، لاگ‌ها ممکن است رفتار داخلی برنامه یا متادیتای حساس را فاش کنند و این موضوع آن‌ها را به هدف جذابی برای مهاجمان تبدیل می‌کند. این بخش الزامات را برای اطمینان از حفاظت لاگ‌ها در برابر دسترسی غیرمجاز، دستکاری و افشای تعریف می‌کند و همچنین تضمین می‌کند که لاگ‌ها به صورت ایمن منتقل و در سیستم‌های امن و ایزوله ذخیره شوند.

#	Description	Level
16.4.1	تمام مؤلفه‌های لاگ‌گیری باید داده‌ها را به‌طور مناسب رمزگذاری کنند تا از حملات injection log جلوگیری شود.	2
16.4.2	لاگ‌ها باید در برابر دسترسی غیرمجاز محافظت شده و امکان تعویض آن‌ها وجود نداشته باشد.	2
16.4.3	لاگ‌ها به صورت امن باید به یک سیستم منطقی جداگانه برای تحلیل، تشخیص، هشدار و ارجاع منتقل شوند. هدف این است که در صورت نفوذ به برنامه، لاگ‌ها به خطر نیفتند.	2

Handling Error V16.5

این بخش الزامات لازم را برای اطمینان از این که برنامه‌ها به صورت امن و بدون افشای جزئیات حساس داخلی دچار خطا می‌شوند، تعریف می‌کند.

#	Description	Level
16.5.1	هنگام بروز خطاهای غیرمنتظره یا حساس به امنیتی، پیامی عمومی به کاربر بایده بازگردانده شود، به گونه ای که هیچ گونه اطلاعات حساس داخلی سیستم مانند stack، trace، کوئری ها، کلیدها و توکن های محرمانه افشا نشود.	2
16.5.2	برنامه بایدهنگام بروز خطا در دسترس به منابع خارجی همچنان به صورت امن عمل نماید، برای مثال با استفاده از الگوهایی مانند breaker circuit یا کاهش تدریجی عمل کرد (graceful degradation).	2
16.5.3	برنامه بایده به صورت امن و کنترل شده دچار خطا شود، حتی هنگام وقوع استثنا، و از شرایط fail-open جلوگیری نماید؛ برای مثال، از پردازش تراکنش در صورت بروز خطا در منطق اعتبارسنجی جلوگیری شود.	2
16.5.4	یک «last resort» بایده تدریج شده باشد که تمام exception unhandled را مدیریت کند. این کار هم برای جلوگیری از دست رفتن جزئیات خطا که بایده در فایلهای لاگ ثبت شوند و هم برای اطمینان از این است که خطا باعث از کار افتادن کل فرآیند برنامه و در نتیجه از دست رفتن دسترس نشود، ضروری است.	3

توجه: برخی زبانها (از جمله Swift و Go) و همچنین بسیاری از زبانهای تابعی بر اساس رویه های متداول طراحی از exceptions یا last resort پشتیبانی نمی کنند. در این موارد، معماران و توسعه دهندگان بایده از یک الگو، زبان، یا روش سازگار با چارچوب کاری استفاده کنند تا اطمینان حاصل شود که برنامه ها میتوانند روی داده های استثنائی، غیرمنتظره یا مرتبط با امنیتی را به صورت امن مدیریت کنند.

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- Handling Error for Testing Guide: Testing Security Web OWASP
- messages error about section Sheet Cheat Authentication OWASP
- Sheet Cheat Logging OWASP
- Sheet Cheat Vocabulary Logging Application OWASP

WebRTC V17

هدف

Web Communication Real-Time Web (WebRTC) امکان تبادل صوت، تصویری و داده به صورت لحظه ای در برنامه های مدرن را فراهم می کند. با افزایش پذیرش این فناوری، تأمین امنیتی زیرساخت WebRTC

اهمیت حیاتی پیدایمی کند. این بخش الزامات امنیتی را برای ذی‌نفعانی ارائه می‌دهد که سیستم‌های WebRTC را توسعه، میزبانی یا یکپارچه می‌کنند

صنعت WebRTC را می‌توان به‌طور کلی به سه بخش تقسیم کرد:

1. توسعه‌دهندگان محصول: فروشندگان اختصاصی و متنباز که محصولات و راه‌لهای WebRTC را ایجاد و عرضه می‌کنند. تمرکز آنها بر توسعه فناوری‌های قوی و امن WebRTC است که توسط دی‌گران قابل استفاده باشند.

2. ارائه‌دهندگان خدمات: سازمان‌هایی که از محصولات توسعه‌دهندگان یا ارائه‌دهندگان CPaaS استفاده می‌کنند یا راه‌کارهای WebRTC خود را توسعه می‌دهند. آنها برنامه‌هایی را برای کنفرانس آنلاین، سلامت، آموزش الکترونیکی و سایر حوزه‌هایی که در آنها ارتباط بلندرنگ حیاتی است، ایجاد و پیاده‌سازی می‌کنند.

3. ارائه‌دهندگان خدمات: سازمان‌هایی که از محصولات توسعه‌دهندگان یا ارائه‌دهندگان CPaaS استفاده می‌کنند یا خود راه‌کارهای WebRTC خود را توسعه می‌دهند. آنها برنامه‌هایی برای کنفرانس‌های آنلاین، حوزه سلامت، آموزش الکترونیکی و سایر زمینه‌هایی که ارتباط بلندرنگ حیاتی است، ایجاد و پیاده‌سازی می‌کنند.

نیازمندی‌های امنیتی مطرح شده در اینجا عمدتاً بر توسعه‌دهندگان محصول، ارائه‌دهندگان CPaaS و ارائه‌دهندگان خدمات تمرکز است که:

- از راه‌لهای متنباز برای ساخت برنامه‌های WebRTC خود استفاده می‌کنند.
- از محصولات تجاری WebRTC به‌عنوان بخشی از زیرساخت خود استفاده می‌کنند.
- از راه‌لهای WebRTC که به‌صورت داخلی توسعه داده‌اند استفاده می‌کنند یا اجزای مختلف را در یک سرویس یکپارچه ترکیب می‌کنند.

قابل توجه است که این الزامات امنیتی شامل توسعه‌دهندگان نمی‌شود که صرفاً از SDK و API ارائه‌شده توسط ارائه‌دهندگان CPaaS استفاده می‌کنند. برای چنین توسعه‌دهگانی، مسئولیت بی‌شترمسائل امنیتی زیرساختی معمولاً برعهده ارائه‌دهندگان CPaaS است و استناد امنیتی عمومی مانند ASVS ممکن است به‌طور کامل نی‌آزهای آنها را پوشش ندهد.

Server TURN V17.1

این بخش الزامات امنیتی را برای سیستم‌هایی که سرورهای around Relays Using (Traversal TURN NAT) خود را راه‌اندازی می‌کنند، تعریف می‌کند. سرورهای TURN در هدایت Media در محیط‌های شبکه-ای محدود کم می‌کنند، اما در صورت پی‌کربندی نادرست می‌توانند خطراتی ایجاد کنند. این کنترل‌ها بر فیلتر کردن امن آدرس‌ها و جلوگیری از تمام شدن منابع سیستمی (resource exhaustion) تمرکز دارند.

#	Description	Level
17.1.1	سرویس NAT around Relays Using Traversal (TURN) بایدها به آدرسهای IP دسترسی دهد که برای اهداف ویژه رزرو نشده اند (مثلاً شبکه‌های داخلی، broadcast، loopback). داشته باشد که این مورد هم برای آدرسهای IPv4 و هم برای IPv6 صدق می‌کند.	2
17.1.2	سرویس NAT around Relays Using Traversal (TURN) نبایدها به resource exhaustion آسیبی بپذیر باشد، حتی زمانی که کاربران قانونی تلاش کنند تعداد زیادی پورت روی سرور TURN باز کنند.	3

Media V17.2

این الزامات فقط به سیستم‌های اعمال می‌شود که سرورهای رسانه‌ای WebRTC خود را می‌زبانی می‌کنند، مانند Recording (MCU)، Units Control Multipoint (SFU)، Units Forwarding Selective Gateway. سرورهای رسانه‌ای (Media) وظیفه پردازش و توزیع جریانه‌های رسانه‌ای را برعهده دارند و امنیت آنها برای محافظت از ارتباطات بین هم‌تاهای است. حفاظت از جریانه‌های رسانه‌ای در برنامه‌های WebRTC از اهمیت بالایی برخوردار است تا از شنود، دستکاری و حملات محروم‌سازی از سرویس (denial-of-service) جلوگیری شود که می‌تواند حریص‌خصوصی کاربران و کیفی ارتباط را به خطر بیندازد.

به‌ویژه، لازم است تدابیری برای محافظت در برابر حملات flood مانند rate limiting، ساعت‌های هم‌گام برای تطبیق بازه‌های زمانی واقعی و مدیریت بفرها به‌منظور جلوگیری از overflow و حفظ زمان‌بندی مناسب، اعمال شود. اگر بسته‌های مربوط به یک جلسه رسانه‌ای خاص خیلی سریع برسند، بسته‌های اضافی باید حذف شوند. همچنین محافظت از سیستم در برابر بسته‌های خراب یا مخرب اهمیت دارد که با اعمال اعتبارسنجی ورودی، مدیریت ایمن سرریز عدد صحیح، جلوگیری از overflows buffer و استفاده از سایر تکنیک‌های مقاوم در برابر خطا، حاصل می‌شود.

سیستم‌هایی که صرفاً بر ارتباط رسانه‌ای هم‌تابه‌ها بین مرورگرهای وب متکی هستند و سرورهای میانی رسانه‌ای در آنها دخالتی ندارند، از این الزامات امنیتی خاص مرتبط با رسانه مستثنی هستند.

این بخش به استفاده از Security Layer Transport Datagram (DTLS) در زمینه WebRTC اشاره دارد. الزامی مربوط به داشتن سیستم‌های امنیتی برای مدیریت کلیدهای رمزنگاری در فصل «Cryptography» قابل یافت است. اطلاعات مربوط به روش‌های رمزنگاری تأیید شده را می‌توان در پیوست Cryptography استاندارد ASVS یا در اسنادی مانند 2 Rev. 800-52 SP NIST (نسخه 2025-01) پیدا کرد.

#	Description	Level
17.2.1	کلید مربوط به گواهی (DTLS) Security Layer Transport Datagram بر اساس سیستم مستند مدیریتی کلیدهای رمزنگاری مدیریتی و محافظت می شود.	2
17.2.2	سرور Media طوری پی کربندی شده باشد که از مجموعه های رمزنگاری تایید شده (DTLS) Security Layer Transport Datagram استفاده کند و از یک پروفایل محافظت امن برای گسترش DTLS به منظور ایجاد کلیدها برای پروتکل (DTLS-SRTP) Protocol Transport Real-time Secure پشتیبانی نماید.	2
17.2.3	احراز هویت (SRTP) Protocol Transport Real-time Secure در سرور Media بررسی شود تا از حملات تزریق (RTP) Protocol Transport Real-time چلوگری شده و این حملات باعث بروز شرایط Service of Denial یا وارد شدن محتوای صوتی یا تصویری ناخواسته به جریانه های رسانه ای نشوند.	2
17.2.4	سرور Media باید قادر باشد پردازش ترافیکی رسانه های ورودی را حتی هنگام مواجهه با بسته های (SRTP) Protocol Transport Real-time Secure ناقص یا خراب ادامه دهد.	2
17.2.5	سرور Media باید قادر باشد پردازش ترافیکی رسانه های ورودی را حتی در هنگام دریافت سیلی (flood) از بسته های (SRTP) Protocol Transport Real-time Secure از کاربران قانونی ادامه دهد.	3
17.2.6	سرور Media نسبت به آسیب پذیری "ClientHello Race Condition" در Datagram (DTLS) Security Layer Transport نباید آسیب پذیری نباشد، این کار می تواند با بررسی اینکه آیا سرور رسانه به صورت عمومی به عنوان آسیب پذیری شناخته شده است یا با انجام آزمون Race Condition انجام شود.	3
17.2.7	مکانیسم های ضابطه های تصویب مرتب با سرور رسانه در هنگام طوفان بسته های (Flood) از پروتکل انتقال بلادرنگ امن (SRTP) توسط کاربران مجاز، باید قادر به ادامه پردازش ترافیکی رسانه ورودی است.	3
17.2.8	گواهی نامه امنیتی لایه انتقال دیجیتال (DTLS) باید با ویژگی اثر انگشت (Fingerprint) پروتکل توصیف نشسته (SDP) بررسی می شود و در صورت شکست در بررسی، جریانه رسانه قطع می شود تا احوال جریانه رسانه تضمین گردد.	3

Signaling V17.3

این بخش الزامات سیستم های را تعریف می کند که سرورهای سیگنالینگ WebRTC خود را اداره می کنند. سیگنالینگ وظیفه هماهنگی ارتباطات peer-to-peer را دارد و باید در برابر حملاتی که می توانند بر ایجاد یا کنترل جلسات تأثیری بگذارند مقاوم باشد.

برای اطمینان از سیگنالینگ امن، سیستم ها باید ورودی های ناقص یا مخدوش را به صورت امن مدیریتی کنند وحتی تحت بار زیاد نیز در دسترس باقی بمانند.

#	Description	Level
17.3.1	سرور سیگنالینگ بای قادر باشد حتی در چری ان attack، flood پیام‌های سیگنالینگ ورودی مشروع را پردازش کند. این بای با اعمال محدودیت نرخ (rate limiting) در سطح سیگنالینگ محقق شود.	2
17.3.2	سرور سیگنالینگ بای قادر باشد پیام‌های سیگنالینگ مشروع را حتی در مواجهه با پیام‌های سیگنالینگ ناقص یا مخرب که می‌توانند باعث ایجاد شرایط عدم دسترسی (Denial of Service) شوند، پردازش کند. این می‌تواند شامل اعمال اعتبارسنجی ورودی، مدیریت ایمن سرریز عدد صحیح (integer overflow)، جلوگیری از سرریز بافر (buffer overflow) و استفاده از دیگرتکنیک‌های مقاوم مدیریت خطا باشد.	2

References

برای اطلاعات بیشتر، همچنین به منابع زیر مراجعه کنید:

- security at aimed post blog Security's Enable at documented best is DoS ClientHello DTLS WebRTC The developers WebRTC at aimed paper white associated the and professionals
- Applications Real-Time for Protocol Transport A RTP: - 3550 RFC
- (SRTP) Protocol Transport Real-time Secure The - 3711 RFC
- Real- Secure the for Keys Establish to Extension (DTLS) Security Layer Transport Datagram - 5764 RFC
- (SRTP) Protocol Transport time
- Applications Browser-Based for Protocols Real-Time Overview: - 8825 RFC
- WebRTC for Considerations Security - 8826 RFC
- Architecture Security WebRTC - 8827 RFC
- Profiles Protection DTLS-SRTP

Glossary A: Appendix

- **Lifetime Session Maximum Absolute** - که توسط NIST با عنوان «مهلت زمانی کلی (Overall Timeout) نیز شناخته می‌شود، حداکثر مدت زمانی است که یک نشست پس از احراز هویت می‌تواند فعال باقی بماند، صرف نظر از هرگونه تعامل کاربر. این مفهومی که از مؤلفه‌های انقضای نشست (Session Expiration) محسوب می‌شود.
- **Allowlist** - فهرستی از داده‌های اعمالی مجاز؛ برای مثال، فهرستی از کاراکترهای که انجام اعتبارسنجی ورودی (Input Validation) بر اساس آن‌ها مجاز است.
- **token Anti-forgery** - مکانیزمی که در آن یک یا چند توکن همراه با درخواست ارسال شده و توسط سرور برنامه اعتبارسنجی می‌شوند تا اطمینان حاصل شود درخواست از یک نقطه انتهایی (Endpoint) مورد انتظار ارسال شده است.
- **Security Application** - امنیتی در سطح برنامه بر تحلیلی مؤلفه‌هایی تمرکز دارد که لایه کاربردی در مدل مرجع اتصال سیستم‌های باز (OSI) را تشکیل می‌دهند، نه بر اجزای زیربنایی

مانند سیستم عامل یا شبکه‌های متصل.

- **Verification Security Application** - ارزیابی فنی یک برنامه کاربردی بر اساس استانداردها ASVS, OWASP
- **Report Verification Security Application** - گزارشی که نتایج کلی و تحلیلهای پشتیبان تهیه شده توسط ارزیاب (Verifier) را برای یک برنامه کاربردی مشخص مستند می‌کند.
- **Authentication** - فرایند تأیید هویت ادعاشده کاربری یک برنامه کاربردی.
- **Verification Automated** - استفاده از ابزارهای خودکار (شامل ابزارهای تحلیلی پویا، ابزارهای تحلیلی ایستا، یا هر دو) که باتکلیف برامضاهای آسبپذیری برای شناسایی مشکلات به کار می‌روند.
- **testing box Black** - روشی از آزمون نرم افزار که عمل کردی یک برنامه را بررسی می‌کند بدون آنکه به ساختارهای اعمال کرد داخلی آن نگاه کند.
- **(CWE) Enumeration Weakness Common** - فهرستی که توسط جامعه توسعه دهندگان ایجاد شده و شامل نقاط ضعف رایج امنیتی نرم افزار است. این فهرست به عنوان زبان مشترک، معرفی برای ابزارهای امنیتی نرم افزار، و پایهای برای شناسایی، کاهش و پیشگیری از نقاط ضعف عمل می‌کند.
- **Component** - یک واحد مستقل از کد است که دارای رابطهای مرتبط با دیسک و شبکه می‌باشد و با سایر مولفه‌ها ارتباط برقرار می‌کند.
- **(CSP) Provider Service Credential** - که به آن ارائه دهنده هویت (Identity Provider - IdP) نیز گفته می‌شود، منبعی از داده‌های کاربری است که می‌تواند به عنوان منبع احراز هویت توسط سایر برنامه‌ها استفاده شود.
- **(XSSI) Inclusion Script Cross-Site** - گونه‌ای از حمله اسکریپت نویسی بی‌نسایتی (XSS) که در آن یک برنامه وب کد مخرب را از یک منبع خارجی دریافت کرده و آن کد را به عنوان بخشی از محتوای خود وارد می‌کند.
- **(XSS) Scripting Cross-Site** - اسکریپت‌پذیری امنیتی مخمول در برنامه‌های وب که امکان تزریق اسکریپت‌های سمت کاربر (Client-Side Scripts) به محتوا را فراهم می‌کند.
- **module Cryptographic** - سخت افزار، نرم افزار و/یا فریمور که الگوریتم‌های رمزنگاری را اجرا کرده و/یا کلیدهای رمزنگاری تولید می‌کند.
- **generator number pseudo-random secure Cryptographically** (CSPRNG) - یک تولیدکننده اعداد شبه تصادفی با ویژگی‌هایی که آن را مناسب برای استفاده در رمزنگاری می‌کند؛ که گاهی به آن تولیدکننده اعداد تصادفی رمزنگاری (Cryptographic Generator Number Random - CRNG) نیز گفته می‌شود.
- **(DTLS) Security Layer Transport Datagram** - یک پروتکل رمزنگاری که امنیت ارتباطی را بر روی اتصال شبکه فراهم می‌کند. این پروتکل بر اساس TLS ساخته شده اما برای محافظت از پروتکلهای مبتنی بر دیتاگرام (معمولاً روی UDP) تطبیق داده شده است. نسخه DTLS 1.3 در RFC 9147 تعریف شده است.
- **Transport Real-time Secure the for Keys Establish to Extension Security Layer Transport Datagram** (DTLS-SRTP) Protocol - مکانیزمی که از DTLS Handshake برای ایجاد مواد کلیدی (Key Material) جهت یک جلسه SRTP استفاده می‌کند. این افزونه در RFC 5764 تعریف شده است.
- **Verification Design** - ارزیابی فنی معماری امنیتی یک برنامه کاربردی.
- **(DAST) Testing Security Application Dynamic** - فناوری‌هایی که برای شناسایی شرایطی که نشان دهنده آسبپذیری امنیتی در یک برنامه در حال اجرا هستند طراحی شده‌اند.

- **Verification Dynamic** - است فاده از ابزارهای خودکار که با تکیه بر امضاهای آس بی بی پی ذری به شناسایی مشکلات در حین اجرای برنامه می پردازند.
- **(FIDO) Online Identity Fast** - مجموعه ای از استانداردهای احراز هویت که امکان است فاده از روش های مختلف احراز هویت از جمله بی وم تری ک، ماژول های پلت فرم مورد اعت ماد، (TPM) توکن های امنیتی USB و غیره را فراهم می کند.
- **(HSM) Module Security Hardware** - یکی از جزء سخت افزاری که کلیدهای رمزنگاری و سایر اسرار را به صورت محافظت شده ذخیره می کند.
- **(HQL) Language Query Hibernate** - یکی از زبان پرس و جو که از نظر ظاهری شبیه SQL است و توسط کتابخانه ORM Hibernate است فاده می شود.
- **(HSTS) Security Transport Strict HTTP** - سیاستی که مرورگر را مجبور می کند فقط از طریق TLS به دامنه ای که هدر HSTS را ارسال می کند متصل شود و تنه زمانی که گواهی نامه معتبر ارائه شده باشد. این سیاست با فی لد هدر پاسخ Strict-Transport-Security فعال می شود.
- **(HTTP) Protocol Transfer HyperText** - یکی از پروتکل برنامه های برای سیستم های اطلاعاتی توزیع شده، مشارکتی و ابررسانه ای است. HTTP پایه و اساس ارتباط داده ها در شبکه جهانی وب محسوب می شود.
- **(HTTPS) SSL/TLS over Protocol Transfer HyperText** - روشی برای امن سازی ارتباط HTTP از طریق رمزنگاری آن با است فاده از (Transport TLS Layer (Security).
- **(IdP) Provider Identity** - که در منابع NIST گاهی به آن ارائه دهنده خدمات اعتبارنامه (Credential Provider Service - CSP) نیز گفته می شود، موجدیتی است که منبع احراز هویت را برای سایر برنامه ها فراهم می کند.
- **Timeout Inactivity** - مدتی است که یکی از نشانه های می تواند در صورت عدم تعامل کاربر با برنامه فعال باقی بماند. این معیار یکی از مؤلفه های انقضای نشانه (Session Expiration) است.
- **Validation Input** - استاندارد سازی (Canonicalization) و اعتبار سنجی ورودی های کاربری رقابل اعتبار داد.
- **(JWT) Token Web JSON** - RFC 7519 یکی از استانداردهای برای یک شیء داده JSON تعریف می کند که از یک بخش هدر مشخصی تشکیل شده که نحوه اعتبار سنجی شیء را توضیح می دهد، یک بخش بدنه که شامل مجموعه ای از ادعاها است، و یک بخش امضا که شامل امضای دیجیتالی است و می تواند از آن برای اعتبار سنجی محتوای بخش بدنه است فاده کرد. این نوعی توکن خود کفا (self-contained token) است.
- **(LFI) Inclusion File Local** - حمله ای که از رویه های آس بی بی پی ذری درج فایل در یکی از برنامه ها سوءاستفاده می کند و منجر به درج فایل های محلی موجود روی سرور می شود.
- **Code Malicious** - کدی که در طول توسعه یکی از برنامه ها بدون اطلاع مالک برنامه وارد شده و سیستم امنیتی مورد نظر برنامه را دور می زند. این با بدافزارهای مانند ویروس یا کرم های کسان نیست!
- **Malware** - کد اجرایی که در حین اجرای برنامه و بدون اطلاع کاربر یا مدیر برنامه وارد می شود.
- **(MAC) code authentication Message** - یکی از چک سام رمزنگاری شده بر داده ها است که توسط الگوریتم تولیدی MAC محاسبه می شود و برای اطمینان از صحت و اصالت داده ها است فاده می شود.
- **(MFA) authentication Multi-factor** - احراز هویتی که شامل دو یا چند عامل از عوامل احراز هویت تکی است.

- **Authentication). Client (TLS TLS Mutual (mTLS) -** مراجعه کنید به احراز هویت کلاینت (Authentication). Client (TLS TLS Mutual (mTLS) -
- **ORM) Mapping Object-relational -** سیستمی که امکان ارجاع و پرسوجوی پایگاه داده‌ای رابطه‌ای/جدولی را در یک برنامه کاربردی با استفاده از مدل شیءمحور سازگار با برنامه فراهم می‌کند.
- **OTP) Password One-time -** رمزی که به صورت منحصر به فرد تولید شده و فقط برای یک بار استفاده کاربرد دارد.
- **OWASP) Project Security Application Worldwide Open -** یک جامعه جهانی، رایگان و باز است که بر به‌بود امنیت نرم‌افزارهای کاربردی تمرکز دارد. مأموریت آن این است که امنیت برنامه‌های کاربردی را «قابل مشاهده» کند تا افراد و سازمان‌ها بتوانند تصمیمات آگاهانه‌ای در مورد ریسک‌های امنیتی برنامه‌های کاربردی بگیرند. برای اطلاعات بیشتر مشاهده شود: <https://www.owasp.org/>
- **2 Function Derivation Key Password-Based (PBKDF2) -** یک الگوریتم یک طرفه ویژه است که برای ایجاد یک کلید رمزنگاری قوی از یک متن ورودی (مانند گزافه) و یک مقدار salt تصادفی اضافی استفاده می‌شود و بنابراین می‌توان از آن برای سخت‌تر کردن شکستن گزافه به صورت آفلاین استفاده کرد، اگر مقدار اصل به جای گزافه اصلی ذخیره شود.
- **PKI) Infrastructure Key Public -** ساختاری که کلیدهای عمومی را با هویت‌های مربوط به موجودیت‌ها پیوند می‌دهد. این پیوند از طریق فرآیند ثبت و صدور گواهی‌نامه توسط یک مرجع صدور گواهی (Certificate Authority - CA) برقرار می‌شود.
- **(PSTN) Network Telephone Switched Public -** شبکه تلفن سنتی که شامل تلفن‌های خط ثابت و تلفن‌های همراه می‌شود.
- **(RTCP) Protocol Control Transport Real-time and (RTP) Protocol Transport Real-time -** دو پروتکل که به صورت ترکیبی برای انتقال جریان‌های چندرسانه‌ای استفاده می‌شوند. این پروتکل‌ها توسط پشته WebRTC استفاده می‌شوند و در RFC 3550 تعریف شده‌اند.
- **Token Reference -** نوعی توکن که به عنوان اشاره‌گری شناسه‌ای برای وضعیت یا متاداده ذخیره شده روی سرور عمل می‌کند و گاهی به آن توکن‌های تصادفی (Random Tokens) یا توکن‌های مات (Opaque Tokens) نیز گفته می‌شود. برخلاف توکن‌های خودکفا (Self-Contained Tokens) که بخشی از داده‌های مرتبط با خود را درون توکن جای می‌دهند، توکن‌های مرجع هیچ اطلاعات ذاتی‌ای ندارند و برای دریافت زمینه (Context) به سرور وابسته هستند. توکن مرجع‌های یک شناسه نشن (Session Identifier) است یا شامل آن می‌باشد.
- **(RP) Party Relying -** معمولاً یک برنامه کاربردی است که بر احراز هویت کاربر توسط یک ارائه‌دهنده احراز هویت چیدانه متکی است. این برنامه بر نوعی توکن یا مجموعه‌ای از ادعاهای امضا شده ارائه شده توسط آن ارائه‌دهنده اعتماد می‌کند تا مطمئن شود کاربر همان کسی است که ادعا می‌کند.
- **(RFI) Inclusion File Remote -** حمله‌ای که از رویه‌های آسی‌ب‌پذیر درج فایل در برنامه سوءاستفاده می‌کند و منجر به درج فایل‌های راه‌دور (Remote) می‌شود.
- **(SVG) Graphics Vector Scalable -** یک زبان نشانه‌گذاری مبتنی بر XML برای توصیف گرافیک‌های برداری دوبعدی.
- **Protocol Control Transport Real-time Secure and (SRTP) Protocol Transport Real-time Secure -** (SRTCP) - یک پروتکل از پروتکل‌های RTP و RTCP که پشتیبانی از رمزنگاری پیام، احراز هویت و محافظت از صحت داده‌ها را فراهم می‌کند. این پروتکل‌ها در RFC 3711 تعریف شده‌اند.

- **Architecture Security** - یک انتزاع از طراحی برنامه کاربردی است که محل و نحوه استفاده از کنترل‌های امنیتی را شناسایی و توصیف می‌کند و همچنین مکان و حساسیت داده‌های کاربر و برنامه را مشخص می‌نماید.
- **Sign-On Single (SAML) Language Markup Assertion Security** - یک استاندارد باز برای احراز هویت Sign-On Single است که بر اساس انتقال ادعاهای امضا شده (معمولاً اشیاء XML بین ارائه‌دهنده هویت (Identity Provider) و Party Relying عمل می‌کند).
- **Configuration Security** - پی‌کربندی زمان اجرا یک برنامه که نحوه استفاده از کنترل‌های امنیتی را تحت تأثیری قرار می‌دهد.
- **Control Security** - یک عمل‌کرد یا مؤلفه که یک بررسی امنیتی انجام می‌دهد (مثلاً بررسی دسترسی) یا هنگام فراخوانی منجر به اثر امنیتی می‌شود (مثلاً تولید یک رکورد حساس‌بندی).
- **(SIEM) management event and information Security** - سیستمی برای شناسایی تهدیدات، انطباق با مقررات و مدیریتی رخ داده‌های امنیتی از طریق جمع‌آوری و تحلیل داده‌های مرتبط با امنیتی از منابع مختلف در زیرساخت فناوری اطلاعات یک سازمان.
- **Token Self-Contained** - توکنی که یک یا چند ویژگی را در خود جای می‌دهد و به وضعیت سمت سرور یا سایر ذخیره‌سازی‌های خارجی متکی نیست. این توکن‌ها اصالت و صحت ویژگی‌های درون خود را تضمین می‌کنند و امکان تبادل امن و بدون حالت (Stateless) اطلاعات بین سیستم‌ها را فراهم می‌سازند. توکن‌های خودکفا معمولاً با توکن‌های رمزنگاری مانند امضاهای دیجیتالی یا کدهای تأیید پیام (MAC) ایمن می‌شوند تا اصالت، صحت و در برخی موارد محرمانگی داده‌ها تضمین گردد. نمونه‌های رایج شامل ادعاهای (SAML SAML Assertions) و JWT هستند.
- **(SSRF) Forgery Request Server-side** - حمله‌ای که از قابلیت‌های سرور برای خواندن یا به‌روزرسانی منابع داخلی سوءاستفاده می‌کند. مهاجم یک URL را ارائه یا تغیی می‌دهد که روی سرور اجرا می‌شود آن را خوانده یا داده‌ای به آن ارسال می‌کند.
- **(SDP) Protocol Description Session** - یک قالب پیام برای راه‌اندازی نشست‌های چندرسانه‌ای (به‌عنوان مثال در WebRTC استفاده می‌شود). این پروتکل در RFC 4566 تعریف شده است.
- **ID Session or Identifier Session** - یک کلیدی که نشست stateful ذخیره‌شده در سمت سرور را شناسایی می‌کند. این شناسه به‌صورت مستقیم یا درون یک «توکن مرجع» (Reference Token) بین سرور و کلاینت منتقل می‌شود.
- **Token Session** - اصطلاحی کلیدی که در این استاندارد برای اشاره به توکن یا مقدار مورد استفاده در مکان‌های نشست stateless (که از توکن استفاده‌شده self-contained می‌کنند) یا مکان‌های نشست stateful (که از توکن مرجع استفاده می‌کنند) به کار می‌رود.
- **(STUN) NAT for Utilities Traversal Session** - یک پروتکل که برای کمک به عبور از NAT به منظور برقراری ارتباط‌های هم‌تا هم‌تا (Peer-to-Peer) استفاده می‌شود. این پروتکل در RFC 3489 تعریف شده است.
- **authenticator Single-factor** - مکان‌یابی برای بررسی احراز هویت کاربر که بایستی یکی از موارد زیر باشد: چیزی که می‌داند (رمزهای عبور، عبارات عبور، PIN، چیزی که هست (بی‌ومتری، اثر انگشت، اسکن چهره) یا چیزی که دارد (توکن‌های OTP، دست‌گاه رمزنگاری مانند کارت هوشمند).
- **(SSO) Authentication Sign-on Single** - زمانی رخ می‌دهد که کاربر وارد یک برنامه می‌شود و به-

- صورت خودکار به برنامه‌های دی‌گر نی‌ز وارد می‌شود بدون نی‌از به احراز هویت مجدد. به عنوان مثال، وقتی کاربر وارد Google می‌شود، به طور خودکار وارد سایر سرویس‌های Google مانند Gmail و Docs و YouTube می‌گردد.
- **materials of bill Software (SBOM)** - یکی که فهرست ساختاریافته و جامع از تمام مؤلفه‌ها، ماژول‌ها، کتابخانه‌ها، فریم‌ورک‌ها و سایر منابع مورد نی‌از برای ساخت یا مونتاژی یک برنامه نرم‌افزاری است.
 - **Analysis Composition Software (SCA)** - مجموعه‌ای از فناوری‌ها که برای تحلیل ترکیب برنامه، وابستگی‌ها، کتابخانه‌ها و بسته‌ها به منظور شناسایی آسیب‌پذیری‌های امنیتی نسخه‌های خاص مؤلفه‌های مورد استفاده طراحی شده است. این مفهوم با تحلیل کد منبع (SAST) اشتباه گرفته نشود.
 - **lifecycle development Software (SDLC)** - فرایند گام به گام توسعه نرم‌افزار که از نی‌ازمندی‌های اولیه تا استقرار و نگهداری پیش می‌رود.
 - **SQL Injection (SQLi)** - یک تکنیک تزریق کد که برای حمله به برنامه‌های مبتنی بر داده استفاده می‌شود و در آن دستورات SQL مخرب به یک نقطه ورودی تزریق می‌گردند.
 - **Mechanism Session Stateful** - در این مکانیزم، برنامه وضعیت نش‌ست را در سمت سرور نگه می‌دارد که معمولاً با توکن نش‌ست مطابقت دارد و این توکن با استفاده از تولیدی‌کننده اعداد شبه تصادفی امن رمزنگاری (CSPRNG) ایجاد و به کاربر نه‌ای ارائه می‌شود.
 - **Mechanism Session Stateless** - یک مکانیزم نش‌ست stateless از یک token self-contained استفاده می‌کند که به کلاینت‌ها منتقل می‌شود و شامل اطلاعات نش‌ست است که لزوماً در سرویس دریافت‌کننده ذخیره نمی‌شود و سپس آن سرویس توکن را اعتبارسنجی می‌کند. در عمل، یک سرویس نی‌از دارد به دسترسی به برخی اطلاعات نش‌ست (مانند فهرست لغو JWT) تا بتواند کنترلهای امنیتی مورد نی‌از را اعمال کند.
 - **Static testing security application (SAST)** - مجموعه‌ای از فناوری‌ها که برای تحلیل کد منبع، بایت‌کد و باینری‌های برنامه به منظور شناسایی شرایط برنامه‌نویسی و طراحی که نش‌ان‌دهنده آسیب‌پذیری‌های امنیتی هستند طراحی شده‌اند. راه‌کارهای SAST برنامه را از داخل به خارج و در حالت غیری‌اجرای تحلیل می‌کنند.
 - **Modeling Threat** - یک تکنیک که شامل توسعه معماری‌های امنیتی به طور پیوسته و دقیقت‌ر برای شناسایی عمل‌انته‌دهی، مناطق امنیتی، کنترلهای امنیتی و دارایی‌های فنی و تجاری مهم است.
 - **time-of-use to Time-of-check (TOCTOU)** - وضعیتی که در آن یک برنامه وضعیتی یک منبع را قبل از استفاده از آن بررسی می‌کند، اما وضعیتی منبع می‌تواند بین زمان بررسی و استفاده تغییری کند. این می‌تواند نتایج بررسی را نامعتبر کند و باعث شود برنامه به دلیل عدم تطابق وضعیتی اقدامات نامعتبر انجام دهد.
 - **One-time based Time Passwords (TOTPs)** - روشی برای تولیدی OTP که در آن زمان فعلی بخشی از الگوریتم تولیدی رمز عبور محسوب می‌شود.
 - **authentication client TLS, TLS Mutual called also (mTLS)** - در یک اتصال استاندارد TLS، کلاینت می‌تواند از گواهی ارائه‌شده توسط سرور برای اعتبارسنجی هویت سرور استفاده کند. زمانی که احراز هویت کلاینت TLS استفاده می‌شود، کلاینت نی‌ز از کلیدی خصوصی و گواهی خود استفاده می‌کند تا سرور بتواند هویت کلاینت را اعتبارسنجی کند.
 - **Security Layer Transport (TLS)** - پروتکلهای رمزنگاری که امنیتی ارتباطات را بر روی اتصال شب‌که فراهم می‌کنند.

- **TURN** به عنوان رله استفاده می کند زمانی که اتصال مستقیم هم تا به هم تا قابل برقراری نیست. این پروتکل در RFC 8656 تعریف شده است.
- **Trusted environment execution (TEE)** - یک محیط پردازشی ایزوله که در آن برنامه ها می توانند به طور امن اجرا شوند، بدون توجه به باقی سیستم.
- **Trusted Module Platform (TPM)** - نوعی HSM که معمولاً به یک جزء سخت افزاری بزرگتر مانند مادربرد متصل است و به عنوان ریشه اعتماد (Root of Trust) برای آن سیستم عمل می کند.
- **Trusted Layer Service** - هر نقطه اجرای کنترلر مورد اعتماد، مانند یک میکروروس روی س، API بدون سرور (Serverless)، API سمت سرور، یک API مورد اعتماد روی دستگاه کلاینت که بوت امن دارد، های API شرکت های خارجی و غیره. مورد اعتماد به این معناست که نگارانی وجود ندارد که یک کاربر غیرمورد اعتماد بتواند لایه های کنترلرهای پیاده سازی شده در آن لایه را دور بزند یا نادیده بگذرد.
- **Uniform Resource Identifier (URI)** - یک رشته ای که یک کتا از کاراکترها که یک منبع را شناسایی می کند، مانند یک صفحه وب، آدرس ایمیل یا مکانها.
- **Uniform Resource Locator (URL)** - رشته ای که مکان یک منبع در اینترنت را مشخص می کند.
- **Universally Unique Identifier (UUID)** - یک شماره مرجع یکتا که به عنوان شناسه در نرم افزار استفاده می شود.
- **Verifier** - فرد یا تیمی که یک برنامه را بر اساس الزامات ASVS OWASP بررسی می کند.
- **Web Real-Time Communication (WebRTC)** - یک پشت پروتکل و API وب مرتبط که برای انتقال چرینه ای چندرسانه ای در برنامه های وب استفاده می شود، معمولاً در زمینه کنفرانس های وی دی وی. این پشت بر اساس SDP DTLS، SRTCP، SRTP، STUN/TURN ساخته شده است.
- **TLS over WebSocket (WSS)** - روشی برای امن سازی ارتباط وبس وکت از طریق قرار دادن وبس وکت بر روی پروتکل TLS.
- **Get You What Is See You What (WYSIWYG)** - نوعی وی رای شگر محت وای غنی که نشان می دهد محت واهنگام نمایش چگونگی به نظر می رسد، به جای آنکه کدهای که برای نمایش محت واه استفاده می شوند را نشان دهد.
- **Certificate X.509** - یک گواهی دیجیتالی است که از استاندارد بین المللی پذیرفته شده X.509 در زیر ساخت کلی عمومی (PKI) استفاده می کند تا تأیید کند کلی عمومی متعلق به هویت کاربر، کامپیوتر یا سرویس است که در گواهی ذکر شده است.
- **XML Entity eXternal (XXE)** - نوعی موجودی XML که می تواند از طریق شناسه سیستم اعلام شده به محت وای محلی یا راه دور دسترسی پیدا کند. این می تواند منجر به انواع حملات تزریقی شود.

References B: Appendix

پروژه های زیر از OWASP به احتمال زیاد برای کاربران و استفاده کنندگان این استاندارد مفید خواهند بود:

Projects Core OWASP

- <https://owasp.org/www-project-top-ten/> Project: 10 Top OWASP 1.
- <https://owasp.org/www-project-web-security-testing-guide/> Guide: Testing Security Web OWASP 2.
- <https://owasp.org/www-project-proactive-controls/> Controls: Proactive OWASP 3.
- <https://owasp.org/www-project-samm/> (SAMM): Model Maturity Assurance Software OWASP 4.
- <https://owasp.org/www-project-secure-headers/> Project: Headers Secure OWASP 5.

project Series Sheet Cheat OWASP

این پروژه دارای چندین برگه راهنما (Cheat Sheet) می باشد که برای موضوعات مختلف در ASVS مرتبط و کاربردی خواهند بود.

نگاشتی به ASVS وجود دارد که می توان آن را در اینجا پیدا کرد: <https://cheatsheetseries.owasp.org/Ind-exASVS.html>

Projects Related Security Mobile

- <https://owasp.org/www-project-mobile-security/> Project: Security Mobile OWASP 1.
- <https://owasp.org/www-project-mobile-top-10/> Risks: 10 Top Mobile OWASP 2.
- <https://owasp.org/www-project-mobile-security-testing-guide/> Standard: Verification Security Application Mobile and Guide Testing Security Mobile OWASP 3.

projects related Things of Internet OWASP

- <https://owasp.org/www-project-internet-of-things/> Project: Things of Internet OWASP 1.

projects Serverless OWASP

- <https://owasp.org/www-project-serverless-top-10/> Project: Serverless OWASP 1.

Others

به طور مشابه، وبسایتهای زیر به احاطه زیاد برای کاربران و استفاده کنندگان این استاندارد مفید خواهند بود.

- <https://github.com/danielmiessler/SecLists> Github: SecLists 1.
- <https://cwe.mitre.org/> Enumeration: Weakness Common MITRE 2.
- <https://www.pcisecuritystandards.org/standards/> Standards: Security PCI 3.

Standards Cryptography C: Appendix

فصل «رمزنگاری» فراتر از آرائه‌ی ساده‌ی به‌ترین رویه‌ها عمل می‌کند. این فصل با هدف افزایش درک اصول رمزنگاری و تشویق به به‌کارگیری روش‌های امنیتی مدرن‌تر و مقاوم‌تر تدوین شده است. این پیوست، اطلاعات فنی تفصیلی مربوط به هر یک از الزامات را آرائه می‌دهد و استانداردهای کلی مطرح شده در فصل «رمزنگاری» را تکمیل می‌کند.

این پیوست سطح تأیید را برای مکانیسم‌های مختلف رمزنگاری تعریف می‌کند:

- مکانیسم‌های تأیید شده (A) می‌توانند در برنامه‌ها استفاده شوند.
- مکانیسم‌های قدیمی (L) نباید در برنامه‌ها استفاده شوند، اما ممکن است فقط برای سازگاری با برنامه‌های کدهای قدیمی موجود به کار روند. اگرچه استفاده از این مکانیسم‌ها در حال حاضر به خودی خود آسیبی به پذیرش محسوب نمی‌شود، اما باید هرچه سریع‌تر با مکانیسم‌های امن‌تر و آسان‌تر جایگزین شوند.
- مکانیسم‌های غیرمجاز (D) نباید استفاده شوند زیرا در حال حاضر شکسته محسوب می‌شوند یا امنیت کافی آرائه نمی‌کنند.

این فهرست ممکن است در زمینه‌ی یک برنامه خاص به دلایل مختلف لغو شود، از جمله:

- تحولات جدید در حوزه رمزنگاری؛
- رعایت مقررات و قوانین.

Documentation and Inventory Cryptographic

این بخش اطلاعات تکمیلی را برای V11.1 موجودی و مستندسازی رمزنگاری آرائه می‌دهد.

مهم است که تمام دارایی‌های رمزنگاری، مانند الگوریتم‌ها، کلیدها و گواهی‌نامه‌ها، به‌طور منظم شناسایی، فهرست‌بندی و ارزیابی شوند. برای سطح 3، این باید شامل استفاده از اسکن‌ایست‌ا و پویا برای شناسایی استفاده از رمزنگاری در یک برنامه باشد. ابزارهای SAST و DAST می‌توانند در این زمینه کمک کنند، اما ممکن است ابزارهای اختصاصی برای پوشش جامع‌تر مورد نیاز باشند. نمونه‌های رایگان از این ابزارها عبارتند از:

- python in written eBPF, using - Monitor Cryptography Network - CryptoMon
- Outputs CodeQL from CBOMs Comprehensive Generating Tool: Forge Cryptobom

Parameters Cryptographic of Strengths Equivalent

قدرت‌های امنیتی نسبی برای سیستم‌های مختلف رمزنگاری در این جدول آمده است (از SP NIST 800-57 Part 1, ص 71):

Strength Security	Key Symmetric Algorithms	Field Finite	Integer Factorization	Curve Elliptic
80 <=	2TDEA	160 = N 1024 = L	1024 = k	160-223 = f

Strength Security	Key Symmetric		Integer	
	Algorithms	Field Finite	Factorization	Curve Elliptic
112	3TDEA	224 = N 2048 = L	2048 = k	224-255 = f
128	AES-128	256 = N 3072 = L	3072 = k	256-383 = f
192	AES-192	384 = N 7680 = L	7680 = k	384-511 = f
256	AES-256	512 = N 15360 = L	15360 = k	512+ = f

نمونه‌ای از برنام‌ها:

- رمزنگاری می‌دان محدود (Finite Field): MQV FFDH، DSA، Cryptography)
- رمزنگاری تجزیه عدد صحیح (Integer Factorization): RSA Cryptography)
- رمزنگاری منحنی بیضوی (Elliptic Curve): MQV ECDH، EdDSA، ECDSA، Cryptography)

توجه: این بخش فرض می‌کند که هیچ کامپیوتری که وانتومی وجود ندارد؛ اگرچه این کامپیوتری وجود داشته باشد، برآورده‌ای سه ستون آخر دی‌گرم‌ت‌بر نخواهند بود.

Values Random

این بخش اطلاعات تک‌میلی برای 5 Values Random را ارائه می‌دهد.

Name	Version/Reference	Notes	Status
/dev/random	also ,2016) (Oct 4.8+ Linux and Android, iOS, in found POSIX Linux-based other on Based systems. operating RFC7539	stream. ChaCha20 Utilizing iOS in Found and SecRandomCopyBytes Random Secure Android settings correct the with each. to provided	A
/dev/urandom	for file special kernel's Linux data random providing	entropy high-quality, Provides hardware from sources randomness	A
AES-CTR-DRBG	SP800-90A NIST	common in used As as such implementations, API CNG Windows by set BCryptGenRandom .BCRYPT_RNG_ALGORITHM	A
HMAC-DRBG	SP800-90A NIST		A
Hash-DRBG	SP800-90A NIST		A

Name	Version/Reference	Notes	Status
getentropy()	Linux in available ,OpenBSD macOS and 2.25+ glibc 10.12+	bytes random secure Provides kernel's the from directly a with source entropy minimal and straightforward and modern more It's API. with associated pitfalls avoids APIs. older	A

تابع هش زیربنای که در HMAC-DRBG یا Hash-DRBG استفاده می‌شود، باید برای این کاربرد تأیید شده باشد.

Algorithms Cipher

این بخش اطلاعات تکمیلی برای الگوریتم‌های رمزنگاری را ارائه می‌دهد.
الگوریتم‌های رمزنگاری تأیید شده به ترتیب اولویت فهرست شده‌اند.

Algorithms Key Symmetric	Reference	Status
AES-256	197 FIPS	A
Salsa20	specification 20 Salsa	A
XChaCha20	Draft XChaCha20	A
XSalsa20	nonce Salsa20 the Extending	A
ChaCha20	8439 RFC	A
AES-192	197 FIPS	A
AES-128	197 FIPS	L
2TDEA		D
(3DES/3DEA) TDEA		D
IDEA		D
RC4		D
Blowfish		D
ARC4		D
DES		D

Modes Cipher AES

رمزنگاری‌های بلوکی، مانند AES، می‌توانند با حالت‌های مختلف عملیاتی استفاده شوند. بسیاری از حالت‌های عملیاتی، مانند Codebook Electronic (ECB)، Message Chaining Block Cipher with Counter و (GCM) Mode Galois/Counter و (CCM) Code Authentication (Authenticated Encryption) ارائه می‌دهند و باید در برنامه‌های مدرن استفاده شوند.

حالت‌های تأیید شده به ترتیب اولویت فهرست شده‌اند.

Mode	Authenticated	Reference	Status	Restriction
GCM	Yes	SP NIST 800-38D	A	
CCM	Yes	SP NIST 800-38C	A	
CBC	No	SP NIST 800-38A	L	
CCM-8	Yes		D	
ECB	No		D	
CFB	No		D	
OFB	No		D	
CTR	No		D	

یادداشت‌ها:

- تمام پیام‌های رمزنگاری شده باید احراز هویت شوند. برای هر استفاده‌ای از حالت CBC، حتماً باید یک الگوریتم هش MAC مرتباً برای اعتبارسنجی پیام وجود داشته باشد. به طور کلی، این باید با روش Encrypt-Then-Hash اعمال شود (اما TLS 1.2 به جای آن از روش Hash-Then-Encrypt استفاده می‌کند). اگر این تضمین امکان‌پذیر نباشد، نباید از CBC استفاده شود. تنها کاربردی که در آن رمزنگاری بدون الگوریتم MAC مجاز است، رمزنگاری دیسک است.
- اگر از حالت CBC استفاده شود، باید تضمین شود که بررسی پدینگ (Padding) در زمان ثابت انجام می‌شود.
- هنگام استفاده از CCM-8، برجسب MAC تنها ۶۴ بیت امنیتی ارائه می‌دهد. این با الزامات بند ۶.۲.۹ که حداقل ۱۲۸ بیت امنیتی را نیاز دارد، مطابقت ندارد.
- رمزنگاری دیسک خارج از محدوده ASVS در نظر گرفته شده است. بنابراین، این تضمین‌ها هیچ روش تأیید شده‌ای برای رمزنگاری دیسک فهرست نمی‌کند. برای این کاربرد، معمولاً رمزنگاری بدون احراز هویت پذیرفته می‌شود و حالت‌های XTS، XEX و LRW معمولاً استفاده می‌شوند.

Wrapping Key

بسته‌بندی کلید رمزنگاری (و باز کردن بسته‌ی مربوطه) روشی برای محافظت از کلید کلید موجود است که با محصور کردن (یعنی بسته‌بندی) آن و استفاده از کلید کلیدی رمزنگاری اضافی انجام می‌شود تا کلید اصلی به‌طور آشکار در معرض دید قرار نگیرد، مثلاً هنگام انتقال. کلید اضافی که برای محافظت از کلید اصلی استفاده می‌شود، کلید بسته‌بندی (wrap key) نامیده می‌شود.

این عملیات ممکن است زمانی انجام شود که بخواهیم کلیده‌ها را در مکان‌های که قابل اعتماد نیستند محافظت کنیم، یا کلیده‌ای حساس را از طریق شبکه‌ای غیرقابل اعتماد یا درون برنامه‌ها ارسال کنیم. با این حال، باید با دقت به ماهیت کلید اصلی (مانند هویت و هدف آن) قبل از انجام فرآیند بسته‌بندی/باز کردن بسته توجه شود، زیرا این می‌تواند برای سیستم‌ها یا برنامه‌های مبدأ و مقصد پیامدهایی در زمینه امنیت و به‌ویژه تطابق قانونی داشته باشد، که ممکن است شامل مسیری‌های حساس بررسی عمل کرد کلید (مانند امضا) و همچنین ذخیره‌سازی مناسب کلید باشد.

به‌طور مشخص، برای بسته‌بندی کلید باید از AES-256 استفاده شود، مطابق با SP NIST 800-38F و با در نظر گرفتن تهمیدات آینده‌نگر در برابر تهدید کوانتومی. حالت‌های رمزنگاری استفاده‌شده با AES به ترتیب اولویت به شرح زیر هستند:

Wrapping Key	Reference	Status
KW	800-38F SP NIST	A
KWP	800-38F SP NIST	A

در صورتی که مورد استفاده‌ی آن داشته باشد، می‌توان از AES-128 و AES-192 نیز استفاده کرد، اما دلیلی استفاده از آن‌ها باید در فهرست رمزنگاری موجود در سازمان مستندسازی شود.

Encryption Authenticated

به استثنای رمزنگاری دیسک، داده‌ای رمزنگاری‌شده باید در برابر تغیری‌رات غیرمجاز با استفاده از نوعی از طرح رمزنگاری احراز هویت‌شده (AE) محافظت شوند، که معمولاً از طرح رمزنگاری احراز هویت‌شده با داده‌ای مرتبط (AEAD) استفاده می‌شود.

به‌تر است برنامه‌ای از یک طرح AEAD تأیید شده استفاده کند. به‌طور جایگزین، می‌تواند یک طرح رمزنگاری تأیید شده و یک الگوریتم MAC تأیید شده را با ساختار Encrypt-then-MAC ترکیب کند.

روش MAC-then-encrypt همچنان برای سازگاری با برنامه‌های قدیمی مجاز است. این روش در TLS نسخه ۱.۲ با مجموعه‌ای رمزنگاری قدیمی استفاده می‌شود.

mechanism AEAD	Reference	Status
AES-GCM	800-38D SP	A
AES-CCM	800-38C SP	A
ChaCha-Poly1305	7539 RFC	A
AEGIS-256	(v1.1) Algorithm Encryption Authenticated Fast A AEGIS:	A
AEGIS-128	(v1.1) Algorithm Encryption Authenticated Fast A AEGIS:	A
AEGIS-128L	(v1.1) Algorithm Encryption Authenticated Fast A AEGIS:	A
Encrypt-then-MAC		A
MAC-then-encrypt		L

Functions Hash

این بخش اطلاعات تکمیلی در مورد هش کردن و توابع مبتنی بر هش را ارائه می‌دهد.

Cases Use General for Functions Hash

جدول زیر توابع هش تأیید شده برای کاربردهای عمومی رمزنگاری، مانند امضاهای دیجیتالی، را فهرست می‌کند:

- توابع هش تأیید شده مقاومت بالایی در برابر برخورد (collision) دارند و برای برنامه‌های با امنیتی بالا مناسب هستند.
- برخی از این الگوریتم‌ها وقتی همراه با مدیریت صحیح کلیدهای رمزنگاری استفاده شوند، مقاومت بالایی در برابر حملات ارائه می‌دهند و بنابراین برای توابع KDF HMAC و RBG نیز تأیید شده‌اند.
- توابع هش با خروجی کمتر از ۲۵۴ بیت مقاومت کافی در برابر برخورد (collision) ندارند و نباید برای امضاهای دیجیتالی یا سایر کاربردهای که نیاز به مقاومت در برابر برخورد دارند، استفاده شوند. برای سایر کاربردها، ممکن است تنها برای سازگاری و اعتبارسنجی با سیستم‌های قدیمی استفاده شوند، اما نباید در طراحی‌های جدید به کار روند.

function Hash	Reference	Status	Restrictions
SHA3-512	202 FIPS	A	
SHA-512	180-4 FIPS	A	
SHA3-384	202 FIPS	A	
SHA-384	180-4 FIPS	A	
SHA3-256	202 FIPS	A	

function Hash	Reference	Status	Restrictions
SHA-512/256	180-4 FIPS	A	
SHA-256	180-4 FIPS	A	
SHAKE256	202 FIPS	A	
BLAKE2s	as fast smaller, simpler, BLAKE2: MD5	A	
BLAKE2b	as fast smaller, simpler, BLAKE2: MD5	A	
BLAKE3	fast function, one BLAKE3 everywhere	A	
SHA-224	180-4 FIPS	L	KDF, HMAC, for suitable Not signatures digital RBG,
SHA-512/224	180-4 FIPS	L	KDF, HMAC, for suitable Not signatures digital RBG,
SHA3-224	202 FIPS	L	KDF, HMAC, for suitable Not signatures digital RBG,
SHA-1	6194 RFC & 3174 RFC	L	KDF, HMAC, for suitable Not signatures digital RBG,
length) (any CRC		D	
MD4	1320 RFC	D	
MD5	1321 RFC	D	

Storage Password for Functions Hash

برای هش کردن امن گفرواژه‌ها، باید از توابع هش مخصوص استفاده شود. این الگوریتم‌های هش می‌تواند، با افزایش سختی محاسباتی شکستن گفرواژه، حملات جستجوی فراگیر (brute-force) و دی‌کشنری را کاهش می‌دهند.

KDF	Reference	Parameters Required	Status
argon2id	9106 RFC	1 = p MiB), (46 47104 m 1: = t	A
		1 = p MiB), (19 19456 m 2: = t	A
		1 = p MiB), (12 12288 m 3: = t	A
scrypt	7914 RFC	8 = r MiB), (128 2 ¹⁷ N 1: = p	A
		8 = r MiB), (64 2 ¹⁶ N 2: = p	A

KDF	Reference	Parameters Required	Status
		8 = r MiB), (32 2 ¹⁵ N 3: p	A
bcrypt	Future-Adaptable A Scheme Password	10 cost	A
PBKDF2-HMAC-SHA-512	FIPS ,800-132 SP NIST 180-4	000 210, iterations	A
PBKDF2-HMAC-SHA-256	FIPS ,800-132 SP NIST 180-4	000 600, iterations	A
PBKDF2-HMAC-SHA-1	FIPS ,800-132 SP NIST 180-4	000 300, 1, iterations	L

توابع استخراج کلید مبتنی بر گزرواژه (Password-Based Derivation Key Functions) تأیید شده می‌توانند برای ذخیره‌سازی گزرواژه استفاده شوند.

(KDFs) Functions Derivation Key

Functions Derivation Key General

KDF	Reference	Status
HKDF	5869 RFC	A
PRF 1.2 TLS	5248 RFC	L
KDFs MD5-based	1321 RFC	D
KDFs SHA-1-based	6194 RFC & 3174 RFC	D

Functions Derivation Key Password-based

KDF	Reference	Parameters Required	Status
argon2id	9106 RFC	1 = p MiB), (46 47104 m 1: = t	A
		1 = p MiB), (19 19456 m 2: = t	A
scrypt	7914 RFC	8 = r MiB), (128 2 ¹⁷ N 1: = p	A
		8 = r MiB), (64 2 ¹⁶ N 2: = p	A
		8 = r MiB), (32 2 ¹⁵ N 3: = p	A

KDF	Reference	Parameters Required	Status
PBKDF2-HMAC-SHA-512	FIPS ,800-132 SP NIST 180-4	000 210, 1 iterations	A
PBKDF2-HMAC-SHA-256	FIPS ,800-132 SP NIST 180-4	000 600, 1 iterations	A
PBKDF2-HMAC-SHA-1	FIPS ,800-132 SP NIST 180-4	000 300, 1, 1 iterations	L

Mechanisms Exchange Key

این بخش اطلاعات تکمیلی رمزنگاری کلیدی عمومی را ارائه می‌دهد.

Schemes KEX

برای همه طرح‌های تبادل کلیدی باید قدرت امنیتی ۱۱۲ بیت یا بیشتر تضمین شود و پیاده‌سازی آن‌ها باید از انتخاب پارامترهای موجود در جدول زیر پیروی کند.

Scheme	Domain Parameters	Forward Secrecy	Status
Finite Field Diffie-Hellman (FFDH)	$L \geq 3072$ & $N \geq 256$	Yes	A
Elliptic Curve Diffie-Hellman (ECDH)	$f \geq 256-383$	Yes	A
Encrypted key transport with RSA-PKCS#1 v1.5		No	D

که پارامترهای زیر عبارتند از:

- k اندازه کلیدی برای کلیدهای RSA است.
- L اندازه کلیدی عمومی و N اندازه کلیدی خصوصی برای رمزنگاری بر مبنای می‌دان محدود است.
- f محدوده اندازه‌های کلیدی برای ECC است.

هیچ پیاده‌سازی جدیدی نباید از طرحی استفاده کند که با استاندارد SP NIST 800-56A & B و SP NIST 800-77 مطابقت نداشته باشد. به طور مشخص، IKEv1 نباید در محیط‌های تولیدی استفاده شود.

groups Diffie-Hellman

گروه‌های زیر برای پیاده‌سازی Diffie-Hellman تأیید شده‌اند. قدرت‌های امنیتی در ضمیمه D استاندارد SP NIST 800-56A و بخش 1 نسخه 5 استاندارد SP NIST 800-57 مستند شده‌اند.

Group	Status
P-224, secp224r1	A
P-256, secp256r1	A
P-384, secp384r1	A
P-521, secp521r1	A
K-233, sect233k1	A
K-283, sect283k1	A
K-409, sect409k1	A
K-571, sect571k1	A
B-233, sect233r1	A
B-283, sect283r1	A
B-409, sect409r1	A
B-571, sect571r1	A
Curve448	A
Curve25519	A
MODP-2048	A
MODP-3072	A
MODP-4096	A
MODP-6144	A
MODP-8192	A
ffdhe2048	A
ffdhe3072	A
ffdhe4096	A
ffdhe6144	A
ffdhe8192	A

(MAC) Codes Authentication Message

کدهای احراز هویت پیام (MAC) سازه‌های رمزنگاری هستند که برای بررسی صحت و اصالت یک پیام استفاده می‌شوند. یک MAC یک پیام و یک کلید مخفی را به عنوان ورودی می‌گیرد و یک برچسب با اندازه ثابت (مقدار MAC) تولید می‌کند. MAC به طور گسترده در پروتکل‌های ارتباطی امن (مانند TLS/SSL) استفاده می‌شوند تا اطمینان حاصل شود که پیام‌های تبادل شده بین طرفین معتبر و دست‌نخورده هستند.

Algorithm MAC	Reference	Status
HMAC-SHA-256	198-1 FIPS & 2104 RFC	A
HMAC-SHA-384	198-1 FIPS & 2104 RFC	A
HMAC-SHA-512	198-1 FIPS & 2104 RFC	A
KMAC128	800-185 SP NIST	A
KMAC256	800-185 SP NIST	A
mode) (keyed_hash BLAKE3	everywhere fast function, one BLAKE3	A
AES-CMAC	800-38B SP NIST & 4493 RFC	A
AES-GMAC	800-38D SP NIST	A
Poly1305-AES	code message-authentication Poly1305-AES The	A
HMAC-SHA-1	198-1 FIPS & 2104 RFC	L
HMAC-MD5	1321 RFC	D

Signatures Digital

طرح‌های امضا باید از اندازه‌ها و پارامترهای کلیدی تأیید شده مطابق با 800-57 SP NIST بخش 1 استفاده کنند.

Algorithm Signature	Reference	Status
Ed448) (Ed25519, EdDSA	8032 RFC	A
Curve448) (Curve25519, XEdDSA	XEdDSA	A
P-521) P-384, (P-256, ECDSA	186-4 FIPS	A
RSA-RSSA-PSS	8017 RFC	A
v1.5 RSA-SSA-PKCS#1	8017 RFC	D
size) key (any DSA	186-4 FIPS	D

Standards Encryption Post-Quantum

پیاده‌سازی‌های رمزنگاری پساکوانتومی (PQC) باید از استانداردهای FIPS-203، FIPS-204 و FIPS-205 پیروی کنند. در حال حاضر، نمونه‌های کدمقاوم‌ی پیاده‌سازی‌های مرجع زیادی برای این استانداردها در دسترس نیست. برای جزئیات بیشتر، به اعلامیه NIST درباره سه استاندارد نه‌ای شده رمزنگاری پساکوانتومی (اوت ۲۰۲۴) مراجعه کنید.

روش پیش‌نه‌ادی mlkem768x25519 برای توافق کلیدی هیبریدی TLS پساکوانتومی توسط مرورگرهای اصلی مانند Firefox نسخه ۱۳۲ و Chrome نسخه ۱۳۱ پشتیبانی می‌شود. این روش

می‌تواند در محیط‌های آزمایشی رمزنگاری یا زمانی که در کتابخانه‌های تأیید شده توسط صنعت یا دولت در دسترس است، استفاده شود.

Recommendations D: Appendix

Introduction

در حین آماده‌سازی نسخه ۵.۰ استاندارد اعتراف بارسنجی امنیتی برنامه‌ها (ASVS)، مشخص شد که تعدادی از موارد موجود و همچنین موارد پیش‌نه‌ادی جدید وجود دارند که نباید به عنوان الزام در نسخه ۵.۰ گنجانده شوند. این موضوع یا به این دلیل بود که مطابق تعریف نسخه ۵.۰ در محدوده ASVS قرار نمی‌گرفتند، یا به این خاطر که هرچند ایده‌های خوبی بودند، اما امکان الزام‌آور کردن آنها وجود نداشت. برای اینکه همه‌ی این موارد به‌طور کامل از دست نروند، برخی از آنها در این پیوست ثبت شده‌اند.

mechanisms in-scope Recommended,

موارد زیر در محدوده ASVS قرار دارند. این موارد نباید الزام‌آور شوند، اما قویاً توصیه می‌شود که به عنوان بخشی از یک برنامه‌ی امن مورد توجه قرار گیرند.

- باید ابزاری برای نمایش میزان قوی یاضعیف بودن گفراژه در اختیاریار کاربران قرار داده شود تا به آنها در انتخاب یک گفراژه امن‌تر کمک کند.
- یک فایل security.txt عمومی در ریشه سایت یا پوشه well-known. برنامه ایجاد کنی‌د که به-وضوح یک لینک یا آدرس ایمیل برای تماس افراد با مال‌کین در مورد مسائل امنیتی را مشخص کند.
- اعتراف بارسنجی ورودی‌ها در سمت کاربر باید علاوه بر اعتراف بارسنجی در لایه سرویس مورد اعتراف اعمال شود، زیرا این کار فرصت خوبی برای شناسایی زمانی فراهم می‌کند که کسی تلاش کرده است با عبور از کنترل‌های سمت کاربر به برنامه حمله کند.
- جلوگیری از دسترس تصادفی و نمایش صفحات حساس در موتورهای جستجو با استفاده از فایل robots.txt، در پاسخ X-Robots-Tag یا تگ متای robots. HTML.
- هنگام استفاده از GraphQL، منطق احراز هویت را در لایه منطق کسب‌وکار پیاده‌سازی کنید، نه در لایه GraphQL یا resolver، تا نیازی به مدیریت احراز هویت در هر رابط جداگانه نباشد.

مجمع:

- More information including security.txt on the RFC

principles Security Software

موارد زیر قبلاً در ASVS ذکر شده بودند، اما در واقع الزام نیستند. در عوض، اینها اصولی هستند که هنگام پیاده‌سازی کنترل‌های امنیتی باید در نظر گرفته شوند و رعایت آنها منجر به کنترل‌های مقاوم‌تر می‌شود. این موارد شامل:

- کنترل‌های امنیتی باید مت‌مرکز، ساده (اقتصادی در طراحی)، قابلیت‌بارسنجی و قابل است‌فاده مجدد باشند. این روی‌کرد باید از ت‌کراری، ناقص یا ناکارآمد بودن کنترل‌ها جلوگیری کند.
- هرچ‌ا‌م‌کن‌است، به‌ج‌ای پی‌اده‌سازی کنترل‌ها از ابتدا، از پی‌اده‌سازی‌های قبلاً نوشته شده و به‌خوبی بررسی شده کنترل‌های امنیتی است‌فاده کنی‌د.
- به‌طور ای‌ده‌آل، باید از ی‌ک‌م‌کان‌ی‌زم کنترل دست‌رسی واحد برای دست‌رسی به داده‌ها و منابع محافظت شده است‌فاده شود. تمام درخ‌واست‌ها باید از طریق هم‌ی‌ن‌م‌کان‌ی‌زم واحد عبور کنند تا از مس‌یره‌ای جای‌گزین ناام‌ن‌ی‌ا‌ک‌پی و چ‌س‌ب‌اندن جلوگیری شود.
- کنترل دست‌رسی مبت‌نی بر وی‌ژگی یا ص‌فت، ی‌ک‌ال‌گوی پی‌شن‌ه‌ادی است که در آن که مج‌وز کاربر را برای ی‌ک‌وی‌ژگی یا مورد داده بررسی می‌کند، نه صرفاً نق‌ش‌او. با این حال، ت‌خ‌ص‌ص دست‌رسی‌ها همچ‌نان باید از طریق نق‌ش‌ها انجام شود.

processes Security Software

ت‌ع‌دادی از فرآی‌نده‌ای امنیتی وجود دارند که از ASVS نسخه ۵.۰ حذف شده‌اند اما همچ‌نان ای‌ده‌خوبی هستند. پروژه SAMM OWASP می‌تواند منبع مناسبی برای نحوه پی‌اده‌سازی مؤثر این فرآی‌نده‌ها باشد. مواردی که قبلاً در ASVS گن‌ج‌ان‌ده شده بودند، شامل:

- است‌فاده از چ‌رخه عمر ت‌وس‌عه نرم‌افزار امن را که امنیتی را در ت‌مامی مراح‌ل ت‌وس‌عه در نظر می‌گیری‌د، تأی‌د کنی‌د.
- است‌فاده از مدل‌سازی ت‌هدی‌د برای هر ت‌غ‌ی‌ر طراحی یا برنام‌ه‌ری‌زی اس‌پ‌ری‌نت را تأی‌د کنی‌د تا ت‌هدی‌ده‌ا‌شناس‌ای شوند، برنام‌ه‌ای برای مقاب‌له با آن‌ها تدوین شود، پاس‌خ‌های مناسب به ریس‌ک ت‌سه‌یل گردد و ت‌ست‌های امنیتی هدایت شوند.
- تأی‌د کنی‌د که تمام داس‌ت‌ان‌ه‌ای کاربری و وی‌ژگی‌ها شامل مح‌دودی‌های عمل‌کردی امنیتی باشند، مان‌ند: «به‌ع‌نوان ی‌ک کاربر، باید قادر باش‌م پ‌روف‌ای‌ل خود را مشاهده و وی‌رای‌ش کنم. نباید قادر باش‌م پ‌روف‌ای‌ل سایر کاربران را مشاهده یا وی‌رای‌ش کنم.
- اطم‌ی‌نان حاصل کنی‌د که ی‌ک‌چ‌کل‌ی‌ست برنام‌ه‌نوی‌سی امن، الزامات امنیتی، راه‌ن‌ما یا سی‌است امنیتی در دست‌رس تمام ت‌وس‌عه‌ده‌ندگان و ت‌ست‌کنندگان قرار دارد.
- اطم‌ی‌نان حاصل کنی‌د که ی‌ک فرآی‌ند مس‌ت‌مر برای بررسی وجود دارد تا ک‌د منبع برنام‌ه‌عاری از درب‌های پشتی، ک‌ده‌ای مخ‌رب (مثلاً ح‌ملات سالامی، بم‌ب‌های منطقی، بم‌ب‌های زم‌انی) و وی‌ژگی‌های مس‌ت‌ند‌سازی نشده یا مخ‌فی (مثلاً Easter egg، اب‌زارهای اش‌ک‌ال‌زدای ناام‌ن) باشد. رعایت این ب‌خ‌ش بدون دست‌رسی کامل به ک‌د منبع، شامل ک‌تاب‌خ‌انه‌های ش‌خ‌ص‌ث‌ال‌ث، ام‌کان‌پذیری‌ست و ب‌ناب‌رای‌ن اح‌تم‌الاً فقط برای برنام‌ه‌های مناسب است که به بالاترین سط‌وح امنیتی نیاز دارند.
- اطم‌ی‌نان حاصل کنی‌د که م‌کان‌ی‌زم‌های برای شناس‌ای و پاس‌خ به ت‌غ‌ی‌رات ناخ‌واس‌ت‌ه پی‌ک‌رب‌ندی در مح‌یط‌های مس‌ت‌قر شده وجود دارد. این م‌م‌کن‌است شامل است‌فاده از زیرساخت غی‌رقابل ت‌غ‌ی‌ر، بازاس‌ت‌قرارخ‌ودکار از ی‌ک پای‌ه امن، یا اب‌زارهای شناس‌ای ت‌غ‌ی‌ر که وضعیتی ف‌ع‌لی را با پی‌ک‌رب‌ندی‌های تأی‌د شده مقای‌سه می‌کنند، باشد.
- اطم‌ی‌نان حاصل کنی‌د که امن‌سازی پی‌ک‌رب‌ندی روی تمام مح‌صولات، ک‌تاب‌خ‌انه‌ها، فری‌م‌ورک‌ها و سروی‌س‌های ش‌خ‌ص‌ث‌ال‌ث طبق ت‌وص‌یه‌ای هر ی‌ک انجام شده باشد.

مراجع:

- Sheet Cheat Modeling Threat OWASP •
- modeling Threat OWASP •
- Project Model Maturity Assurance Software OWASP •
- SDL Microsoft •

Contributors - E Appendix

ما با س پاس گ زاری ازم شارکت افراد زیر که از زمان انتشار نسخه ۴.۰.۰ ASVS نظر داده اند یا درخواست های ادغام (Pull Request) باز کرده اند، قدردانی می کنیم.

اگر از وجود هرگونه خطای آگاه هستید یا ترجیح می دهید نام شما به شکل کلی متفاوت نمایش داده شود، لطفاً به ما اطلاع دهید.

Johan Sydseter (sydseter)	lfservin(servin luis	Oleksii IZUKA Dovy- Masahiro d-)maizuka(kov)oleksiidov(
James Sulinski (jsulinski)	ThunderSon(Saad Eli	kkshitisAndrew van der Stock)vanderaj(
M Rick (kingthorin)	Bankde(Eakasit Bankde	MichaelRaphael GargiulDunant)mgargiBlater(
Kohl Cesar (cesarkohl)	inaz0	Joerg David Bruen- Deather- ner age)JoergBrennert(dave(
John Carroll (yosignals)	jimfenton(Fenton Jim	Matteo Sebastien Pace gio-)M4tteoPria)SPoint42(
van Steven Baan der (vdbaan)	jeremychoi(Choi Bonghwan Jeremy	craig- Riccardo shony Sirigu)ricsirigu(

Tomasz Wrobel)tw2as(clallier94	)belalena(Dubeshko Alena)kwwall(Wall W. Kevin	Rafael mjang- Green cobalt)RafaelGreen1(Jordan Ingo Sher- Rauner man ingo-(jsherm- rauner fwdsec /)deleterepo
Dirk Wetter)drwetter(	)moshe-apiiro(Zioni Moshe	Patrick David Dwyer Clarke)coderp at visclarke-()au
Takaharu Ogasa)takaharuogasa(	)arkid15r(Yakovets Arkadii	Motoyasu Saburi motoyasu-()saburi
wet- certitude	timhemel	RL Thomas Thorn-Bandt ton)aspnetde()thornshadow99(
Roel Storms)roelstorms(	)commjoen(Willemsen Jeroen	anonym Kas ran 31 Sai- ful- lah)deFr0ggy(
Steve Springett)stevespringett(	)northdpole(Spyros	Hans Marx314 Her- rera)hansphp(
CarlosAllendes	)yruss972(Russ Yonah	Sander Luboš Mai- Bretschnei- jers der sanmai- bretik ()NL

Eva Sarafianou)esarafianou(van Steven Baan der)svdb-ncc(hi-unc1e Michael Gissing)scolytus(Väli Silvia)silviavali(Gareth Heyes)hackvertor(DanielPharos)AtlasHackert(Sage)SajjadPourali(Mark Burnett)m8urnett(BitnessWise)bitnesswise(	ataseren Aif4thah(Vacarella Michael starbuck3000(sb3k thatsjet(Anderson Jet jackgates73 appills willfarrell(Farrell will rbsec dschwarz91 damienbod(damienbod 	Steve Dominique ThomasRIGHETTO)Sc00bz(righettod(TonimiStefan Kisas- Stre- ondi ichs-)tkisason(hier streichsbaer(mario- Devdatta platt Akhawe)devd(Dave Jonny Wich- Schnittger ers)JonnySchnittger()davewichers(1songb lliidnur -)timurozkul(suvikaactinals)chaals(Alina Paul Vasil- Mc- jeva Cann)avasiljeva(misepaul(Benedik James Bauer Jar-)mastachiana(jamesjardine(Cyber- Tib3rius AppSec Cyber-()AppSec Jared Stefan Meit Seel- jmeit-(mann)fwdsec)sseelmann(
--	---	--

Brendan O'Connor)ussjoin(	)andrettv(Titov Andrei	Hans- markehack Petter Fjeld)atluxity(
Neil Madden)NeilMadden(	)mgeramb(Geramb Michael	Osama mackowski El- nag- gar ossie-()git
Balla Ravi)raviballa(	)hazanasec(Hazana	David Alexander Means Stein)dmean981ch4(
BaeSenseii)baesenseii(	)VincentDS(Schutter De Vincent	S Mitsuaki Bani Akiyama)sbani()mak1yama(
Christopher Loessl)hashier(	victorxm	Michal Veeresh Rada De-)michalradadly)drveresh(
MaknaSEO	darkzero2022	Liam Frank)LiamDelaere(nis)jedisct1(
Sulin Otto)ottosulin(	carllaw6885	Anders Richard Jo- Fritsch han)rfricz(Holme- ford)aholmis(
mesutgungor	)ScottHelme(Helme Scott	Carlo Suyash Reg- Sri- giani vas-)carloreggiani()suyash5053(
Mark Potter)markonweb(	)alamers(Lamers Arjan	Göran flo- Breivik blg)gobrtg(

Guillaume Déflache guillaume-()d iloving	)toufik-airane(Airane Toufik	Keith Sinner HoodleSoftwareSinner()securingdev(Joubin yu Jab- fu- bari jioka)joubin()fujiokayu(Sidney Gabriel RibeiroMar-)srjsoftwre(Gby56(bedirhan muralitRonnie Flathers)ropnop()mal33(Malte MazeOfMindights Falk)andifalk()averell23(Hahn Daniel borislavRobin c Wood)digininja()jandockx(Dockx Jan vipinsaiipr434shukumar3)BenjaminHae(Häublein Benjamin unknownAli user- Ra- from mazan TAŞDE- LEN)alitasdln()josh-hemphill(Josh Tim AvID Würteleavidouglen()SEctim(
execjosh)execjosh(Drew Schulz)drschulz(Philippe Ryck De)philippederyck(Javi)javixeneize(miro2ns Nat Sakimura)sakimura(Pedro Escaleira)oEscal(		

SheHacksPurple)shehackspurple(	fcerullo-cycubix	Hector Irene Eryx Mich- Pare- lin des)irene221b(Ca- ma- cho)heryxpc(
Y-M Jonah TG-()Techie	)bahroos(Bahroos Dhiraj	Jef IzmaDoesItbeta Mei- jvis)jefmeijvis(
Abdessamad TEMMAR)TmmmmmmR(james-t james-()bitherder	sectroyer)aramhovsepyan(Hovsepyan Aram	Soh regoravalaz SatoH)sohsatoH(JaimeGouldGintolaSan
iwatachan)ishowta(	)VinodAnandan(Anandan Vinod	Kevin paul- Kien williamson-)KevinKwanOp
endergzr	)Rado0z(Alshamamri Radhwan	Grant Cure53 Ongers)cure53()rewtd(
AliR2Linux	)GangGreenTemperTatum(Dawson Ads	Williamgabe Reyor)gcrow()BillReyor(
mascotter	luissaiz	Suren Piotr ManukGaliżniewicz vx-()pglizniewicz()sec
Tadeusz Wa- chowski)tadeuszwachowski(	)andsec(Nate aka Nasir	settantaJette Haulin)LarsH(

Terence	JasmineScholz	Arun Yusuf
Eden		Sivadasan
)edent(		)teavani)(usuffigur(
Troy	)tprynn(Prynn Tanner	Nick raoul361
Marshall		K.
)troymarshall(		)nickific(
Azeem	)avioli(Stamatov Evo	Tim Gavin
Ilyas		Pot- Ray
)TheAxZim(		ter)GavinRay97(
		)timpotter87(
monis	)MarcinHoppe(Hoppe Marcin	GramblyJfrdan
)demideus(		)ramshazPike
		)computersarebad
Jason	)benbhall(Hall Ben	JamesPolymyGHackLabs
Rogers		)jamesly)(w1)(chacklabs(
jason-(		
)invision		
Alex	)filipvanlaenen(Laenen van Filip	jeurgenGraoMelo
Gaynor		
)alex(		
Andreas	)TomTervoort(Tervoort Tom	old Marco
Kurtz		man Schnüriger
)ay-kay(		)deveray)(marcortw(
stiiin	)teaminfoseclearn(infoseclearn	hljupkij Noe
		)nmarher(
Lyz	)mrtnrdl(Riedel Martin	KIM Barbara
)lyz-code(		Jae- Schachner
		suck)bschach(
		)tcaesvk(
René	carhackpils	Tyler Hugo
Reuter		)tyler2c)(kasousa(
)AresSec(		
Wouter	)markderijkinfosec(Rijk de Mark	Ramin Philip
Bloeyaert		)picohub(D.
)Someniak(		Turner
		)philipdturner(

Will
Chatham
)willc(
